Supplementary Information

A flexible modelling framework for estimating thermal sensitivity across life

Daniel W.A. Noble*[1], Pieter A. Arnold[1], Patrice Pottier[2]

[1] *Division of Ecology and Evolution, Research School of Biology, Australian National University, Canberra, Australia*

[2] *Department of Biological and Environmental Sciences, University of Gothenburg, Gothenburg, Sweden*

2026-05-15

Table of contents

*Corresponding author: daniel.noble@anu.edu.au

```
# Setup chunk must NOT be cached - caching skips the package-loading side
# effects on subsequent renders, so library() calls in pacman::p_load() never
# run and downstream chunks fail with "could not find function ...".
pacman::p_load(here, dplyr, tidyr, tibble, purrr, ggplot2, brms, posterior, tidybayes, tinytabl

# Force tinytable to register its LaTeX preamble dependencies (tabularray,
# siunitx, etc.) on every render. Tinytable injects these via
# knit_meta_add() when tt() is called, but if all tt() chunks are cached,
# the side-effect doesn't fire and PDF builds fail with "Environment tblr
# undefined". Calling tt() here (in this cache: false chunk) guarantees
# the dependencies are always registered.
invisible(tinytable::tt(data.frame(x = 1)))

set.seed(123)

# Cache directory for brms fits (per CLAUDE.md: save fits manually as .rds).
models_dir <- here::here("output", "models")
if (!dir.exists(models_dir)) dir.create(models_dir, recursive = TRUE)
```

1 Introduction

This supplement walks through the joint Bayesian 4-parameter logistic (4PL) framework from the main manuscript using the **bayesTLS** R package, which ships with this project. It provides functions to fit the 4PL, extract the classical TLS quantities (z, $CT_{max_{1hr}}$, T_{crit}) with full posterior uncertainty, and use them to predict heat-injury accumulation and survival fraction under arbitrary temperature time-series, optionally with a Sharpe-Schoolfield repair kernel.

Install once from GitHub, then load:

```
# install.packages("remotes")   # if not already installed
remotes::install_github("daniel1noble/bayesTLS")
```

```
library(bayesTLS)
```

The functions chain together as a single pipeline:

1. **Standardise** raw experimental data to a common column schema (`temp`, `duration`, `n_total`, `n_surv`, …).
2. **Fit** the joint Bayesian 4PL with the built-in `beta_binomial(link = "identity")` family and asymptote-bounded reparameterisation.
3. **Extract** z, $CT_{max_{1hr}}$, and T_{crit} — all with full posterior uncertainty.
4. **Predict** survival and heat-injury accumulation under fluctuating field temperature traces, with optional repair.
5. **Plot** each step with shared theming so figures across the workflow read consistently.

Every exported function is documented with roxygen2 (`?fit_4pl`, etc.) and the package ships with `testthat` tests — fast unit tests plus a gated set of `brms`-fitting integration tests that assert recovery of known simulation truths. `R CMD check` runs cleanly. The remainder of the supplement first walks the pipeline through a controlled simulation where the truth is known (§ A Tutorial with Simulations), then applies the same machinery end-to-end to a real lethal-TDT dataset on brown shrimp (§ Case Study 1: Shrimp data).

1.1 *What each function does*

Table S1 lists every exported function in the library, grouped by pipeline stage. Each function is single-purpose; the full roxygen2 documentation (with runnable `@examples`) lives in the corresponding R file.

```
tdt_function_tour <- tibble::tribble(
  ~Stage,            ~Function,                          ~Purpose,
  "Data",            "standardize_data()",               "Rename raw columns to project sch
  "Model spec",      "make_4pl_priors()",                "Weakly informative priors for the
  "Model spec",      "make_4pl_formula()",               "brms non-linear formula; identity
  "Fit",             "fit_4pl()",                        "Joint Bayesian 4PL fit; returns a
  "Diagnostics",     "diagnose_tdt_fit()",               "Rhat / ESS / divergences / treede
  "Diagnostics",     "tdt_parameter_table()",            "Posterior summary on the natural
```

```
  "Predictions",      "predict_survival_curves()",          "Survival curves on a temp x durati
  "Predictions",      "derive_tdt_landscape()",             "Dense 2-D survival surface for a l
  "TDT quantities",   "derive_tdt_curve()",                 "Time to the threshold survival acr
  "TDT quantities",   "derive_temperature_for_duration()", "Temperature at threshold survival
  "TDT quantities",   "derive_tdt_parameters()",            "Per-draw log-linear regression of
  "TDT quantities",   "extract_tdt()",                      "Bundled wrapper: z and CTmax alway
  "Heat injury",      "make_temperature_scenarios()",       "Three reference traces (flat / sin
  "Heat injury",      "planted_dose_from_trace()",          "Analytical HI under known z, CTmax
  "Heat injury",      "predict_heat_injury()",              "Posterior HI and survival under a
  "Heat injury",      "repair_rate_schoolfield()",          "Sharpe-Schoolfield repair TPC (Kel
  "Plotting",         "plot_survival_curves()",             "Posterior curves with observed pro
  "Plotting",         "plot_tdt_curve()",                   "LT_x curve on a log-time axis (cla
  "Plotting",         "plot_tdt_landscape()",               "Survival heatmap with contour over
  "Plotting",         "plot_temperature_density()",         "Posterior density for CTmax / T_cr
  "Plotting",         "plot_temperature_scenarios()",       "Three traces stacked vertically."
  "Plotting",         "plot_heat_injury()",                 "HI and predicted survival trajecto
  "Plotting",         "plot_repair_rate()",                 "Sharpe-Schoolfield TPC curve."
)

tinytable::tt(tdt_function_tour, escape = TRUE)
```

2 A Tutorial with Simulations

This section is a guided tutorial using simulated data. We show, step by step, how the Bayesian hierarchical 4PL — under both binomial sampling and the more realistic beta-binomial (overdispersed) case – can recover thermal sensitivity (z) and $CT_{max_{1hr}}$ from the single joint model, yet provide substantial downstream benefits in terms of propagating uncertainty and giving more flexibility. We then complement the simulation with a few case studies which walk through real data that is presented within the main manuscript in other sections.

2.0.1 *Simulating Data*

2.0.1.1 Step 1 — Setting up parameters

We choose values that might be typical of a thermal mortality assay. The four 4PL parameters are the lower and upper asymptotes (ℓ, u), the slope on the $\log_{10}$-time axis (k), and a midpoint that varies linearly with temperature: $\text{mid}(T) = \beta_0 + \beta_1(T - \bar{T})$. Each value below has a clear biological meaning.

```
true_params <- list(
  ell      = 0.03,   # 3% of individuals survive even at the longest exposures
  u        = 0.97,   # 97% are alive at very short exposures
  k        = 8,      # steepness of the survival drop on the log10(t) axis
  m_beta0  = 1.5,    # midpoint at the grand-mean temperature: ~31.6 min
  m_beta1  = -0.15,  # midpoint drops 0.15 log10-min per °C; z = -1/beta1   6.7 °C
```

Table S1: Functions exported by the TDT analysis library, grouped by pipeline stage. Each function is documented with roxygen2 in its R file; runnable examples accompany every function.

Stage	Function	Purpose
Data	standardize_data()	Rename raw columns to project schema; attach metada
Model spec	make_4pl_priors()	Weakly informative priors for the disjoint-bounds 4PL.
Model spec	make_4pl_formula()	brms non-linear formula; identity link, all four sub-para
Fit	fit_4pl()	Joint Bayesian 4PL fit; returns a 'bayes_tls' object (us
Diagnostics	diagnose_tdt_fit()	Rhat / ESS / divergences / treedepth, with pass-fail fl
Diagnostics	tdt_parameter_table()	Posterior summary on the natural scale (low, up, k, mi
Predictions	predict_survival_curves()	Survival curves on a temp x duration grid with credibl
Predictions	derive_tdt_landscape()	Dense 2-D survival surface for a heatmap.
TDT quantities	derive_tdt_curve()	Time to the threshold survival across temperature. De
TDT quantities	derive_temperature_for_duration()	Temperature at threshold survival for a fixed exposure
TDT quantities	derive_tdt_parameters()	Per-draw log-linear regression of LT_x on temperature
TDT quantities	extract_tdt()	Bundled wrapper: z and CTmax always; T_crit opt-in
Heat injury	make_temperature_scenarios()	Three reference traces (flat / single spike / multi-spike
Heat injury	planted_dose_from_trace()	Analytical HI under known z, CTmax, T_c (validation
Heat injury	predict_heat_injury()	Posterior HI and survival under a temperature trace.
Heat injury	repair_rate_schoolfield()	Sharpe-Schoolfield repair TPC (Kelvin internally).
Plotting	plot_survival_curves()	Posterior curves with observed proportion overlay.
Plotting	plot_tdt_curve()	LT_x curve on a log-time axis (classical TDT view).
Plotting	plot_tdt_landscape()	Survival heatmap with contour overlay.
Plotting	plot_temperature_density()	Posterior density for CTmax / T_crit with a 95-percen
Plotting	plot_temperature_scenarios()	Three traces stacked vertically.
Plotting	plot_heat_injury()	HI and predicted survival trajectories with CrI bands.
Plotting	plot_repair_rate()	Sharpe-Schoolfield TPC curve.

```
  T_bar    = 34      # grand-mean temperature
)
```

The implied thermal sensitivity is $z = -1/\beta_1 \approx 6.7°C$ — the temperature change that shifts LT50 by a factor of 10. From the same parameters we can also compute the simulation truth for the uncentred TDT intercept $\alpha = \beta_0 + \frac{1}{k}\log\frac{u-0.5}{0.5-\ell} - \beta_1\bar{T}$ and the critical thermal limit at 1 hour $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — both approaches below will be checked against these known values.

Recall that we can calculate $ $, z and $CT_{max_{1hr}}$ from the true parameters using the equations in the main manuscript. We'll do that now so we have a comparator for later.

```
true_z      <- -1 / true_params$m_beta1
true_alpha  <- true_params$m_beta0 +
  (1 / true_params$k) *
  log((true_params$u - 0.5) / (0.5 - true_params$ell)) -
  true_params$m_beta1 * true_params$T_bar
true_CTmax  <- (log10(60) - true_alpha) / true_params$m_beta1

# T_crit under the rate-multiplier definition (see §[Deriving z, CTmax, and
# T_crit from the posterior]). For each posterior draw, T_crit is computed as
# CTmax_1hr + z * log10(r*/100) where r* is sampled uniformly on log10
# across the default range c(0.1, 1) % HI per hour. The MEDIAN of that
# distribution lies at log10(r*/100) = -2.5 (the geometric mean of the
# range), so:
true_T_crit <- true_CTmax - 2.5 * true_z
```

2.0.1.2 Step 2 — Simulated study design

We mimic a textbook TDT experiment: a grid of five assay temperatures crossed with six log-spaced exposure durations, repeated 30 times per cell, with 30 individuals tested per replicate. That gives 900 trials and 27,000 individuals — large enough to recover the parameters with negligible uncertainty, but small enough to fit in a few minutes. Of course, in practice we will not have this much data as will be seen in the case studies, but simulations are a good starting point to understand the models being fit and the conversions.

```
temps     <- c(30, 32, 34, 36, 38)            # 5 assay temperatures (°C)
durations <- c(1, 5, 15, 45, 135, 405)        # 6 durations (minutes, log-spaced)
n_rep     <- 30                               # 30 replicates per cell
n_per_rep <- 30                               # 30 individuals per replicate

design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
```

```
    log10_t = log10(t),
    T_c     = T - true_params$T_bar,                          # mean-centred T
    mid     = true_params$m_beta0 + true_params$m_beta1 * T_c,  # mid(T)
    p_true  = true_params$ell +
      (true_params$u - true_params$ell) /
      (1 + exp(true_params$k * (log10_t - mid))),
    n = n_per_rep
  )
```

2.0.1.3 Step 3 — Look at the truth before simulating

Before generating noisy data, let's plot the *true* survival surface (Figure S1) so we know what the model is being asked to recover. Each line is the 4PL at one assay temperature, plotted against $\log_{10}$ exposure time.

```r
truth_grid <- tidyr::expand_grid(
  T = temps,
  t = 10 ^ seq(log10(0.5), log10(1000), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params$T_bar,
    mid     = true_params$m_beta0 + true_params$m_beta1 * T_c,
    p_true  = true_params$ell +
      (true_params$u - true_params$ell) /
      (1 + exp(true_params$k * (log10_t - mid)))
  )

ggplot2::ggplot(truth_grid,
                ggplot2::aes(log10_t, p_true, colour = factor(T))) +
  ggplot2::geom_hline(yintercept = 0.5, linetype = "dashed", colour = "grey50") +
  ggplot2::geom_line(linewidth = 0.8) +
  ggplot2::labs(
    x = expression(log[10]~exposure~time~(min)),
    y = "True survival probability",
    colour = "Temperature (°C)"
  ) +
  ggplot2::theme_minimal()
```

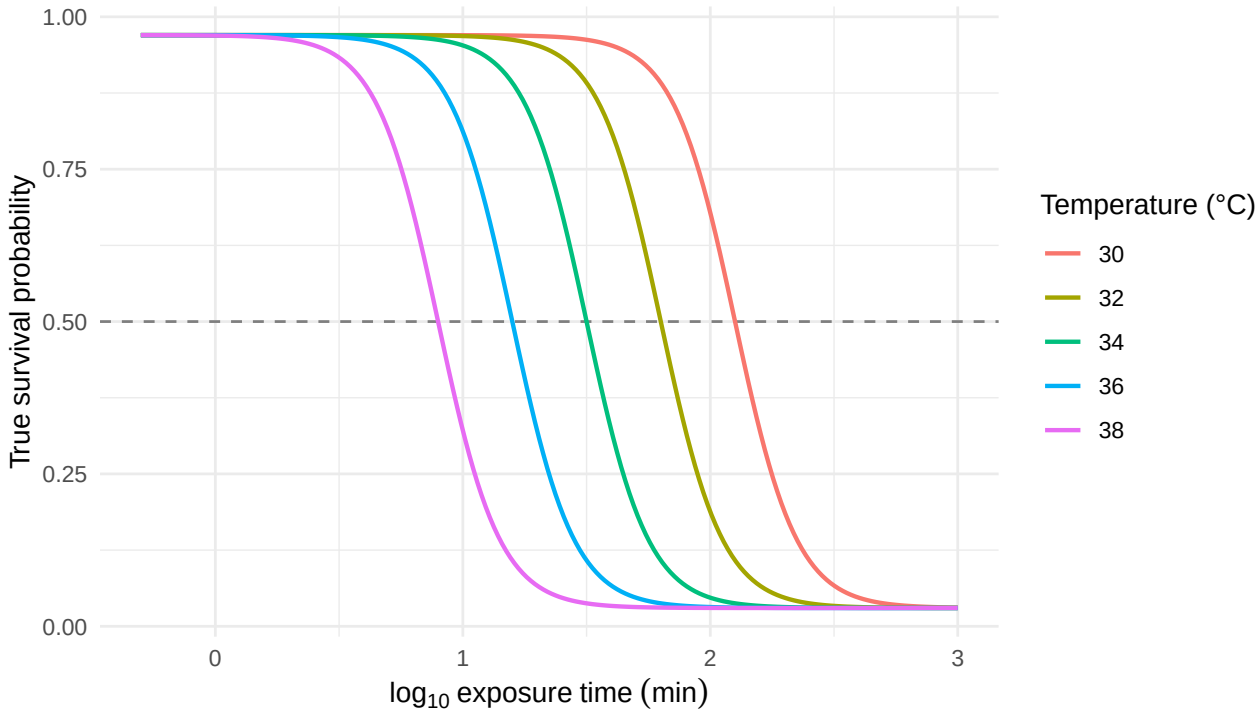


Figure S1: The known 4PL dose-response surface used to generate the simulated data, evaluated on a dense time grid at each of the five assay temperatures. The horizontal line marks 50% survival; where each curve crosses it gives the true LT50 at that temperature.

Notice the curves shift left as temperature rises: hotter temperatures kill faster, so the LT50 is reached at shorter durations. That horizontal shift in midpoints is exactly what β_1 encodes.

2.0.1.4 Step 4 — Generate noisy data: binomial and beta-binomial

Real survival counts have at least two flavours of noise. **Binomial** sampling is the textbook case: each replicate's survival count is a draw from Binomial(n, p) with p from the 4PL surface. **Beta-binomial** sampling adds *overdispersion* — replicate-to-replicate variability in p that the binomial cannot capture, e.g., from cohort effects or unmodelled heterogeneity. Real TDT data almost always show overdispersion so in many cases the beta binomial will be a better fit.

```
# (a) Binomial: y ~ Binomial(n, p_true)
sim_bin <- design |>
  dplyr::mutate(y = rbinom(dplyr::n(), size = n, prob = p_true))

# (b) Beta-binomial: replicate-level p drawn from Beta(p*phi, (1-p)*phi)
phi <- 5  # smaller phi = more overdispersion
sim_bb <- design |>
  dplyr::mutate(
    p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
    y      = rbinom(dplyr::n(), size = n, prob = p_draw)
  )
```

2.0.2 *The classical two-stage TDT pipeline*

We first run the simulated data through the **classical two-stage TDT pipeline** (Ørsted *et al.* 2022; Rezende *et al.* 2014), which is dominant in the thermal-tolerance literature.

1. **Stage 1.** At each assay temperature, fit a logistic dose-response curve to the count data and read off a point estimate of $\log_{10} \mathrm{LT50}_T$.
2. **Stage 2.** Regress those Stage-1 point estimates on assay temperature with ordinary least squares, then derive $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ from the fitted line.

2.0.2.1 Stage 1 — Dose-response curves at each temperature

We fit a binomial GLM with a logit link separately at each assay temperature. This is the most-common Stage-1 specification in the TDT literature: a two-parameter logistic with asymptotes constrained to 0% and 100%. The midpoint $\log_{10} \mathrm{LT50}_T = -\beta_0/\beta_1$ falls out of the GLM's two coefficients.

```
fit_stage1 <- function(sim_data) {
  sim_data |>
    dplyr::group_by(T) |>
    dplyr::group_modify(function(d, key) {
      f  <- glm(cbind(y, n - y) ~ log10_t, data = d, family = binomial())
      co <- coef(f)
      tibble::tibble(log10_lt50 = as.numeric(-co[1] / co[2]))
```

Table S2: Per-temperature Stage-1 estimates of $\log_{10}$ LT50 from a binomial GLM with logit link, fit separately at each of the five assay temperatures. Truth values are the simulated midpoints (which coincide with $\log_{10}$ LT50 here because the simulation asymptotes are near 0 and 1).

T (°C)	Truth log10(LT50)	Binomial log10(LT50)	Beta-binomial log10(LT50)
30	2.1	2.067	2.092
32	1.8	1.794	1.791
34	1.5	1.491	1.498
36	1.2	1.193	1.197
38	0.9	0.921	0.939

```
    }) |>
    dplyr::ungroup()
}

stage1_bin <- fit_stage1(sim_bin)
stage1_bb  <- fit_stage1(sim_bb)
```

```
stage1_table <- dplyr::full_join(
  stage1_bin |> dplyr::rename(bin_log10_lt50 = log10_lt50),
  stage1_bb  |> dplyr::rename(bb_log10_lt50  = log10_lt50),
  by = "T"
) |>
  dplyr::mutate(
    truth_log10_lt50 = true_params$m_beta0 +
      true_params$m_beta1 * (T - true_params$T_bar)
  ) |>
  dplyr::select(T, truth_log10_lt50, bin_log10_lt50, bb_log10_lt50) |>
  dplyr::rename(
    `T (°C)`             = T,
    `Truth log10(LT50)`  = truth_log10_lt50,
    `Binomial log10(LT50)` = bin_log10_lt50,
    `Beta-binomial log10(LT50)` = bb_log10_lt50
  )

tinytable::tt(stage1_table, digits = 3, escape = TRUE)
```

With the simulation's true asymptotes near 0 and 1, the 2PL's forced-asymptote constraint introduces negligible bias here, so each Stage-1 estimate sits close to the simulation truth.

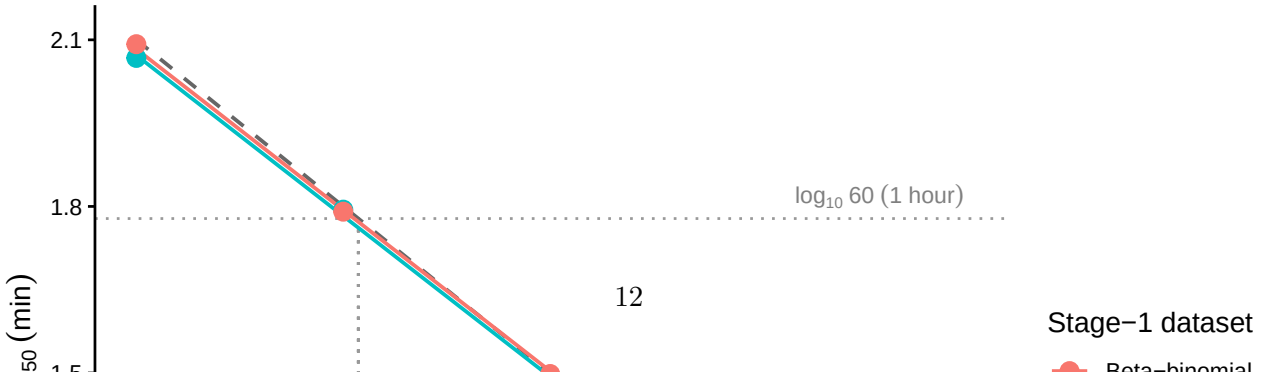
2.0.2.2 Stage 2 — log-linear TDT regression

Stage 2 regresses Stage-1 $\log_{10}$ LT50 on assay temperature with ordinary least squares. The classical TDT quantities — $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — fall out of the fitted line.

```r
fit_stage2 <- function(stage1) {
  fit2 <- lm(log10_lt50 ~ T, data = stage1)
  co   <- coef(fit2)

  tibble::tibble(
    quantity = c("z (°C)", "CTmax at 1 hr (°C)"),
    est      = c(as.numeric(-1 / co["T"]),
                 as.numeric((log10(60) - co["(Intercept)"]) / co["T"]))
  )
}

ts_bin <- fit_stage2(stage1_bin)
ts_bb  <- fit_stage2(stage1_bb)
```

Figure S2 visualises the Stage-2 fit. Each point is one Stage-1 $\log_{10}$ LT50 estimate from Table S2; the solid line is the OLS regression through them. The slope is β_1, from which $z = -1/\beta_1$ — i.e., the temperature increase that causes LT50 to change by a factor of 10. The line's height at $\log_{10} 60 \approx 1.78$ (1 hour) is at the temperature equal to $CT_{max_{1hr}}$ — the vertical dotted line drops from this intersection straight to the temperature axis to mark it. The dashed grey line is the simulation truth.

```r
stage1_combined <- dplyr::bind_rows(
  stage1_bin |> dplyr::mutate(dataset = "Binomial"),
  stage1_bb  |> dplyr::mutate(dataset = "Beta-binomial")
)

truth_line <- tibble::tibble(
  T = seq(min(temps), max(temps), length.out = 100)
) |>
  dplyr::mutate(
    log10_lt50 = true_params$m_beta0 +
      true_params$m_beta1 * (T - true_params$T_bar)
  )

ggplot2::ggplot(stage1_combined,
                ggplot2::aes(x = T, y = log10_lt50, colour = dataset)) +
  ggplot2::geom_hline(yintercept = log10(60),
                      linetype = "dotted", colour = "grey60") +
  ggplot2::geom_segment(
    ggplot2::aes(x = true_CTmax, xend = true_CTmax,
                 y = log10(60), yend = -Inf),
    inherit.aes = FALSE,
    linetype = "dotted", colour = "grey60"
  ) +
  ggplot2::geom_line(data = truth_line,
                     ggplot2::aes(x = T, y = log10_lt50),
                     inherit.aes = FALSE,
                     linetype = "dashed", colour = "grey40",
                     linewidth = 0.7) +
  ggplot2::geom_smooth(method = "lm", se = FALSE, linewidth = 0.7) +
  ggplot2::geom_point(size = 3) +
  ggplot2::annotate("text", x = max(temps), y = log10(60),
                    label = "log[10]~60~(1*' hour')", parse = TRUE,
                    hjust = 1, vjust = -0.4, colour = "grey50", size = 3) +
  ggplot2::annotate("text", x = true_CTmax, y = -Inf,
                    label = "CT[max[1*hr]]", parse = TRUE,
                    hjust = -0.15, vjust = -0.7, colour = "grey50", size = 3) +
  ggplot2::labs(
    x = "Assay temperature (°C)",
    y = expression(log[10]~LT[50]~(min)),
    colour = "Stage-1 dataset"
  ) +
  ggplot2::theme_classic()
```



This is the entire two-stage workflow. The point estimates that come out — z and $CT_{max_{1hr}}$ — are what the field reports. We carry them forward (`ts_bin` and `ts_bb`) for the side-by-side comparison once we have the joint Bayesian results.

> **!** Important
>
> Two-stage steps often ignore uncertainty in estimates in both fitting and error propagation. Uncertainty in estimates can be propagated using the SE's and the delta method, but only at each individual stage.

2.0.3 *A joint Bayesian 4PL*

Instead of fitting a separate logistic at each assay temperature and then regressing the LT50s, the joint fit estimates every parameter — ℓ, u, k, β_0 and β_1 — in a single hierarchical model on the raw counts. We now use the same simulated data to derive the z and $CT_{max_{1hr}}$ from the posterior, in one model rather than two.

2.0.3.1 Specifying the 4PL in `brms`

The model has two pieces. The first is the 4PL itself — survival probability as a function of $\log_{10}$ exposure time:

$$p_{ij} \;=\; \ell \;+\; \frac{u - \ell}{1 + \exp\!\big(k\,(\log_{10} t_{ij} - \mathrm{mid}_{ij})\big)}, \tag{S1}$$

where ℓ and u are the lower and upper asymptotes, k is the slope on the $\log_{10} t$ axis, and mid_{ij} is the value of $\log_{10} t$ at which the curve crosses the midpoint between ℓ and u. The second piece lets the midpoint vary linearly with mean-centred temperature:

$$\mathrm{mid}_{ij} \;=\; \beta_0 + \beta_1\,(T_{ij} - \bar{T}), \tag{S2}$$

so that β_1 encodes how fast the dose-response curve shifts on the time axis as assay temperature changes; $\bar{T}$ is the grand-mean temperature, included so that β_0 is interpretable as the midpoint at the average assay temperature.

The function library exposes this model as a single call: `fit_4pl()` constructs the brms `bf()` formula, the default weakly-informative priors, and the sampling controls, and returns a workflow object containing the fit. Internally the formula uses the disjoint-bounds reparameterisation described in §[Loading the function library] — `low = 0.001 + \text{inv\_logit}(\text{lowraw}) \cdot 0.498` and `up = 0.501 + \text{inv\_logit}(\text{upraw}) \cdot 0.498` — so the 4PL output is guaranteed to lie in $(0, 1)$ regardless of where MCMC takes the unconstrained `lowraw`, `upraw`, and `logk` parameters. The default also puts a `temp_c` slope on every 4PL sub-parameter; when temperature has no real effect on a given parameter, the slope shrinks toward zero under the prior. We inspect the brms formula `fit_4pl()` builds with:

```
make_4pl_formula()
```

```
n_surv | trials(n_total) ~ (0.001 + inv_logit(lowraw) * 0.498) + ((0.501 + inv_logit(upraw) * (
lowraw ~ temp_c
upraw ~ temp_c
logk ~ temp_c
mid ~ temp_c
```

> **!** Important
>
> The default model puts a `temp_c` slope on all four 4PL sub-parameters and includes no random effects. Random intercepts on `mid` can be added by passing `random_effects = c("Date", "Tank", ...)` to `fit_4pl()`. The simulation here has no batch structure so we leave it off; the shrimp case study below uses `random_effects = c("Date", "Tank")`.

2.0.3.2 Standardising and fitting

The first step is to pass the simulated data through `standardize_data()` so the columns match the schema the rest of the library expects (`temp`, `duration`, `logd`, `temp_c`, `n_total`, `n_surv`).

```
std_bin <- standardize_data(sim_bin,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")
std_bb  <- standardize_data(sim_bb,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")
```

We now call `fit_4pl()` twice — once with the built-in `binomial(link = "identity")` family for the non-overdispersed simulation, and once with the default `beta_binomial(link = "identity")` for the overdispersed case. Caching is handled by brms's `file = ...` mechanism, so subsequent renders reload the cached fits unless the formula or data changes.

```
# file_refit = "never": once the .rds exists, always reuse it. brms's
# default ("on_change") over-triggers refits on these simulated-data fits
# because it picks up subtle differences in tibble attributes between
# renders. Tutorial sims are stable - to force a refit, delete the .rds.
wf_bin <- fit_4pl(std_bin,
                  family = binomial(link = "identity"),
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file       = file.path(models_dir, "sim_4pl_binomial_v2"),
                  file_refit = "never")

wf_bb  <- fit_4pl(std_bb,
```

```
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file       = file.path(models_dir, "sim_4pl_betabinomial_v2"),
                  file_refit = "never")

# Convenience: extract the brmsfit object for downstream brms helpers.
fit_bin <- wf_bin$fit
fit_bb  <- wf_bb$fit
```

We can also get an idea of how much variation our model explains that considers all sources of relevant variation.

```
# Obtain the Bayes R2 for each model
brms::bayes_R2(fit_bin)
```

```
    Estimate   Est.Error      Q2.5     Q97.5
R2 0.9885111 0.000117092 0.9882313 0.9886848
```

```
brms::bayes_R2(fit_bb)
```

```
    Estimate   Est.Error      Q2.5     Q97.5
R2 0.9278098 0.001148178 0.9251855 0.9296789
```

2.0.3.3 Checking parameter recovery

We can now check how well our fitted model recovered the true parameters that generated the data. The most direct check is to compare posterior medians (with 95% credible intervals) to the known truth (Table S3). We have big sample sizes here so our values should be pretty close to our true values.

```
post_summary <- function(fit) {
  d <- posterior::as_draws_df(fit)
  # Transform the raw posterior columns back to the natural 4PL scale.
  # See compute_4pl_bounds(0, 1): low  (0.001, 0.499), up  (0.501, 0.999).
  draws <- tibble::tibble(
    ell               = 0.001 + plogis(d$b_lowraw_Intercept) * 0.498,
    u                 = 0.501 + plogis(d$b_upraw_Intercept)  * 0.498,
    k                 = exp(d$b_logk_Intercept),
    `mid (intercept)` = d$b_mid_Intercept,
    `mid (slope)`     = d$b_mid_temp_c
  )
  purrr::map_dfr(names(draws), \(p) tibble::tibble(
    param  = p,
    median = stats::median(draws[[p]]),
    lower  = stats::quantile(draws[[p]], 0.025, names = FALSE),
    upper  = stats::quantile(draws[[p]], 0.975, names = FALSE)
```

Table S3: Posterior medians and 95% credible intervals for the four 4PL parameters and the temperature slope, compared to the known truth used to generate the simulated data. Recovery is judged successful when the credible interval covers the truth.

Parameter	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
ell	0.03	0.029	0.0245	0.0335	0.0239	0.0141	0.0346
u	0.97	0.967	0.9633	0.9709	0.9701	0.962	0.9772
k	8	8.316	7.8925	8.7709	7.3271	6.5893	8.176
mid (intercept)	1.5	1.501	1.4927	1.5099	1.5176	1.4969	1.5378
mid (slope)	-0.15	-0.15	-0.1525	-0.1464	-0.1496	-0.157	-0.1423

```
  ))
}

truth <- tibble::tibble(
  param = c("ell", "u", "k", "mid (intercept)", "mid (slope)"),
  truth = c(true_params$ell, true_params$u, true_params$k,
            true_params$m_beta0, true_params$m_beta1)
)

recovery <- truth |>
  dplyr::left_join(post_summary(fit_bin) |>
                     dplyr::rename_with(~ paste0("bin_", .x), -param),
                   by = "param") |>
  dplyr::left_join(post_summary(fit_bb) |>
                     dplyr::rename_with(~ paste0("bb_", .x), -param),
                   by = "param") |>
  dplyr::rename(
    Parameter        = param,
    Truth            = truth,
    `Bin. median`    = bin_median,
    `Bin. lower`     = bin_lower,
    `Bin. upper`     = bin_upper,
    `BB median`      = bb_median,
    `BB lower`       = bb_lower,
    `BB upper`       = bb_upper
  )

tinytable::tt(recovery, digits = 3, escape = TRUE)
```

We can also overlay the posterior fitted curves on the simulated data (Figure S3). If the model has recovered the true surface, the fitted lines should track the simulated proportions at every assay temperature.

```r
# Use the library's predict_survival_curves() and plot_survival_curves().
psc_bin <- predict_survival_curves(
  wf_bin, temps = temps,
  durations = 10 ^ seq(log10(0.5), log10(800), length.out = 100),
  ndraws = 200
)

plot_survival_curves(psc_bin, observed = std_bin) +
  ggplot2::scale_x_log10() +
  ggplot2::labs(x = expression(exposure~time~(min)))
```

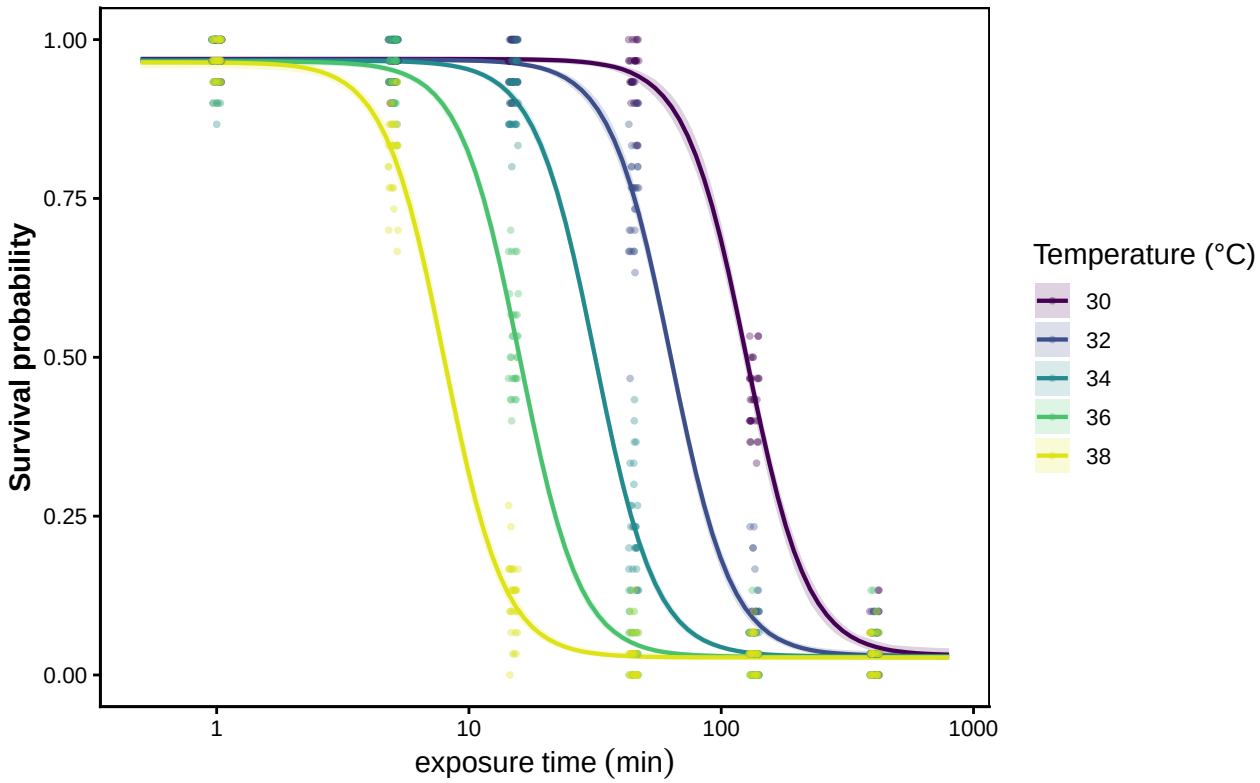


Figure S3: Posterior fitted 4PL curves (solid lines, with shaded 95% credible bands) overlaid on the simulated binomial data (points). One panel-coloured line per assay temperature. Where the fitted curves track the simulated proportions across all five temperatures, the model has recovered the underlying dose-response surface.

2.0.4 *Deriving z, $CT_{max_{1hr}}$, and T_{crit} from the posterior*

The whole point of fitting the joint Bayesian 4PL is that the classical TLS quantities fall out as transformations of the posterior we just fit — no second regression, no bootstrap. `extract_tdt()` always returns the first two and, opt-in via `lethal = TRUE`, also returns the third:

- z — thermal sensitivity, derived by a per-draw log-linear regression of $\log_{10} \mathrm{LT}_{\mathrm{threshold}}$ on temperature (Equation 7 of the manuscript).

- $CT_{max_{1hr}}$ — temperature at which the threshold is reached after 1 hour exposure, obtained from the fitted 4PL at the reference time, per posterior draw.

The threshold is set by the `target_surv` argument. The default `"relative"` evaluates at (low + up)/2 per draw, which reduces to the classical absolute 50 % survival when the asymptotes are anchored at 0 and 1 but is robust to sub-unit upper asymptotes (e.g. intrinsic background mortality unrelated to heat stress; see Section 4.4). Passing `target_surv = "absolute"` (or any numeric in $(0, 1)$) recovers the classical absolute threshold. The simulation in this section uses asymptotes anchored near 0 and 1, so the two definitions coincide here and the numbers below are unaffected by the choice. - T_{crit} *(only when* `lethal = TRUE`*)* — the temperature below which damage accumulates more slowly than a chosen rate floor. The damage rate at temperature T in the TDT framework is $r(T) = 100 \cdot 10^{(T - CT_{max,1hr})/z}$ % LT50-dose per hour, so inverting for a chosen rate floor r^* gives

$$T_{crit}(r^*) \;=\; CT_{max,1hr} \;+\; z \cdot \log_{10}(r^*/100). \tag{S3}$$

Different r^* values give different T_{crit} thresholds, and the literature does not converge on a single value. Empirical breakpoint analyses across taxa as different as *Drosophila suzukii* and *Lemna gibba* fall in the range $r^* \in [0.1, 1]$ % HI per hour, corresponding to "$\sim$ 4 days to 1000 hours of continuous exposure to reach LT_{50}" (Arnold *et al.* 2025; Faber *et al.* 2026; Jørgensen *et al.* 2021). Rather than commit to one r^*, we treat it as an unknown with a uniform prior on $\log_{10} r^*$ over that range and integrate; the posterior of T_{crit} then carries both parameter uncertainty (in $CT_{max,1hr}$ and z) and operational uncertainty (in the choice of r^*). The median of that pooled posterior sits at $CT_{max,1hr} - 2.5 \cdot z$, which corresponds to $r^* \approx 0.3$ % HI per hour (the geometric mean of the rate range).

> ⚠ T_{crit} is meaningful only for lethal-endpoint data
>
> The rate-multiplier T_{crit} in Equation Equation S3 interprets z as the slope of *damage accumulation* on temperature — strictly a property of lethal-endpoint dose-response data (proportion survival, knock-out counts where the endpoint is irreversible). For **sublethal** endpoints — knockdown time, photosystem-II decline, performance drop — the fitted z instead measures the slope of *performance reduction* with temperature, which is typically far steeper than its damage-accumulation analogue. Plugging a sublethal z into Equation Equation S3 pushes T_{crit} implausibly low (the supplement's sublethal shrimp example demonstrates this: a sublethal $z \approx 1$ °C yields a T_{crit} that would predict lethal exposures within ~2 hours at temperatures well within the organism's ecological range). Field practice treats T_{crit} as the breakpoint where the thermal-performance curve and the thermal-death-time curve intersect (Faber *et al.* 2026; Jørgensen *et al.* 2021; Ørsted *et al.* 2022), which requires *both* lethal and sublethal data — not just one.
> `extract_tdt()` reflects this by making T_{crit} **opt-in**: the default `lethal = FALSE` returns only z and $CT_{max_{1hr}}$, and users with lethal-endpoint data set `lethal = TRUE` to additionally derive T_{crit}. When `lethal = TRUE`, a one-line console message reiterates the lethal-only validity of the quantity.

The two always-returned quantities inherit the same posterior, so every credible interval reflects joint uncertainty in $(\ell, u, k, \beta_0, \beta_1)$. The opt-in T_{crit} pools that same posterior with the additional uniform prior on $\log_{10} r^*$.

> **Note**
>
> The default `fit_4pl()` puts a `temp_c` slope on every 4PL sub-parameter, so ℓ, u and k can each carry a linear effect of temperature in addition to the linear-mid term — when those slopes shrink to zero under the prior the model reduces cleanly to the constant-shape case. When the data carry real temperature signal on any of the shape parameters, the asymmetry correction $\frac{1}{k(T)} \log \frac{u(T)-0.5}{0.5-\ell(T)}$ varies with T and the resulting $\log_{10} \mathrm{LT50}(T)$ curve is bent rather than straight:
>
> $$\log_{10} \mathrm{LT50}(T) \;=\; \beta_0 + \beta_1 (T - \bar{T}) \;+\; \tfrac{1}{k(T)} \log \tfrac{u(T)-0.5}{0.5-\ell(T)}. \tag{S4}$$
>
> `extract_tdt()` requires no special handling in either regime, because both of its primitives operate on the *fitted 4PL surface* rather than on a closed-form expression: $CT_{max_{1hr}}$ is obtained by inverting the surface at the reference duration t_{ref} per posterior draw (`derive_temperature_for_duration()`), and the per-draw OLS regression that yields z runs over the per-draw $\log_{10} \mathrm{LT50}(T)$ curve recovered numerically by `derive_tdt_curve()`. When the curve is straight, the OLS slope returns the closed-form z; when it is bent, the same OLS returns the *average local slope* across the assay grid — a sensible single-summary z, with the per-temperature local values accessible from the underlying $\mathrm{LT50}(T)$ curve when needed. The worked example in Section 2.0.6 demonstrates this explicitly with a simulation where u and k carry strong T effects.

```
# Data are in minutes (standardize_data was called with duration_unit = "minutes"),
# so set time_multiplier = 1 to keep the model's time axis unchanged on output,
# and t_ref = 60 to read CTmax at the 1-hour reference. The simulated data are
# binomial / beta-binomial *lethal* survival counts, so we opt into T_crit via
# the rate-multiplier integration (lethal = TRUE; default TC_rate_range =
# c(0.1, 1) % HI per hour). See the next paragraph for why T_crit is opt-in.
et_bin <- extract_tdt(wf_bin, t_ref = 60, time_multiplier = 1,
                      ndraws = 1000, lethal = TRUE)
et_bb  <- extract_tdt(wf_bb,  t_ref = 60, time_multiplier = 1,
                      ndraws = 1000, lethal = TRUE)
```

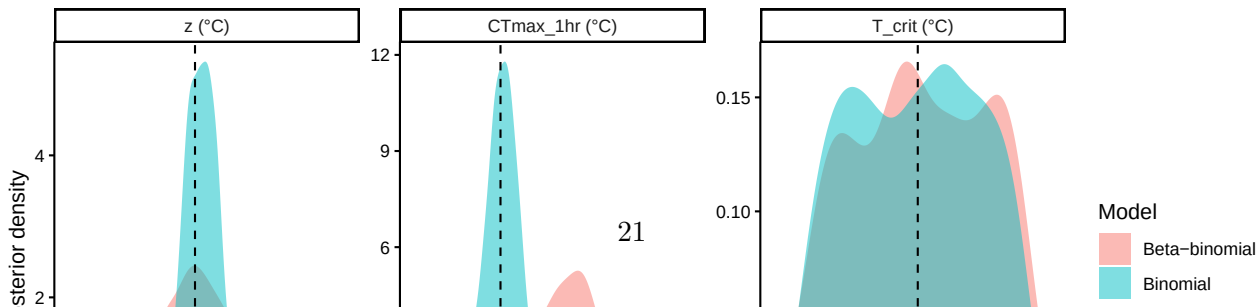
```
# Helper to pull (median, lower, upper) out of either a $z or temperature summary.
row_from_summary <- function(s, kind = c("z", "temp")) {
  kind <- match.arg(kind)
  if (kind == "z") {
    c(s$z_median, s$z_lower, s$z_upper)
  } else {
    c(s$temp_median, s$temp_lower, s$temp_upper)
  }
}

tls_recovery <- tibble::tibble(
  Quantity = c("z (°C)", "CTmax at 1 hr (°C)", "T_crit (°C)"),
  Truth    = c(true_z, true_CTmax, true_T_crit),
  `Bin. median` = c(row_from_summary(et_bin$z$summary,      "z")[1],
```

Table S4: Posterior medians and 95% credible intervals for thermal sensitivity (z), the critical thermal limit at a 1-hour reference duration ($CT_{max_{1hr}}$), and the rate-multiplier critical temperature (T_{crit}, integrated over $r^* \in [0.1, 1]$ % HI per hour). All three derived via `extract_tdt()` from a single posterior. Each row's truth value is computed analytically from `true_params`.

Quantity	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
z (°C)	6.67	6.69	6.57	6.83	6.69	6.37	7.06
CTmax at 1 hr (°C)	32.15	32.15	32.08	32.22	32.26	32.1	32.4
T_crit (°C)	15.48	15.46	12.18	18.59	15.51	12.04	18.81

```
                    row_from_summary(et_bin$CTmax$summary,  "temp")[1],
                    row_from_summary(et_bin$T_crit$summary, "temp")[1]),
  `Bin. lower`  = c(row_from_summary(et_bin$z$summary,      "z")[2],
                    row_from_summary(et_bin$CTmax$summary,  "temp")[2],
                    row_from_summary(et_bin$T_crit$summary, "temp")[2]),
  `Bin. upper`  = c(row_from_summary(et_bin$z$summary,      "z")[3],
                    row_from_summary(et_bin$CTmax$summary,  "temp")[3],
                    row_from_summary(et_bin$T_crit$summary, "temp")[3]),
  `BB median`   = c(row_from_summary(et_bb$z$summary,      "z")[1],
                    row_from_summary(et_bb$CTmax$summary,  "temp")[1],
                    row_from_summary(et_bb$T_crit$summary, "temp")[1]),
  `BB lower`    = c(row_from_summary(et_bb$z$summary,      "z")[2],
                    row_from_summary(et_bb$CTmax$summary,  "temp")[2],
                    row_from_summary(et_bb$T_crit$summary, "temp")[2]),
  `BB upper`    = c(row_from_summary(et_bb$z$summary,      "z")[3],
                    row_from_summary(et_bb$CTmax$summary,  "temp")[3],
                    row_from_summary(et_bb$T_crit$summary, "temp")[3])
)

tinytable::tt(tls_recovery, digits = 3, escape = TRUE)
```

```r
# Per-draw posteriors for the three quantities, both likelihoods. Trim each
# to a common length so we can stack columns in one tibble.
n_keep <- min(nrow(et_bin$z$draws), nrow(et_bin$CTmax$draws),
              nrow(et_bin$T_crit$draws),
              nrow(et_bb$z$draws), nrow(et_bb$CTmax$draws),
              nrow(et_bb$T_crit$draws))

draws_bin_d <- tibble::tibble(
  z         = et_bin$z$draws$z[seq_len(n_keep)],
  CTmax_1hr = et_bin$CTmax$draws$temp[seq_len(n_keep)],
  T_crit    = et_bin$T_crit$draws$temp[seq_len(n_keep)],
  model     = "Binomial"
)
draws_bb_d <- tibble::tibble(
  z         = et_bb$z$draws$z[seq_len(n_keep)],
  CTmax_1hr = et_bb$CTmax$draws$temp[seq_len(n_keep)],
  T_crit    = et_bb$T_crit$draws$temp[seq_len(n_keep)],
  model     = "Beta-binomial"
)

post_long <- dplyr::bind_rows(draws_bin_d, draws_bb_d) |>
  tidyr::pivot_longer(c(z, CTmax_1hr, T_crit),
                      names_to = "quantity", values_to = "value") |>
  dplyr::mutate(quantity = factor(quantity,
                                  levels = c("z", "CTmax_1hr", "T_crit"),
                                  labels = c("z (°C)",
                                             "CTmax_1hr (°C)",
                                             "T_crit (°C)")))

truth_long <- tibble::tibble(
  quantity = factor(c("z", "CTmax_1hr", "T_crit"),
                    levels = c("z", "CTmax_1hr", "T_crit"),
                    labels = levels(post_long$quantity)),
  truth    = c(true_z, true_CTmax, true_T_crit)
)

ggplot2::ggplot(post_long, ggplot2::aes(value, fill = model)) +
  ggplot2::geom_density(alpha = 0.5, colour = NA) +
  ggplot2::geom_vline(data = truth_long,
                      ggplot2::aes(xintercept = truth),
                      linetype = "dashed", colour = "black") +
  ggplot2::facet_wrap(~ quantity, scales = "free") +
  ggplot2::labs(x = NULL, y = "Posterior density", fill = "Model") +
  ggplot2::theme_classic()
```



All three quantities recover their simulation truth in both fits (Table S4, Figure S4), with credible intervals reflecting joint uncertainty in all 4PL parameters. The key practical advantage of the joint approach over the two-step pipeline: every classical TLS quantity is one library call away from the fit we already have, and every quantity inherits the same calibrated uncertainty.

Posterior summary statistics — mean, SD, median, and 95% credible interval — are also readily available from the `$draws` slot of `extract_tdt()`'s output:

```
summarise_draws <- function(x) {
  q <- stats::quantile(x, c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
  tibble::tibble(mean = mean(x, na.rm = TRUE),
                 sd   = stats::sd(x, na.rm = TRUE),
                 median = q[2], lower = q[1], upper = q[3])
}

uncert_summary <- dplyr::bind_rows(
  summarise_draws(et_bin$z$draws$z)          |> dplyr::mutate(quantity = "z (°C)",
  summarise_draws(et_bin$CTmax$draws$temp)   |> dplyr::mutate(quantity = "CTmax at 1 hr (°C)",
  summarise_draws(et_bin$T_crit$draws$temp)  |> dplyr::mutate(quantity = "T_crit (°C)", likeli
  summarise_draws(et_bb$z$draws$z)           |> dplyr::mutate(quantity = "z (°C)",
  summarise_draws(et_bb$CTmax$draws$temp)    |> dplyr::mutate(quantity = "CTmax at 1 hr (°C)",
  summarise_draws(et_bb$T_crit$draws$temp)   |> dplyr::mutate(quantity = "T_crit (°C)", likeli
) |>
  dplyr::select(quantity, likelihood, mean, sd, median, lower, upper)

# Scalars for inline reporting in the next paragraph.
sd_z_bin  <- round(stats::sd(et_bin$z$draws$z),         2)
sd_z_bb   <- round(stats::sd(et_bb$z$draws$z),          2)
sd_ct_bin <- round(stats::sd(et_bin$CTmax$draws$temp),  2)
sd_ct_bb  <- round(stats::sd(et_bb$CTmax$draws$temp),   2)
ci_z_bin  <- round(diff(stats::quantile(et_bin$z$draws$z,         c(0.025, 0.975), names = FALS
ci_z_bb   <- round(diff(stats::quantile(et_bb$z$draws$z,          c(0.025, 0.975), names = FALS
ci_ct_bin <- round(diff(stats::quantile(et_bin$CTmax$draws$temp,  c(0.025, 0.975), names = FALS
ci_ct_bb  <- round(diff(stats::quantile(et_bb$CTmax$draws$temp,   c(0.025, 0.975), names = FALS

uncert_summary |>
  dplyr::rename(
    Quantity   = quantity,
    Likelihood = likelihood,
    Mean       = mean,
    SD         = sd,
    Median     = median,
    Lower      = lower,
    Upper      = upper
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

In short, both z and $CT_{max_{1hr}}$ are estimated with high precision and their credible intervals come

Table S5: Posterior summaries of z, $CT_{max_{1hr}}$, and T_{crit} from the joint Bayesian 4PL: posterior mean, standard deviation, median, and 95% credible interval, reported separately for the binomial and beta-binomial fits.

Quantity	Likelihood	Mean	SD	Median	Lower	Upper
z (°C)	Binomial	6.67	0.0652	6.67	6.55	6.8
CTmax at 1 hr (°C)	Binomial	32.13	0.0322	32.13	32.07	32.2
T_crit (°C)	Binomial	15.44	1.8965	15.45	12.23	18.6
z (°C)	Beta-binomial	6.49	0.1478	6.49	6.22	6.8
CTmax at 1 hr (°C)	Beta-binomial	32.23	0.0723	32.23	32.08	32.4
T_crit (°C)	Beta-binomial	16.02	1.9075	15.96	12.72	19.2

straight from the joint posterior. If we fit this model using frequentist approaches we could do the same and use the delta-method, or bootstrapping to get uncertainty.

Under the binomial likelihood, the standard error (or SD of posterior) for z is 0.07 °C with a 95% credible interval 0.26 °C wide; $CT_{max_{1hr}}$ has SD 0.03 °C and a 95% credible interval width of 0.12 °C.

The beta-binomial fit is slightly wider on both quantities (SD on z: 0.15 °C; on $CT_{max_{1hr}}$: 0.07 °C) because ϕ has absorbed the extra-binomial variance and that uncertainty propagates honestly into the derived quantities — none of which falls out of the two-stage pipeline without an additional bootstrap or delta-method step.

> **!** Important
>
> It is always important to evaluate whether a binomial and a beta-binomial model better fits real data. The extra dispersion, typical of real biological data, should be estimated when it's present as it has important consequences for inferences.

2.0.5 *Comparing the two pipelines*

2.0.5.1 Point-estimate agreement on the same data

Table S6 puts the two-stage point estimates next to the joint Bayesian posterior medians for both simulated datasets.

```
compare_table <- tibble::tibble(
  quantity   = c("z (°C)", "CTmax at 1 hr (°C)"),
  truth      = c(true_z, true_CTmax),
  ts_bin     = ts_bin$est,
  joint_bin  = c(median(draws_bin_d$z),
                 median(draws_bin_d$CTmax_1hr)),
  ts_bb      = ts_bb$est,
  joint_bb   = c(median(draws_bb_d$z),
```

Table S6: Point estimates (two-stage) and posterior medians (joint Bayesian) for thermal sensitivity (z) and $CT_{max_{1hr}}$, recovered by each of four estimator–dataset combinations from the same simulated datasets. Truth values are the simulation parameters.

Quantity	Truth	Two-stage (binomial)	Joint Bayesian (binomial)	Two-stage (beta-binomial)
z (°C)	6.67	6.91	6.69	6.9
CTmax at 1 hr (°C)	32.15	32.03	32.15	32.1

```
                median(draws_bb_d$CTmax_1hr))
)

compare_table |>
  dplyr::rename(
    Quantity                          = quantity,
    Truth                             = truth,
    `Two-stage (binomial)`            = ts_bin,
    `Joint Bayesian (binomial)`       = joint_bin,
    `Two-stage (beta-binomial)`       = ts_bb,
    `Joint Bayesian (beta-binomial)`  = joint_bb
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

All four estimators land on essentially the same point estimates for z and $CT_{max_{1hr}}$, though we notice the two stage is biased upwards slightly, and all sit close to the simulation truth. The joint Bayesian 4PL gives the same answer the classical pipeline gives — it is *not* in disagreement with the field's existing point estimates, it reproduces them.

2.0.6 *A worked example: temperature effects on every shape parameter*

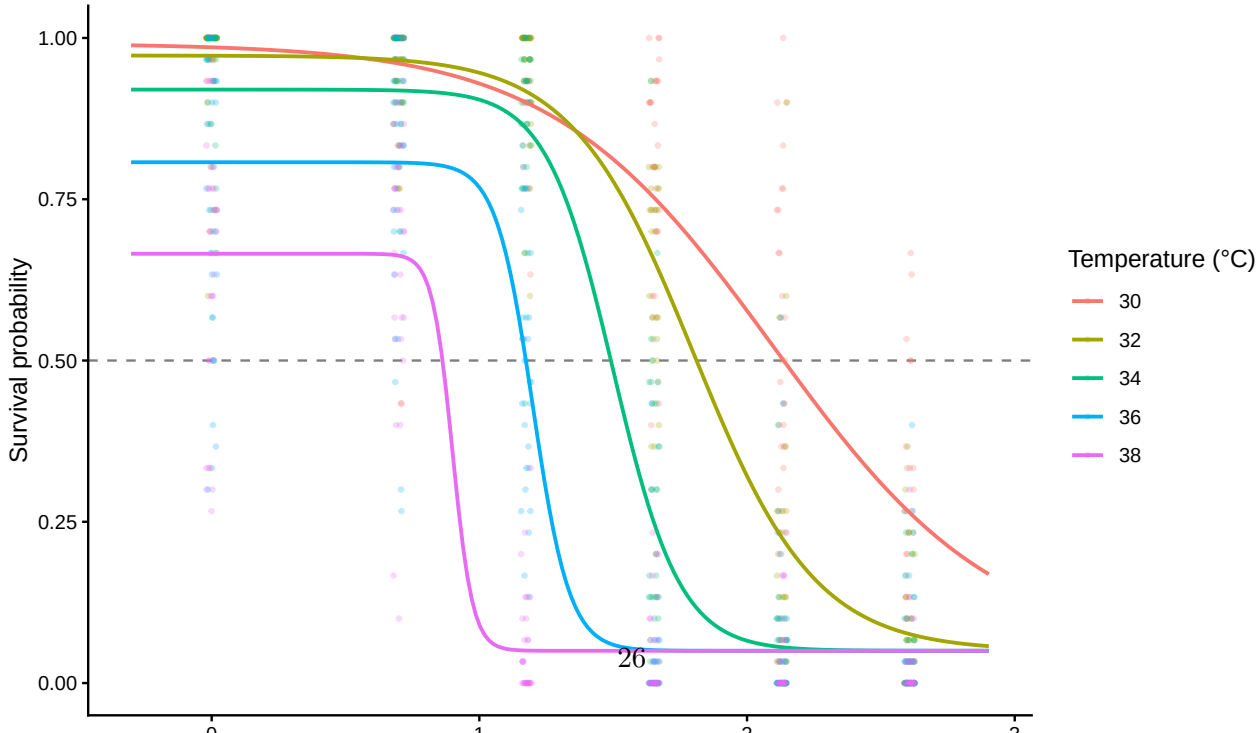
The default `fit_4pl()` already puts a `temp_c` slope on ℓ, u and k, so any temperature effect on the shape parameters is fit alongside the linear midpoint. To validate that `extract_tdt()` returns sensible z, $CT_{max_{1hr}}$ and $\log_{10}\mathrm{LT50}(T)$ posteriors when the truth violates the constant-shape assumption, we re-simulate beta-binomial data on the same factorial design as Section 2.0.1.2, but now u declines steeply with T and k rises steeply with T. The fit is then exactly the same `fit_4pl()` call as before — no special formula, no special priors — and the analysis afterwards is exactly the same `extract_tdt()` call.

```
# Same factorial design as @sec-sim-step2; re-declared here so this chunk is
# self-contained.
temps     <- c(30, 32, 34, 36, 38)
durations <- c(1, 5, 15, 45, 135, 405)
n_rep     <- 30
n_per_rep <- 30
phi       <- 5
```

```r
# Truth on the same raw scale fit_4pl() uses internally (so we can compare the
# posterior coefficients to truth directly):
#   ell = 0.49 * inv_logit(ell_raw)                    (held constant in T)
#   u   = 0.51 + 0.49 * inv_logit(u_raw_0 + u_raw_T*T_c)  (declines in T)
#   k   = exp(log_k_0 + log_k_T*T_c)                     (rises in T)
true_params_ext <- list(
  ell_raw  = qlogis(0.05 / 0.49),                # ell = 0.05
  u_raw_0  = qlogis((0.92 - 0.51) / 0.49),       # u = 0.92 at T_bar
  u_raw_T  = -0.60,                               # strong decline in u with T
  log_k_0  = log(8),                              # k = 8 at T_bar
  log_k_T  = 0.30,                                # strong rise in k with T
  m_beta0  = 1.5,
  m_beta1  = -0.15,
  T_bar    = 34
)

design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params_ext$T_bar,
    n       = n_per_rep
  )

sim_bb_ext <- design |>
  dplyr::mutate(
    ell    = 0.49 * plogis(true_params_ext$ell_raw),
    u      = 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                                  true_params_ext$u_raw_T * T_c),
    k      = exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c),
    mid    = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c,
    p_true = ell + (u - ell) / (1 + exp(k * (log10_t - mid))),
    p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
    y      = rbinom(dplyr::n(), size = n, prob = p_draw)
  )
```

Under this truth the lower asymptote stays flat at $\ell = 0.05$, the upper asymptote u falls from 0.99 at $T = 30$ to 0.67 at $T = 38$, and the steepness k rises from 2.4 to 26.6 across the same range. The resulting dose-response curves are visibly different in shape at the two extremes (Figure S5).

```
truth_grid_ext <- tidyr::expand_grid(
  T = temps,
  t = 10 ^ seq(log10(0.5), log10(800), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params_ext$T_bar,
    ell     = 0.49 * plogis(true_params_ext$ell_raw),
    u       = 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                                   true_params_ext$u_raw_T * T_c),
    k       = exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c),
    mid     = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c,
    p_true  = ell + (u - ell) / (1 + exp(k * (log10_t - mid)))
  )

ggplot2::ggplot() +
  ggplot2::geom_hline(yintercept = 0.5, linetype = "dashed",
                      colour = "grey50") +
  ggplot2::geom_jitter(
    data = sim_bb_ext,
    ggplot2::aes(x = log10_t, y = y / n, colour = factor(T)),
    width = 0.02, height = 0, alpha = 0.25, size = 0.7
  ) +
  ggplot2::geom_line(
    data = truth_grid_ext,
    ggplot2::aes(x = log10_t, y = p_true, colour = factor(T)),
    linewidth = 0.8
  ) +
  ggplot2::labs(x = expression(log[10]~exposure~time~(min)),
                y = "Survival probability",
                colour = "Temperature (°C)") +
  ggplot2::theme_classic()
```



Before fitting we can compute, analytically, what the truth implies for $\log_{10}\mathrm{LT50}(T)$, the local thermal sensitivity $z(T)$ at each assay temperature (defined as -1 divided by the local slope of $\log_{10}\mathrm{LT50}$ on T), and $CT_{max_{1hr}}$ (the T at which $\log_{10}\mathrm{LT50}$ crosses $\log_{10} 60$). These are the targets we will ask `extract_tdt()` to recover.

```
# True log10(LT50) at any T from the T-varying 4PL: solve p(log10_t) = 0.5
# for log10_t. With the 4PL,
#   log10_t = mid + (1/k) * log((u - 0.5) / (0.5 - ell))
true_log10_LT50 <- function(T_value) {
  T_c <- T_value - true_params_ext$T_bar
  ell <- 0.49 * plogis(true_params_ext$ell_raw)
  u   <- 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                              true_params_ext$u_raw_T * T_c)
  k   <- exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c)
  mid <- true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c
  mid + (1 / k) * log((u - 0.5) / (0.5 - ell))
}

# Local z(T) via central finite difference of true log10(LT50)(T).
true_local_z <- function(T_at, half_step = 0.5) {
  -1 / ((true_log10_LT50(T_at + half_step) -
         true_log10_LT50(T_at - half_step)) / (2 * half_step))
}

# True CTmax_1hr: search the fine grid for the T at which log10(LT50)
# crosses log10(60).
T_grid_truth   <- seq(20, 45, by = 0.001)
ll_grid_truth  <- vapply(T_grid_truth, true_log10_LT50, numeric(1))
true_CTmax_1hr <- T_grid_truth[which.min(abs(ll_grid_truth - log10(60)))]

truth_ext <- tibble::tibble(
  T               = temps,
  log10_LT50_true = vapply(temps, true_log10_LT50, numeric(1)),
  LT50_min_true   = 10 ^ log10_LT50_true,
  local_z_true    = vapply(temps, true_local_z, numeric(1))
)

true_mean_local_z <- mean(truth_ext$local_z_true)
```

Across the assay range the true $\log_{10}\mathrm{LT50}(T)$ curve is bent: the implied local z varies from 6.05 °C at $T = 30$ to 6.53 °C at $T = 38$ (truth mean $z = 6.28$ °C). The truth implies $CT_{max_{1hr}} = 32.21$ °C. With straight-line $\log_{10}\mathrm{LT50}(T)$ — the constant-shape case — local z would not vary across temperature at all.

We now fit with exactly the same `fit_4pl()` call as before. The default formula already puts a `temp_c` slope on each of the four 4PL sub-parameters, so the model has the capacity to recover the truth's temperature signal on u and k without any modification.

```
std_ext <- standardize_data(sim_bb_ext,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")

wf_ext <- fit_4pl(std_ext,
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file       = file.path(models_dir,
                                         "sim_4pl_ext_betabin_DEFAULT_strong_v1"),
                  file_refit = "never")     # see fit-4pl-tutorial note above
```

The four 4PL sub-parameters on the natural (biological) scale at each assay temperature are recovered in Table S7. The lower asymptote ℓ stays near 0.05 at every T (consistent with the truth that ℓ does not vary with temperature), while u falls and k rises across the assay range — exactly the truth.

```
ext_post  <- posterior::as_draws_df(wf_ext$fit) |> as.data.frame()
ext_meta  <- wf_ext$meta
ext_bnds  <- ext_meta$bounds         # compute_4pl_bounds(0, 1)
ext_temp_mean <- ext_meta$temp_mean  # what the fit centred on

ext_param_long <- purrr::map_dfr(temps, function(T_value) {
  T_c_fit  <- T_value - ext_temp_mean
  T_c_true <- T_value - true_params_ext$T_bar

  draws_at_T <- tibble::tibble(
    ell = ext_bnds$low_min +
          stats::plogis(ext_post$b_lowraw_Intercept +
                        ext_post$b_lowraw_temp_c * T_c_fit) * ext_bnds$low_w,
    u   = ext_bnds$up_min  +
          stats::plogis(ext_post$b_upraw_Intercept +
                        ext_post$b_upraw_temp_c * T_c_fit)  * ext_bnds$up_w,
    k   = exp(ext_post$b_logk_Intercept + ext_post$b_logk_temp_c * T_c_fit),
    mid = ext_post$b_mid_Intercept + ext_post$b_mid_temp_c * T_c_fit
  )

  truth_at_T <- tibble::tibble(
    ell = 0.49 * plogis(true_params_ext$ell_raw),
    u   = 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                               true_params_ext$u_raw_T * T_c_true),
    k   = exp(true_params_ext$log_k_0 +
              true_params_ext$log_k_T * T_c_true),
    mid = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c_true
  )

  purrr::map_dfr(c("ell", "u", "k", "mid"), function(p) {
    q <- stats::quantile(draws_at_T[[p]], c(0.025, 0.5, 0.975),
```

```
                          na.rm = TRUE, names = FALSE)
    tibble::tibble(
      Parameter         = p,
      `T (°C)`          = T_value,
      Truth             = truth_at_T[[p]],
      Median            = q[2],
      Lower             = q[1],
      Upper             = q[3]
    )
  })
})

ext_param_long |>
  tinytable::tt(digits = 3, escape = TRUE)
```

Now we apply the same `extract_tdt()` call as in Section 2.0.4 — there is no separate "T-varying" workflow because `extract_tdt()` already inverts the fitted surface numerically per draw:

```
et_ext <- extract_tdt(wf_ext, t_ref = 60, time_multiplier = 1,
                      ndraws = 1000, lethal = TRUE)
```

Table S8 compares the three classical TDT quantities from `extract_tdt()` to the analytical truth derived above.

```
tibble::tibble(
  Quantity     = c("z (°C)",
                   "CTmax at 1 hr (°C)",
                   "T_crit (°C)"),
  Truth        = c(round(true_mean_local_z, 3),
                   round(true_CTmax_1hr, 3),
                   NA_real_),  # T_crit truth involves r* integration; see below
  Median       = c(et_ext$z$summary$z_median,
                   et_ext$CTmax$summary$temp_median,
                   et_ext$T_crit$summary$temp_median),
  Lower        = c(et_ext$z$summary$z_lower,
                   et_ext$CTmax$summary$temp_lower,
                   et_ext$T_crit$summary$temp_lower),
  Upper        = c(et_ext$z$summary$z_upper,
                   et_ext$CTmax$summary$temp_upper,
                   et_ext$T_crit$summary$temp_upper)
) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

Both posteriors comfortably cover their analytical truth. The single-summary z is the OLS slope across the assay grid — which, on a bent curve, estimates the *average* local slope (truth = 6.28 °C, posterior median 6.65 °C, 95% CrI [6.14, 7.3] °C). $CT_{max_{1hr}}$ is recovered by direct 4PL inversion

Table S7: Posterior recovery of the four 4PL sub-parameters on the natural scale at each assay temperature. The fit uses the disjoint-bounds reparameterisation (ℓ, u via inv_logit, k via exp); raw posterior columns are transformed back to the biological scale before summarising. Posterior medians and 95% credible intervals are reported alongside the analytical truth.

Parameter	T (°C)	Truth	Median	Lower	Upper
ell	30	0.05	0.0625	0.0293	0.1084
u	30	0.991	0.9935	0.989	0.9962
k	30	2.41	2.4043	2.1161	2.7386
mid	30	2.1	2.1065	2.0379	2.1701
ell	32	0.05	0.0536	0.0311	0.0797
u	32	0.973	0.9791	0.9697	0.9863
k	32	4.39	4.5723	4.1102	5.1149
mid	32	1.8	1.8061	1.7625	1.847
ell	34	0.05	0.0457	0.0328	0.0596
u	34	0.92	0.932	0.9166	0.9463
k	34	8	8.7122	7.1951	10.6985
mid	34	1.5	1.5056	1.4818	1.5292
ell	36	0.05	0.0391	0.0317	0.0475
u	36	0.807	0.8168	0.7885	0.842
k	36	14.577	16.5544	12.261	22.8179
mid	36	1.2	1.2049	1.1814	1.2328
ell	38	0.05	0.0335	0.0248	0.0444
u	38	0.666	0.6592	0.6201	0.6995
k	38	26.561	31.5023	20.6197	48.8663
mid	38	0.9	0.9041	0.8624	0.9533

Table S8: Classical TDT quantities from `extract_tdt()` on the T-varying fit, compared against analytical truth. The single-summary z is the slope of a per-draw OLS regression of $\log_{10}$ LT50 on T across the assay grid; the truth row reports the mean of the analytical local $z(T)$ at the same assay temperatures, which is what the OLS slope estimates when the curve is bent. $CT_{max_{1hr}}$ comes from direct numerical inversion of the fitted 4PL at 60 min per draw.

Quantity	Truth	Median	Lower	Upper
z (°C)	6.28	6.65	6.14	7.3
CTmax at 1 hr (°C)	32.21	32.18	31.89	32.4
T_crit (°C)	NA	15.44	11.95	19.1

at the reference duration and lands on the truth (32.21 °C → posterior median 32.18 °C, 95% CrI [31.89, 32.43] °C); the numerical inversion does not depend on the asymmetry correction being constant in T.

The bent $\log_{10}$ LT50(T) curve itself is the intermediate object both quantities are built from; it lives in `et_ext$lt50_curve` (the output of `derive_tdt_curve()`) and can be plotted directly (Figure S6).

```
truth_overlay <- {
  Ts <- seq(min(temps) - 2, max(temps) + 2, by = 0.05)
  tibble::tibble(temp = Ts,
                 duration_median = 10 ^ vapply(Ts, true_log10_LT50,
                                               numeric(1)))
}

plot_tdt_curve(et_ext$lt50_curve) +
  ggplot2::geom_line(data = truth_overlay,
                     ggplot2::aes(x = temp, y = duration_median),
                     linetype = "dashed", colour = "black",
                     inherit.aes = FALSE)
```

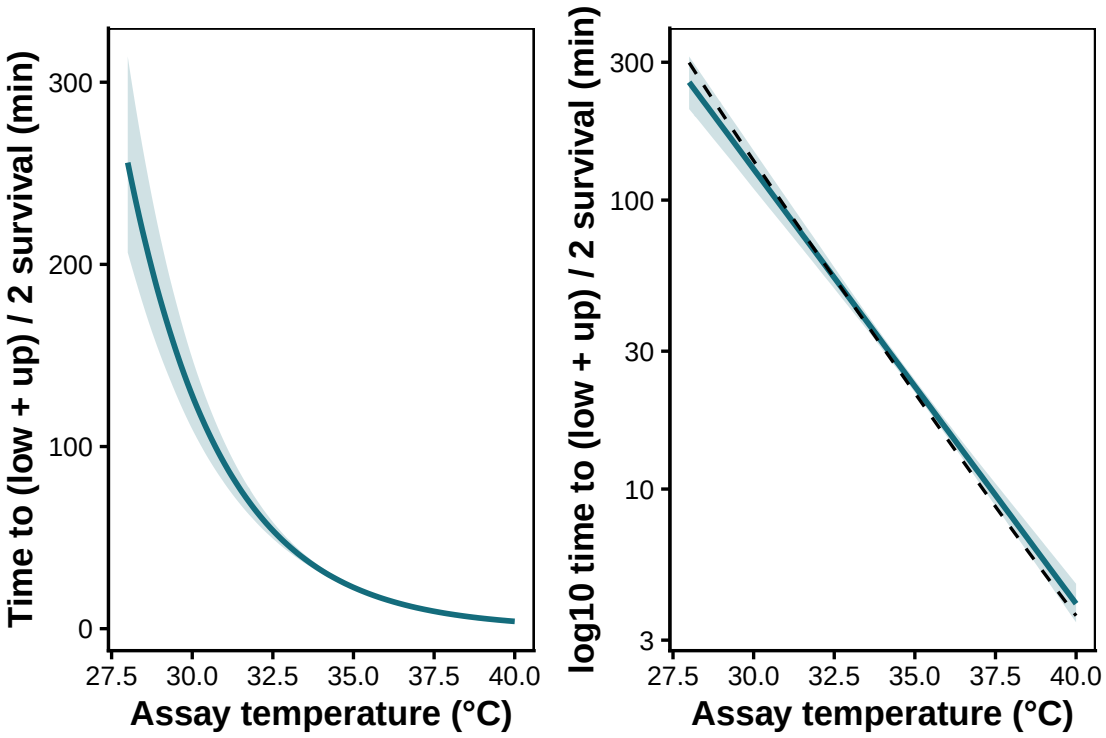


Figure S6: Posterior LT50(T) curve (median + 95% CrI, blue) recovered by `derive_tdt_curve()` from the T-varying fit, with the analytical truth overlaid (dashed black). The curve is visibly bent rather than straight in T, and the posterior tracks the truth.

We can also drop down to the per-draw level and compute a *local* $z(T)$ at each assay temperature from the same `et_ext$lt50_curve` posterior — a finite difference of $\log_{10}$ LT50 around T per draw,

summarised across draws (Figure S7).

```
# Finite-difference local z at each assay T, per posterior draw, derived
# from et_ext$lt50_curve (the same per-draw LT50(T) curve extract_tdt
# already computed for the OLS).
half_step <- 0.5

local_z_summary <- purrr::map_dfr(temps, function(T_at) {
  z_per_draw <- et_ext$lt50_curve$draws |>
    dplyr::group_by(.draw) |>
    dplyr::summarise(
      z_local = -1 / ((stats::approx(temp, log10(duration_out),
                                     xout = T_at + half_step)$y -
                       stats::approx(temp, log10(duration_out),
                                     xout = T_at - half_step)$y) /
                      (2 * half_step)),
      .groups = "drop"
    )
  q <- stats::quantile(z_per_draw$z_local, c(0.025, 0.5, 0.975),
                       na.rm = TRUE, names = FALSE)
  tibble::tibble(T_at           = T_at,
                 z_local_median = q[2],
                 z_local_lower  = q[1],
                 z_local_upper  = q[3])
})

local_z_summary |>
  dplyr::left_join(
    tibble::tibble(T_at = temps, z_local_truth = truth_ext$local_z_true),
    by = "T_at"
  ) |>
  dplyr::select(T_at, z_local_truth, z_local_median,
                z_local_lower, z_local_upper) |>
  dplyr::rename(
    `T (°C)`              = T_at,
    `z local truth (°C)`  = z_local_truth,
    Median                = z_local_median,
    Lower                 = z_local_lower,
    Upper                 = z_local_upper
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

T (°C)	z local truth (°C)	Median	Lower	Upper
30	6.05	6.33	5.72	6.97
32	6.22	6.5	6.04	7.05
34	6.27	6.53	6.08	7.07
36	6.34	6.57	6.08	7.16
38	6.53	6.75	6.21	7.39

```
truth_curve_z <- {
  Ts <- seq(min(temps), max(temps), by = 0.1)
  tibble::tibble(T_at = Ts,
                 z_local_truth = vapply(Ts, true_local_z, numeric(1)))
}

ggplot2::ggplot(local_z_summary,
                ggplot2::aes(x = T_at, y = z_local_median)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = z_local_lower,
                                    ymax = z_local_upper),
                       fill = "#146C7C", alpha = 0.2) +
  ggplot2::geom_line(colour = "#146C7C", linewidth = 1) +
  ggplot2::geom_line(data = truth_curve_z,
                     ggplot2::aes(x = T_at, y = z_local_truth),
                     linetype = "dashed", colour = "black",
                     inherit.aes = FALSE) +
  ggplot2::labs(x = "Assay temperature (°C)",
                y = expression(local~italic(z)~"(°C)")) +
  ggplot2::theme_classic()
```

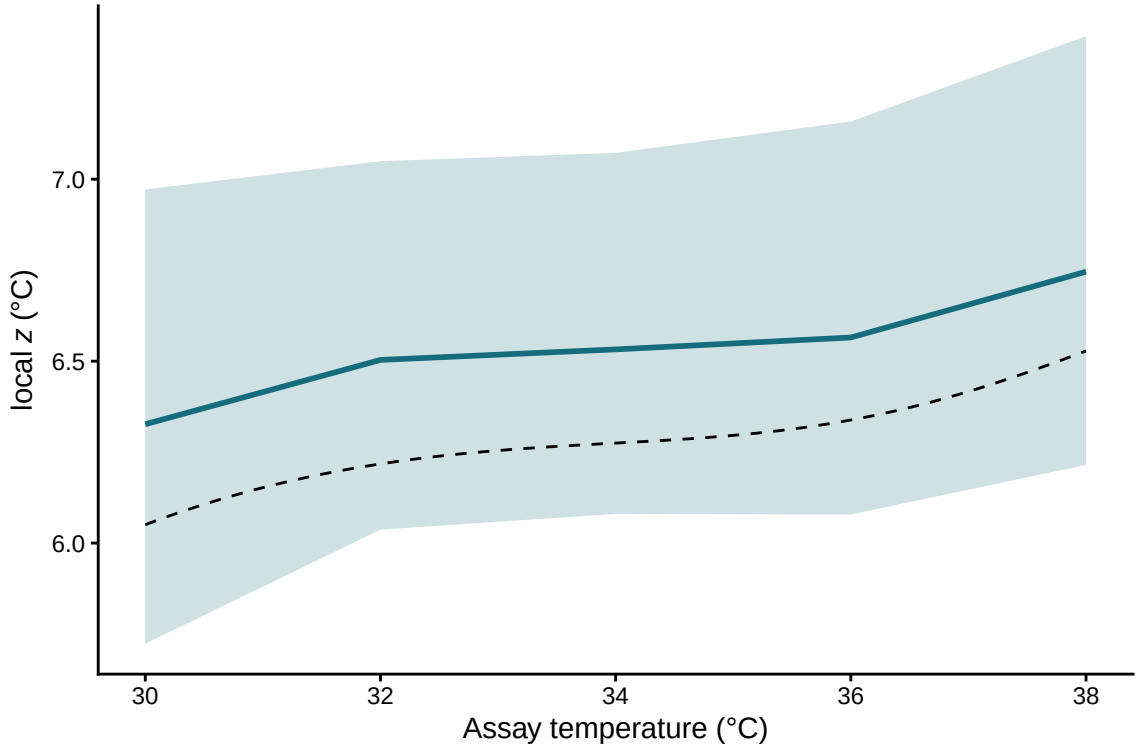


Figure S7: Local thermal sensitivity $z(T)$ recovered from the per-draw $\log_{10}\mathrm{LT50}(T)$ curve (blue median + 95% CrI), with the analytical truth overlaid (dashed black). Where the constant-shape simple-case fit would give the same z at every assay temperature, here

The *same* `fit_4pl()` and `extract_tdt()` calls recover the bent $\log_{10}\mathrm{LT50}(T)$ curve, the average z across the assay grid, and the per-draw $CT_{max_{1hr}}$ without modification. When the truth is constant-shape (as in Section 2.0.4) the same fit returns near-zero slopes on u, ℓ and k, the bent-curve calculation collapses to the closed-form one.

2.0.7 Heat injury accumulation and survival under a fluctuating trace

A particularly powerful aspect of TLS models is their ability to translate dynamic temperature time-series from nature and predict how such temperatures lead to heat injury (HI) accumulation towards a threshold (e.g., LT_{50}) (Arnold *et al.* 2025; Jørgensen *et al.* 2021; Noble *et al.* 2026; Ørsted *et al.* 2022; Rezende *et al.* 2020). The joint Bayesian 4PL fit gives us not just point estimates of z and $CT_{max_{1hr}}$ but their full joint posterior, which we can push through the HI integral to obtain a posterior distribution over the predicted HI trajectory. We can also use the *full* 4PL surface to predict the cumulative survival fraction $S_{\mathrm{pred}}(t)$ across the trajectory (Equation 9 of the manuscript). The library function `predict_heat_injury()` does both calculations in a single call and returns both trajectories with full posterior uncertainty.

2.0.7.1 A simulated temperature profile across 5 days

To mimic conditions a moderately heat-tolerant ectotherm might experience, we generate 5 days of hourly temperatures with a sinusoidal diurnal cycle, ranging from 12 °C overnight to a midday peak of 24 °C. The critical temperature T_c — below which damage is repaired as fast as it accrues and so there is no net damage increase (Jørgensen *et al.* 2021; Ørsted *et al.* 2022) — is set to 20 °C. The simulated organism has $CT_{max_{1hr}} \approx 32.1$°C and $z \approx 6.7$°C, so the daily peak at 24 °C remains 8 °C below the 1-hour critical limit — moderate, sub-lethal stress that builds cumulatively.

```
hi_trace <- tibble::tibble(
  time_h = 0:(24 * 5 - 1),
  temp   = 18 + 6 * sin(2 * pi * (0:(24 * 5 - 1) - 6) / 24)   # range 12-24 °C
)
hi_T_c <- 20    # damage threshold (°C)
```

```r
ggplot2::ggplot(hi_trace, ggplot2::aes(time_h, temp)) +
  ggplot2::geom_line(colour = "darkred", linewidth = 0.6) +
  ggplot2::geom_hline(yintercept = hi_T_c, linetype = "dashed", colour = "grey40") +
  ggplot2::geom_hline(yintercept = true_CTmax, linetype = "dotted", colour = "grey40") +
  ggplot2::annotate("text", x = max(hi_trace$time_h), y = hi_T_c, label = "T_c",
                    hjust = 1, vjust = -0.4, size = 3, colour = "grey40") +
  ggplot2::annotate("text", x = max(hi_trace$time_h), y = true_CTmax,
                    label = "CTmax_1hr",
                    hjust = 1, vjust = -0.4, size = 3, colour = "grey40") +
  ggplot2::ylim(10, 45) +
  ggplot2::labs(x = "Hour", y = "Temperature (°C)") +
  ggplot2::theme_classic()
```

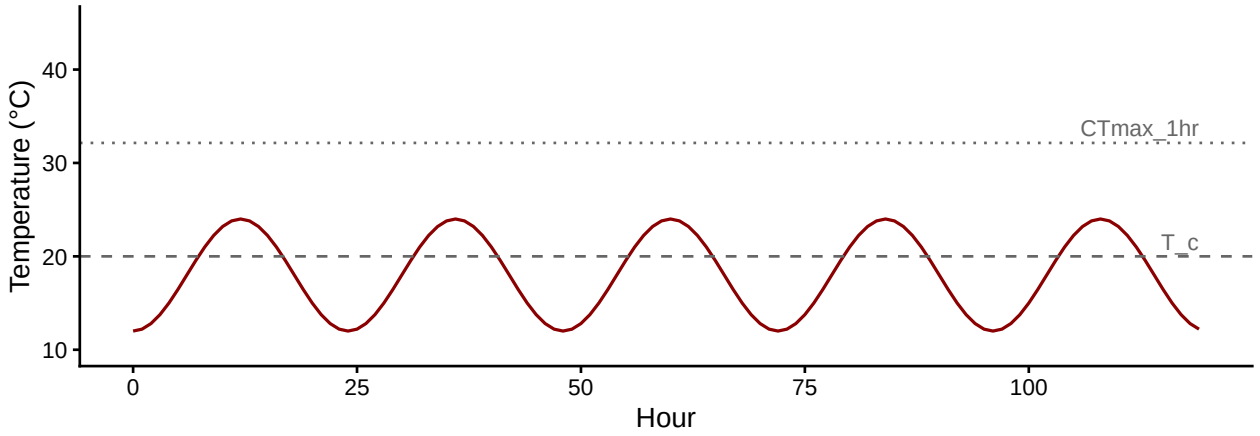


Figure S8: 5-day diurnal temperature trace. The dashed line is the damage-accumulation threshold $T_c = 20$ °C; the dotted line is the simulated organism's $CT_{max_{1hr}} \approx 32.1$°C — well above the daily peaks, so accumulation is sub-lethal and gradual.

2.0.7.2 Posterior HI and survival via `predict_heat_injury()`

`predict_heat_injury()` takes a temperature trace plus a fitted workflow, propagates the full posterior of $(\ell, u, k, \beta_0, \beta_1)$ through the HI integral, and returns the median and 95% credible band of cumulative HI and predicted survival at every time point. We supply `T_c` to enforce zero damage accumulation below the threshold (matching Equation 7 of the manuscript). The model time units here are minutes (because `standardize_data()` was called with `duration_unit = "minutes"`), so the trace's `time_h` is in hours and damage rates are computed accordingly internally.

```r
# Convert hourly trace to minutes to match the model's time units.
hi_trace_min <- dplyr::mutate(hi_trace, time_h = time_h * 60)

hi <- predict_heat_injury(
  trace        = hi_trace_min,
  workflow     = wf_bb,
  target_surv  = 0.5,
  T_c          = hi_T_c,
```

```
  ndraws        = 500
)
```

```r
# Convert back to hours for plotting.
hi_plot <- dplyr::mutate(hi$summary, time_h = time_h / 60)

p_hi <- ggplot2::ggplot(hi_plot, ggplot2::aes(time_h, hi_median)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = hi_lower, ymax = hi_upper),
                       alpha = 0.2, fill = "darkred") +
  ggplot2::geom_line(colour = "darkred", linewidth = 0.8) +
  ggplot2::geom_hline(yintercept = 100, linetype = "dashed", colour = "grey40") +
  ggplot2::labs(x = NULL, y = "Cumulative HI (%)") +
  ggplot2::theme_classic()

p_surv <- ggplot2::ggplot(hi_plot, ggplot2::aes(time_h, surv_median)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = surv_lower, ymax = surv_upper),
                       alpha = 0.2, fill = "steelblue") +
  ggplot2::geom_line(colour = "steelblue", linewidth = 0.8) +
  ggplot2::geom_hline(yintercept = 0.5, linetype = "dashed", colour = "grey40") +
  ggplot2::scale_y_continuous(limits = c(0, 1)) +
  ggplot2::labs(x = "Hour", y = "Predicted survival") +
  ggplot2::theme_classic()

patchwork::wrap_plots(p_hi, p_surv, ncol = 1)
```

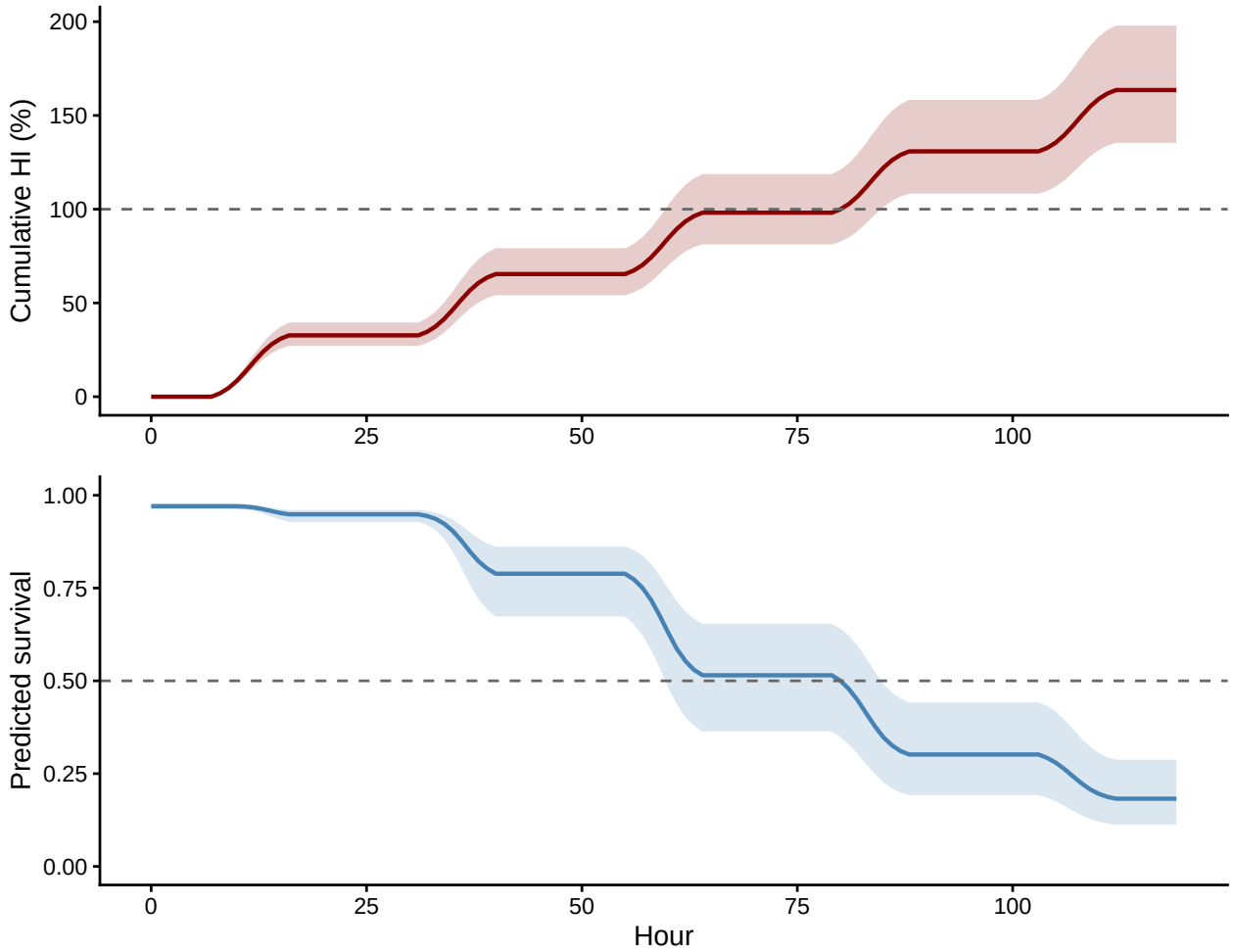


Figure S9: Posterior heat-injury accumulation (top, red) and predicted survival fraction (bottom, blue) under the 5-day field trace. Solid lines are posterior medians across 500 draws from `wf_bb`; shaded bands are 95% credible intervals. The dashed lines mark the LT50 threshold (HI = 100%) and 50% survival respectively. The two trajectories tell the same

By the end of day 5 the posterior median HI is 164% (95% CI: 135% to 198%), and median predicted survival is 0.18 (95% CI: 0.11 to 0.29).

2.0.7.3 Why are the credible bands so wide?

Both posterior bands look generous relative to the precision suggested by Table S4 on the underlying z and $CT_{max_{1hr}}$. That is not a bug — it is the exponential rate-multiplier $10^{(T-CT_{max,1hr})/z}$ doing its job. A 1 °C shift in $CT_{max_{1hr}}$ multiplies the per-hour HI rate by $10^{1/z}$, which for $z \approx 2.6$ °C is a ~2.4× change; a 0.5 °C shift in z at the same peak temperature shifts the rate by a comparable factor. Over a 5-day integral those multiplicative differences compound, so even tight parameter posteriors translate into visibly wide HI bands. The classical two-stage pipeline conceals this by treating the point estimates of z and $CT_{max_{1hr}}$ as known when it computes HI — the joint Bayesian framework instead propagates that uncertainty honestly. The width of the band is therefore a faithful report of how much you should *not* over-interpret a point prediction, not a sign that the fit is too uncertain to be useful.

2.0.8 Validation: recovering known planted doses

Before trusting `predict_heat_injury()` on real field data, we validate it against analytical truth on three reference traces with known dosing structure. `make_temperature_scenarios()` builds the traces; `planted_dose_from_trace()` implements the classical HI integral (Equation 7) directly given known `z` and `CTmax_1hr`. Posterior medians from `predict_heat_injury()` should sit near the analytical truth, and the 95% credible intervals should cover it.

```
# Three traces: flat baseline (zero HI), single hourly spike, three spikes.
val_scens <- make_temperature_scenarios(
  baseline    = 20,
  spike_temp  = 28,
  n_hours     = 96,
  spike_times_single = 24,
  spike_times_multi  = c(24, 48, 72)
)
val_T_c <- 24

# Analytical truth, given the simulation parameters (true_z, true_CTmax).
planted <- purrr::map(val_scens, planted_dose_from_trace,
                      z = true_z, CTmax_1hr = true_CTmax, T_c = val_T_c)

# Posterior predictions from the fitted beta-binomial workflow. The model
# expects minutes; convert each trace's time axis.
predicted <- purrr::map(val_scens, function(sc) {
  predict_heat_injury(
    trace       = dplyr::mutate(sc, time_h = time_h * 60),
    workflow    = wf_bb,
    target_surv = 0.5,
    T_c         = val_T_c,
    ndraws      = 300
```

Scenario	Planted HI (%)	Posterior median (%)	Lower (%)	Upper (%)	Truth in CrI?
Flat	0	0	0	0	TRUE
Single spike	24	22	20	24	TRUE
Multi spike	72	67	60	73	TRUE

```
  )
})

# Final HI per scenario: analytical truth vs posterior median + CrI.
val_table <- tibble::tibble(
  Scenario     = c("Flat", "Single spike", "Multi spike"),
  `Planted HI (%)` = c(tail(planted$flat$hi_cumulative,         1),
                       tail(planted$single_spike$hi_cumulative, 1),
                       tail(planted$multi_spike$hi_cumulative,  1)),
  `Posterior median (%)` = c(tail(predicted$flat$summary$hi_median,        1),
                             tail(predicted$single_spike$summary$hi_median, 1),
                             tail(predicted$multi_spike$summary$hi_median,  1)),
  `Lower (%)` = c(tail(predicted$flat$summary$hi_lower,        1),
                  tail(predicted$single_spike$summary$hi_lower, 1),
                  tail(predicted$multi_spike$summary$hi_lower,  1)),
  `Upper (%)` = c(tail(predicted$flat$summary$hi_upper,        1),
                  tail(predicted$single_spike$summary$hi_upper, 1),
                  tail(predicted$multi_spike$summary$hi_upper,  1))
)
val_table$`Truth in CrI?` <- with(val_table,
                                  `Lower (%)` <= `Planted HI (%)` &
                                  `Planted HI (%)` <= `Upper (%)`)
tinytable::tt(val_table, digits = 2, escape = TRUE)
```

The flat trace stays below T_c for its entire duration and so accrues zero HI, matching the analytical truth exactly. The single- and multi-spike traces deliver known doses analytically; `predict_heat_injury()` recovers each with the truth lying inside its credible interval. This sanity-check pattern — three traces with known doses, plus the recovery check — is what each new dataset's heat-injury prediction should pass before being trusted.

3 Case Study 1: Shrimp data

We now apply the full workflow described above — `standardize_data()` → `fit_4pl()` → `extract_tdt()` → `predict_heat_injury()` — to lethal-TDT data from brown shrimp (*Crangon crangon*). The data are 148 replicate trials spanning 30–33 °C and 5 min to 6 hours of exposure, with each replicate exposing a tank of individuals to a fixed temperature × duration combination on a specific date. The two grouping factors `Date` and `Tank` enter the model as random intercepts on `mid`, allowing day-to-day batch variation and tank-level cohort variation to be partitioned out of the dose-response surface.

Table S9: Summary of the shrimp lethal-TDT dataset after standardisation. Temperature centre is the grand mean used to mean-centre the temperature covariate.

n trials	Temperature range (°C)	Duration range (hr)	Median individuals/trial	Temperature centre (°C)
148	30–33	0.08–6	10	32

3.1 Data and standardisation

```
shrimp_raw <- readr::read_csv(
  here::here("data", "data_lethal_TDT_brown_shrimp.csv"),
  show_col_types = FALSE
)

shrimp_std <- standardize_data(
  data           = shrimp_raw,
  temp           = "Temperature_assay",
  duration       = "Duration_exposure_hours",
  n_total        = "N_individuals_after_trial",
  mortality      = "Mortality_after_trial",
  random_effects = c("Date", "Tank"),
  duration_unit  = "hours"
)
```

```
shrimp_summary <- tibble::tibble(
  `n trials`              = nrow(shrimp_std),
  `Temperature range (°C)` = paste(range(shrimp_std$temp), collapse = "–"),
  `Duration range (hr)`   = paste(round(range(shrimp_std$duration), 2),
                                  collapse = "–"),
  `Median individuals/trial` = stats::median(shrimp_std$n_total),
  `Temperature centre (°C)`  = attr(shrimp_std, "tdt_meta")$temp_mean,
  `n Dates`               = length(unique(shrimp_std$Date)),
  `n Tanks`               = length(unique(shrimp_std$Tank))
)
tinytable::tt(shrimp_summary, digits = 2, escape = TRUE)
```

3.2 Fitting the joint Bayesian 4PL

`fit_4pl()` is the package's high-level fitting wrapper. It takes the standardised data and configures the joint 4PL with sensible defaults: a beta-binomial likelihood, the disjoint-bounds reparameterisation that keeps the asymptotes identifiable, and a `temp_c` slope on each 4PL sub-parameter. We add random intercepts on `mid` for `Date` and `Tank` to absorb the day-to-day and tank-to-tank variation that would otherwise contaminate the dose-response surface.

Table S10: Sampling diagnostics for the shrimp 4PL fit. Healthy targets: Rhat < 1.01, ESS > 400, zero divergences.

rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	bfmi_min	rhat_pass	ess_pass
1	1474	2262	0	3	0.736	TRUE	TRUE

```
wf_shrimp <- fit_4pl(
  shrimp_std,
  random_effects = c("Date", "Tank"),
  chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.99, max_treedepth = 12),
  file    = file.path(models_dir, "fit_shrimp_lethal_4pl")
)
```

3.2.1 Sampling diagnostics

```
tinytable::tt(diagnose_tdt_fit(wf_shrimp), digits = 3, escape = TRUE)
```

The diagnostics table is a numerical pass/fail summary. The canonical visual check is the chain trace ("fuzzy caterpillar"): healthy chains overlap and explore the same posterior support; chains that drift, diverge, or hang in different regions signal a problem with the fit.

```
brms::mcmc_plot(
  wf_shrimp$fit,
  type     = "trace",
  variable = c("b_lowraw_Intercept", "b_upraw_Intercept",
               "b_logk_Intercept",   "b_mid_Intercept",
               "b_mid_temp_c",       "phi")
) +
  ggplot2::theme_classic(base_size = 11) +
  ggplot2::theme(legend.position = "bottom")
```

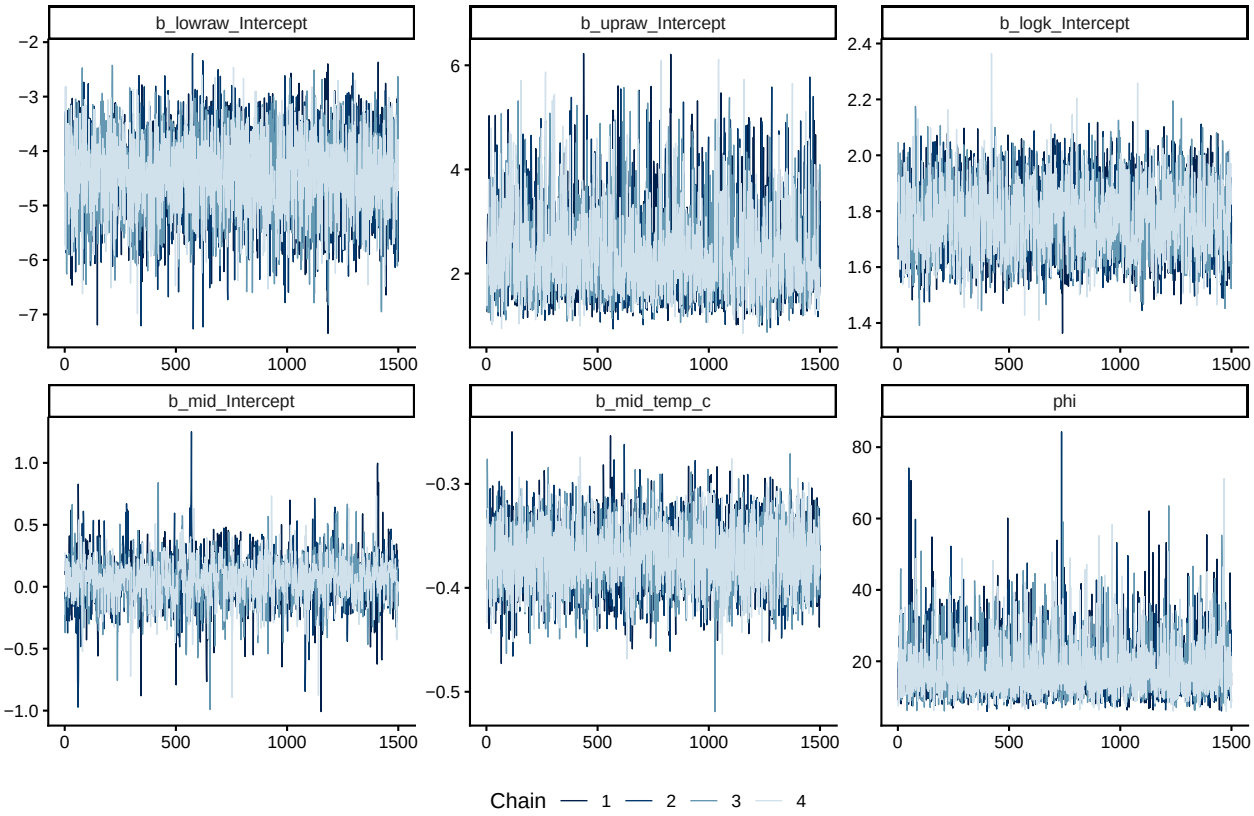


Figure S10: Posterior chain traces for the shrimp 4PL fixed-effect parameters and the beta-binomial dispersion ϕ. Each panel overlays four chains; well-mixed chains overlap throughout.

3.2.2 Posterior parameters on the natural scale

```
tinytable::tt(tdt_parameter_table(wf_shrimp), digits = 3, escape = TRUE)
```

3.3 Survival curves with observed overlay

`predict_survival_curves()` evaluates the fitted 4PL on a grid of assay temperatures and exposure durations and returns posterior survival probabilities. Overlaying the per-replicate observed proportions gives a visual check that the fitted curves track the data at every temperature.

```
psc_shrimp <- predict_survival_curves(
  wf_shrimp,
  temps     = sort(unique(shrimp_std$temp)),
  ndraws    = 500
)
plot_survival_curves(psc_shrimp, observed = shrimp_std) +
  ggplot2::labs(x = "Exposure duration (hours)") +
  ggplot2::coord_cartesian(xlim = c(0, 10))
```

Table S11: Posterior medians and 95% credible intervals for the shrimp 4PL fit, transformed back to the natural scale. `low` and `up` are the bottom and top asymptotes of the survival curve at the grand-mean temperature; `k` is the slope on $\log_{10}$ time; `mid` intercept is $\log_{10}$ LT50 in hours at the grand-mean temperature; `mid temp_c slope` is β_1, from which $z = -1/\beta_1$.

parameter	median	lower	upper
low (lower asymptote)	0.00691	0.00232	0.0219
up (upper asymptote)	0.95115	0.89232	0.9935
k (slope)	5.8533	4.73301	7.5048
mid intercept (at T_bar)	0.05556	-0.30394	0.4007
mid temp_c slope	-0.36919	-0.42587	-0.3126
z (°C)	2.7086	2.34815	3.199

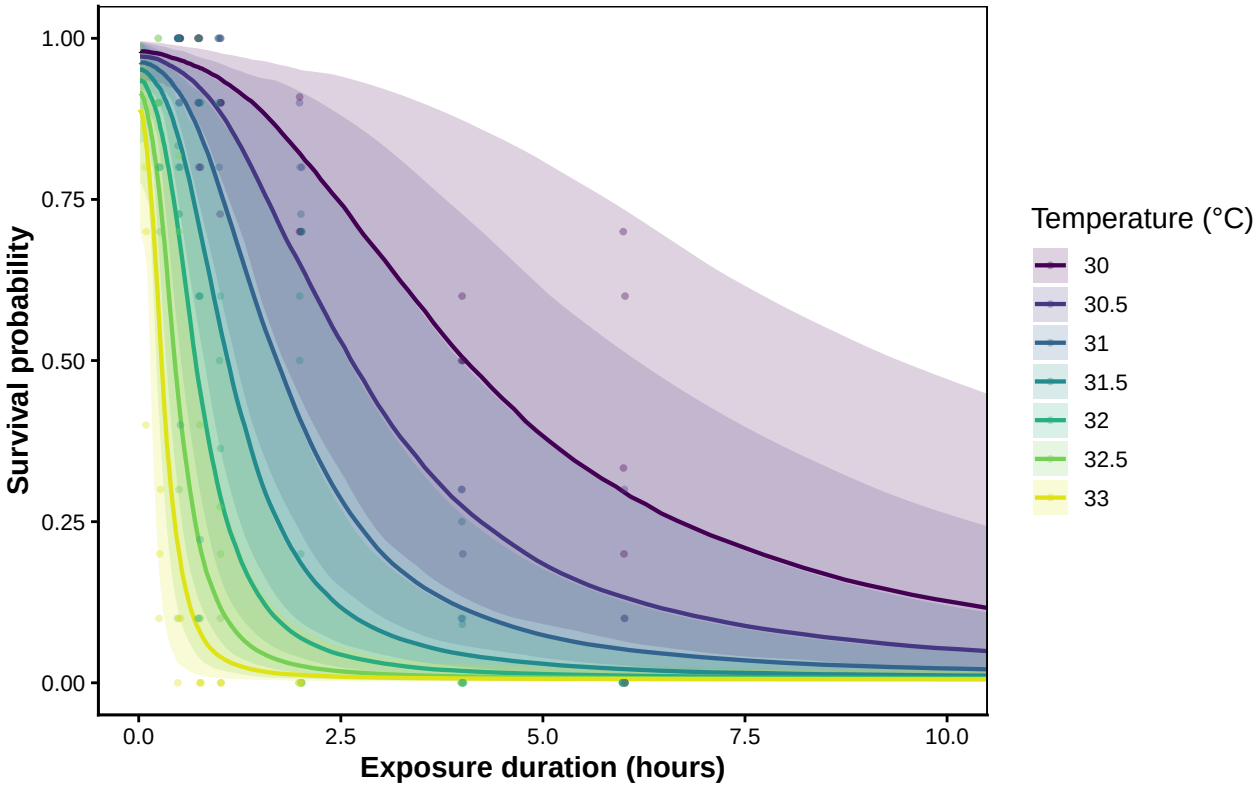


Figure S11: Posterior survival curves for brown shrimp at each assayed temperature, with 95% credible bands. Coloured points are observed per-replicate survival proportions.

3.4 TDT landscape

`derive_tdt_landscape()` evaluates the fitted 4PL on a dense (temperature, duration) grid and `plot_tdt_landscape()` renders it as a heatmap with overlaid contours at 25, 50, and 75% survival.

This is the two-dimensional view of the dose-response surface that the TDT line summarises with one number per temperature.

```
shrimp_dur_grid <- seq(min(shrimp_std$duration), max(shrimp_std$duration),
                       length.out = 120)
lsp_shrimp <- derive_tdt_landscape(wf_shrimp,
                                   duration_grid = shrimp_dur_grid,
                                   ndraws        = 500)
plot_tdt_landscape(lsp_shrimp, observed = shrimp_std)
```

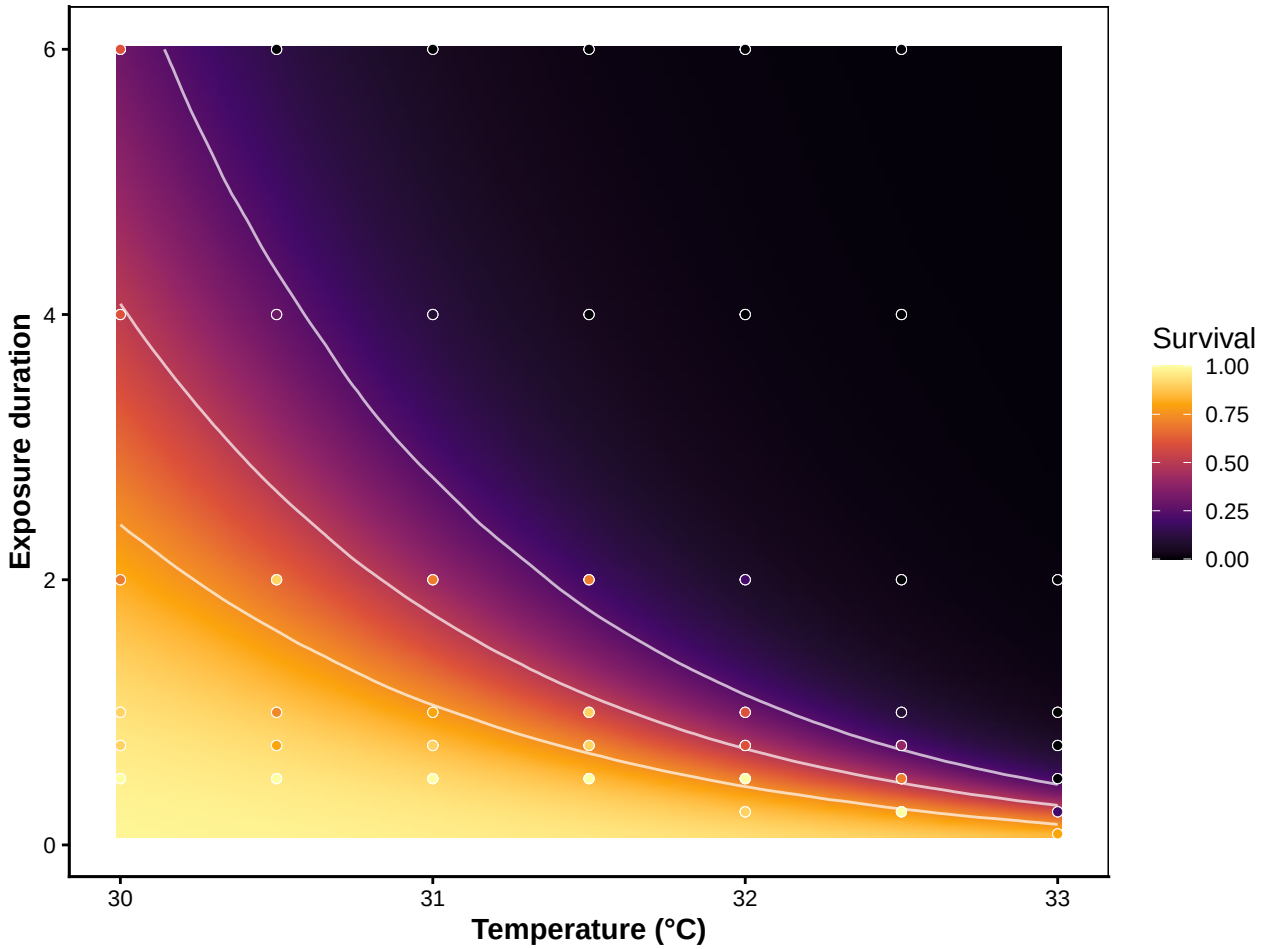


Figure S12: TDT landscape for brown shrimp: posterior median predicted survival across a 120 × 120 grid of assay temperatures × durations. White contours mark 25%, 50%, and 75% survival isolines. Overlaid points are raw observed proportions.

3.5 TDT line: time to the threshold vs temperature

The one-dimensional **TDT line** summarises the dose-response surface as the predicted time to the survival threshold at each temperature. `derive_tdt_curve()` builds it from the joint fit, and `plot_tdt_curve()` shows it on linear and $\log_{10}$ time axes side-by-side: the linear panel reads as

"hours of exposure half the animals tolerate" for ecological interpretation; the $\log_{10}$ panel linearises the relationship into the canonical TDT straight line, whose slope is $-1/z$. The default threshold is mid (the midpoint between the fitted asymptotes); pass `target_surv = "absolute"` for the classical absolute-t_{50} definition. For the shrimp data the two coincide because the asymptotes are anchored near 0 and 1.

```
ltx_shrimp <- derive_tdt_curve(
  wf_shrimp,
  temp_grid        = seq(min(shrimp_std$temp) - 1.5,
                         max(shrimp_std$temp) + 1.5, by = 0.05),
  ndraws           = 500,
  time_multiplier  = 1,
  output_time_unit = "h"
)
```

```
plot_tdt_curve(ltx_shrimp, panels = "both", colour = "#b2182b")
```

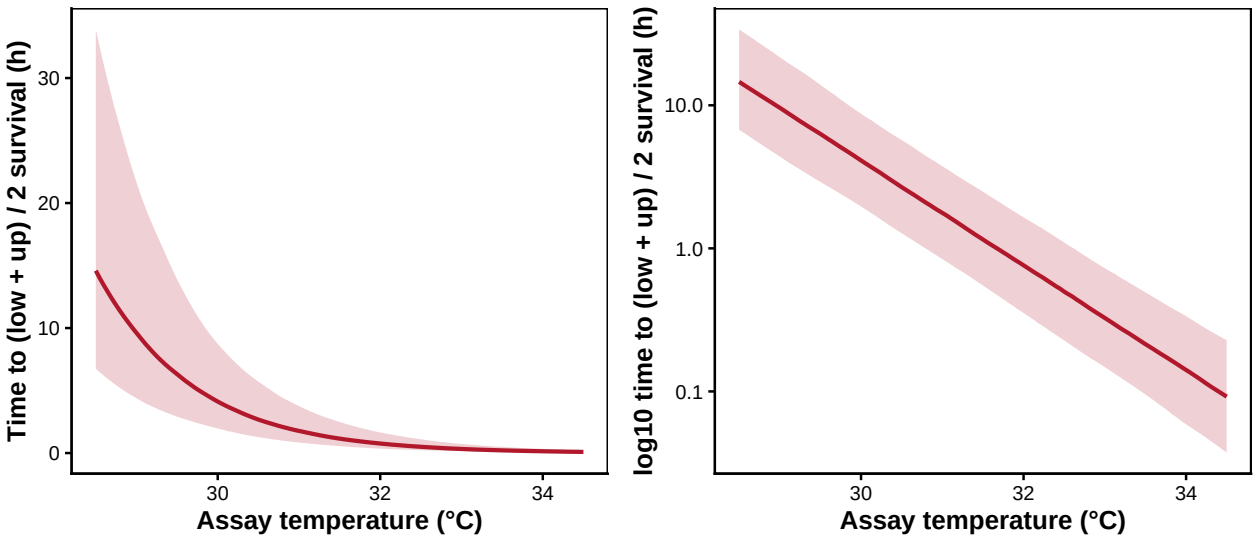


Figure S13: Brown shrimp posterior TDT line: predicted time to threshold vs assay temperature, with 95 % credible band. Linear-time panel on the left, $\log_{10}$-time panel on the right (slope is $-1/z$). Both panels share the same posterior.

3.6 Classical TDT quantities: z, $CT_{max_{1hr}}$, T_{crit}

`extract_tdt()` derives the classical thermal-tolerance summaries from a fitted 4PL workflow. It always returns z (the slope of $\log_{10}$ LT on temperature, derived by per-draw OLS on the TDT curve) and $CT_{max_{1hr}}$ (the temperature at which the threshold is reached after 1 hour of exposure, obtained by per-draw 4PL inversion). Passing `lethal = TRUE` additionally returns T_{crit} under the rate-multiplier definition; this only makes sense when the endpoint is mortality (see Equation S3), so we opt in for the shrimp data.

Table S12: Classical TDT summaries for brown shrimp, with full posterior uncertainty. z comes from a per-draw log-linear regression of $\log_{10}$ LT50 on temperature. $CT_{max_{1hr}}$ is the temperature at 50% survival after 1-hour exposure (4PL inversion per draw). T_{crit} is the temperature at which the TDT damage rate falls below r^* % HI per hour, with r^* sampled uniformly on $\log_{10}$ across $[0.1, 1]$ — the pooled posterior carries both parameter uncertainty and operational uncertainty in r^*.

Quantity	Median	Lower	Upper
z (°C per order of magnitude)	2.7	2.3	3.2
CTmax_1hr (°C)	31.7	30.8	32.7
T_crit (°C, r* in [0.1, 1] per hour)	25	22.8	26.7

```
et_shrimp <- extract_tdt(
  wf_shrimp,
  t_ref         = 60,
  TC_rate_range = c(0.1, 1),
  ndraws        = 500,
  lethal        = TRUE
)
```

```
shrimp_extract <- tibble::tribble(
  ~Quantity,                          ~Median, ~Lower, ~Upper,
  "z (°C per order of magnitude)",      et_shrimp$z$summary$z_median,
                                      et_shrimp$z$summary$z_lower,
                                      et_shrimp$z$summary$z_upper,
  "CTmax_1hr (°C)",                   et_shrimp$CTmax$summary$temp_median,
                                      et_shrimp$CTmax$summary$temp_lower,
                                      et_shrimp$CTmax$summary$temp_upper,
  "T_crit (°C, r* in [0.1, 1] per hour)",
                                      et_shrimp$T_crit$summary$temp_median,
                                      et_shrimp$T_crit$summary$temp_lower,
                                      et_shrimp$T_crit$summary$temp_upper
)
tinytable::tt(shrimp_extract, digits = 2, escape = TRUE)
```

```
z_draws_shrimp   <- tibble::tibble(temp = et_shrimp$z$draws$z)
z_q <- stats::quantile(z_draws_shrimp$temp,
                       c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
z_post_shrimp <- list(
  draws   = z_draws_shrimp,
  summary = tibble::tibble(temp_lower  = z_q[1],
                           temp_median = z_q[2],
                           temp_upper  = z_q[3])
)
```

```
patchwork::wrap_plots(
  plot_temperature_density(z_post_shrimp,
                           x_label = expression(italic(z)~("°C per decade of time"))),
  plot_temperature_density(et_shrimp$CTmax,
                           x_label = expression(CT[max[1*hr]]~("°C"))),
  plot_temperature_density(et_shrimp$T_crit,
                           x_label = expression(T[crit]~("°C"))),
  ncol = 3
)
```

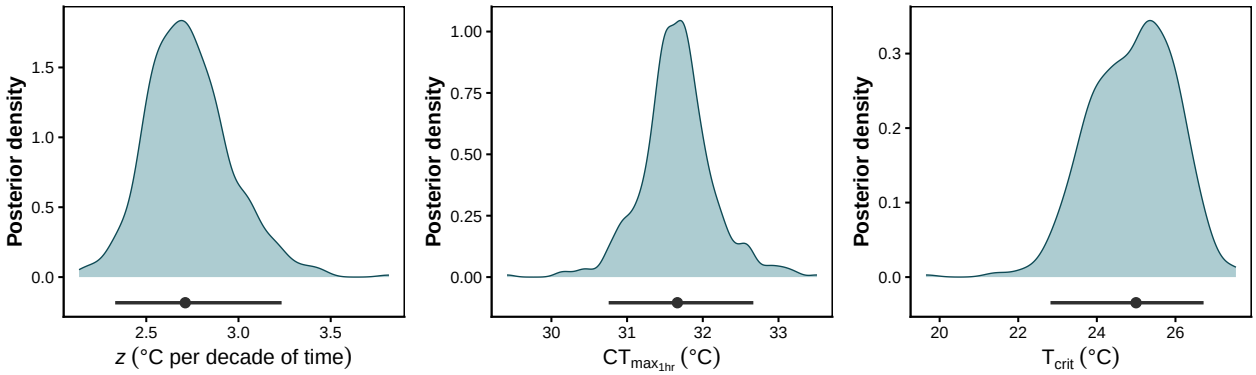


Figure S14: Posterior densities of z (left), $CT_{max_{1hr}}$ (middle), and T_{crit} (right) for brown shrimp. The horizontal bar below each density is the 95% credible interval; the point marks the posterior median. The ~5 °C gap between $CT_{max_{1hr}}$ and T_{crit} is the window between the onset of detectable damage accumulation and the LT50 temperature at 1 h.

3.7 Heat injury under reference traces

`predict_heat_injury()` projects the fitted dose-response surface through a temperature trace and returns cumulative heat injury (HI) and the resulting survival fraction at each time step. The damage threshold T_c below which injury does not accumulate is taken from the posterior median of T_{crit} extracted above, so the threshold is model-derived rather than hand-picked. We illustrate the function on four reference traces built by `make_temperature_scenarios()`: a flat baseline at the shrimp's natural holding temperature, a single one-hour spike, three one-hour spikes, and a four-day diurnal cycle that mimics a clustered coastal heatwave. The single-spike trace is calibrated as an analytical sanity check — its spike temperature is set to the posterior median of $CT_{max_{1hr}}$, so by definition the trace delivers approximately one LT50 dose by the end of the spike hour.

```
shrimp_T_c       <- et_shrimp$T_crit$summary$temp_median
shrimp_CTmax_med <- et_shrimp$CTmax$summary$temp_median

shrimp_scens <- make_temperature_scenarios(
  baseline   = 17,
  spike_temp = shrimp_CTmax_med,
  n_hours    = 96,
  spike_times_single = 24,
```

```
  spike_times_multi  = c(24, 48, 72),
  diurnal_n_days     = 4,
  diurnal_day_peaks  = c(22.0, 27.0, 30.0, 30.5),
  diurnal_night_temp = c(18.0, 19.5, 21.5, 22.0),
  diurnal_peak_fwhm  = 5,
  diurnal_noise_sd   = 0.3,
  diurnal_seed       = 1L
)

shrimp_hi <- purrr::map(shrimp_scens, function(sc) {
  predict_heat_injury(trace = sc, workflow = wf_shrimp,
                      T_c = shrimp_T_c, ndraws = 500)
})
```

Setting `repair = TRUE` adds a temperature-dependent Sharpe-Schoolfield repair kernel so injury accumulated at hot temperatures can be partially cleared during cooler intervals. Here we re-run only the two scenarios that accrue meaningful HI without repair — the multi-spike and the diurnal traces.

```
shrimp_repair_pars <- list(
  TA = 14065, TAL = 50000, TAH = 120000,
  TL = 10.5 + 273.15, TH = 22.5 + 273.15, TREF = 17 + 273.15,
  r_ref = 0.05
)

shrimp_hi_multi_rep <- predict_heat_injury(
  shrimp_scens$multi_spike, wf_shrimp, T_c = shrimp_T_c, ndraws = 500,
  repair = TRUE, repair_pars = shrimp_repair_pars
)
shrimp_hi_diurnal_rep <- predict_heat_injury(
  shrimp_scens$diurnal, wf_shrimp, T_c = shrimp_T_c, ndraws = 500,
  repair = TRUE, repair_pars = shrimp_repair_pars
)
```

```
plot_temperature_scenarios(shrimp_scens, T_c = shrimp_T_c)
```

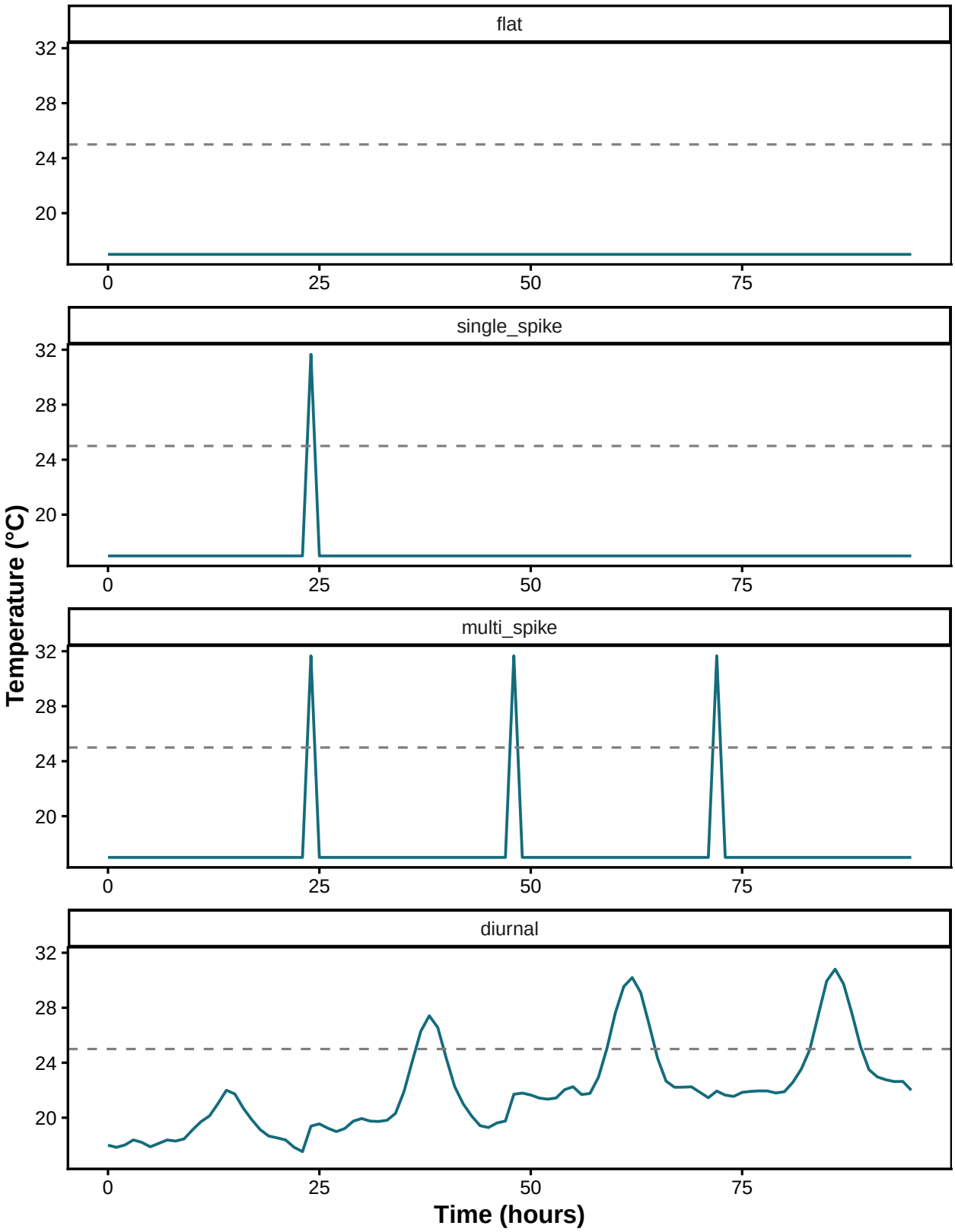


Figure S15: The four reference temperature traces used for the shrimp heat-injury prediction. Baseline 17 °C is the natural holding temperature; the 1-hour spikes are set to the posterior median $CT_{max_{1hr}}$ so each spike delivers approximately one LT50 dose. The diurnal scenario is a clustered 4-day heatwave: a cool day below T_{crit} followed by three progressively hotter days. The dashed line marks the damage threshold $T_c = T_{crit}$.

```
plot_heat_injury(shrimp_hi$single_spike)
```

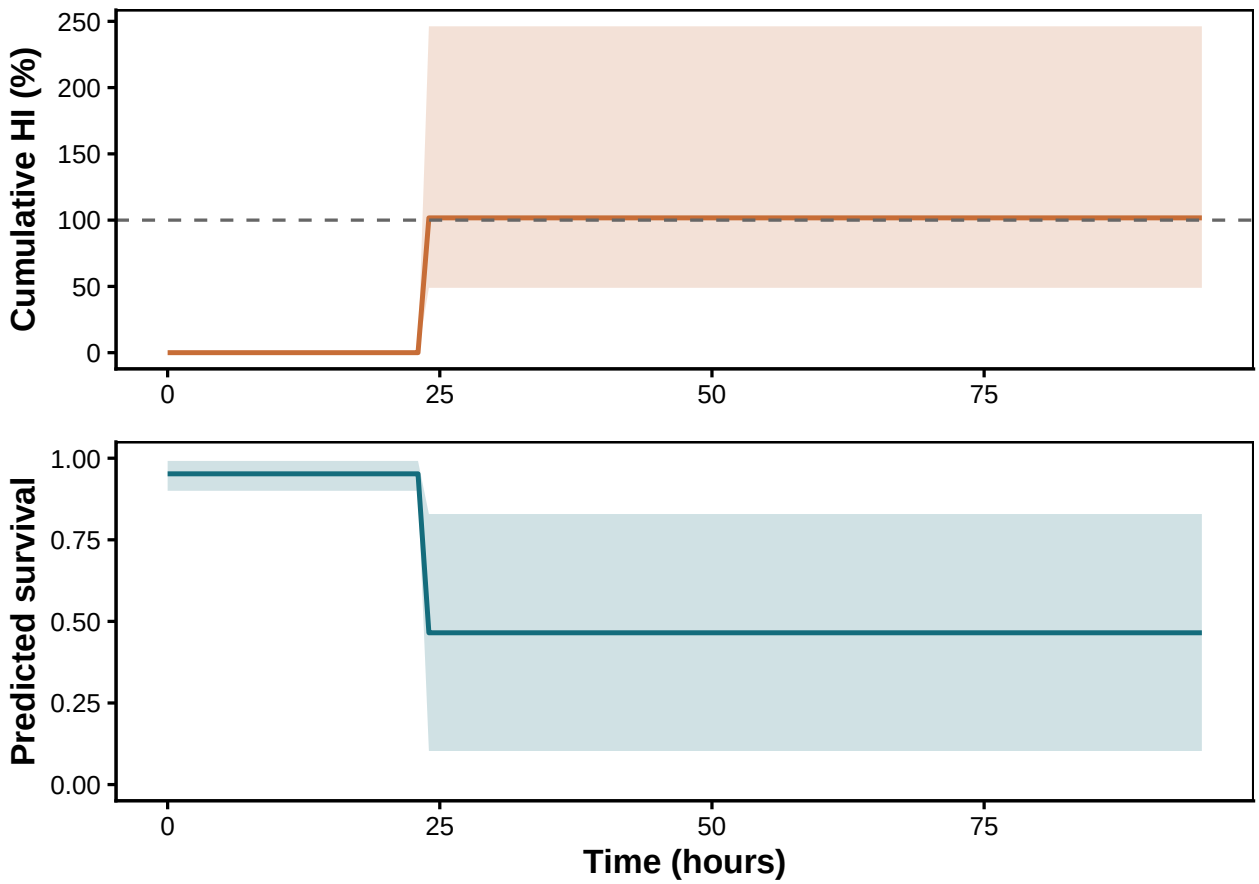


Figure S16: Cumulative heat injury (top) and predicted survival (bottom) under the **single-spike** trace, **no repair**. Because the spike sits at the posterior median $CT_{max_{1hr}}$, ~100% LT50-dose accrues by the end of the spike hour, producing ~50% mortality.

```
plot_heat_injury(shrimp_hi$multi_spike)
```

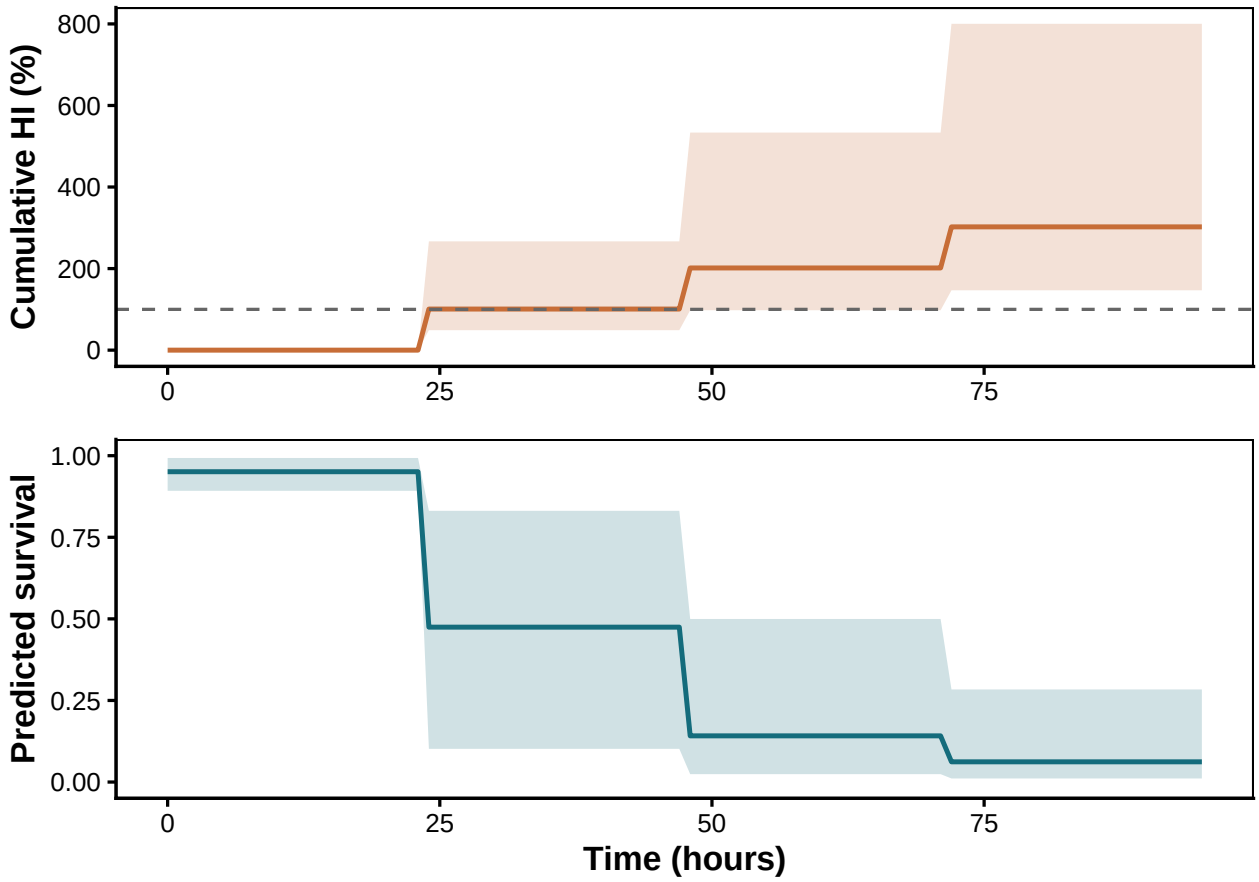


Figure S17: **Multi-spike** trace, **no repair**. Three 1-hour spikes at $CT_{max_{1hr}}$ deliver ~3 × LT50-dose, driving cumulative HI well past 100% and survival to near zero.

```
plot_heat_injury(shrimp_hi_multi_rep)
```

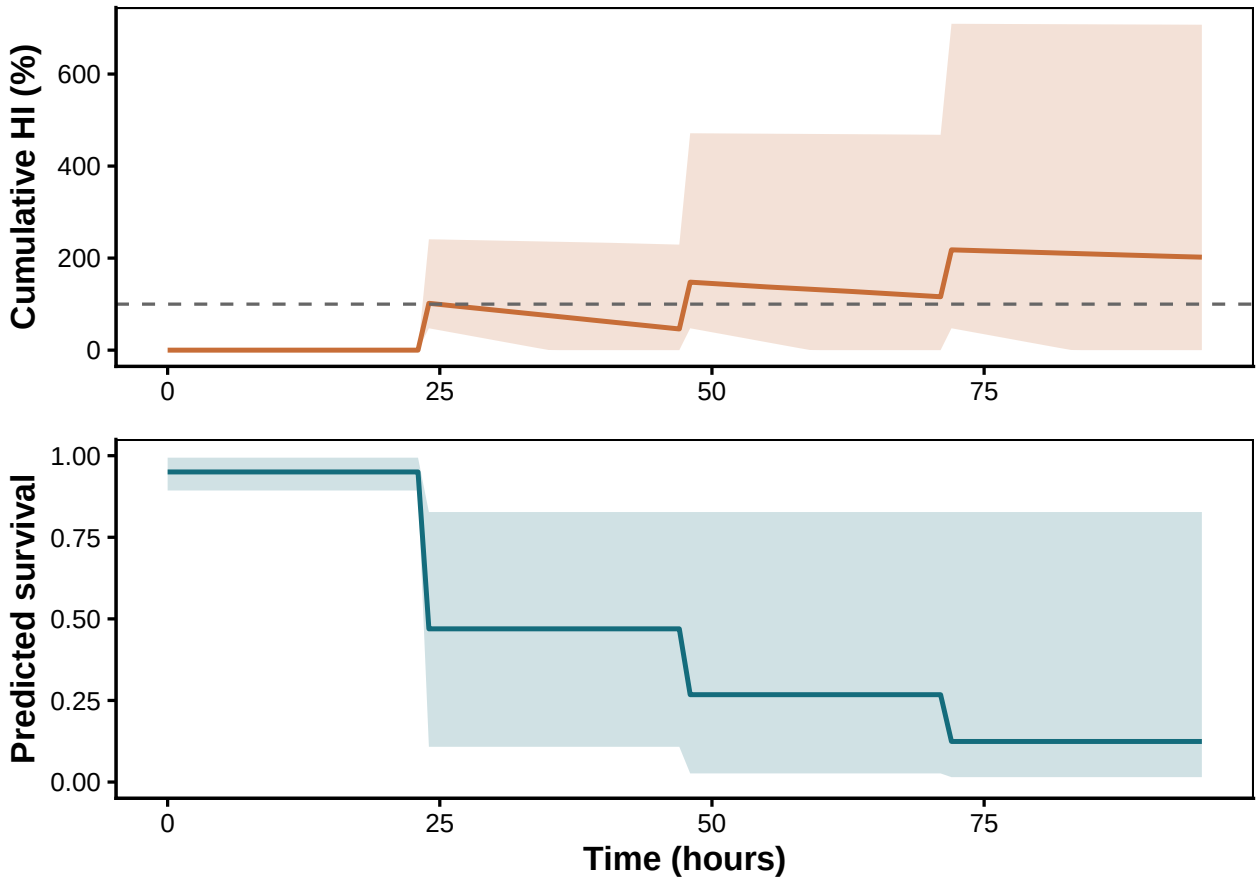


Figure S18: Same multi-spike trace, **with Sharpe-Schoolfield repair active**. Repair clears most of the dose between spikes, stopping the runaway accumulation seen without repair.

The diurnal scenario (Figure S19, Figure S20) is closer to a real coastal record. Day 1 stays below T_{crit} and accrues no injury; days 2–4 each contribute an injury bout that scales with the daily peak. Without repair, cumulative HI is well past 100 % by the end of day 4 and predicted survival has dropped to ~20 %. With repair on, off-peak hours and the cool first day act as recovery phases; the final trajectory is set by the net balance between daily accrual at sub-$CT_{max_{1hr}}$ peaks and overnight/cool-day repair.

```
plot_heat_injury(shrimp_hi$diurnal)
```

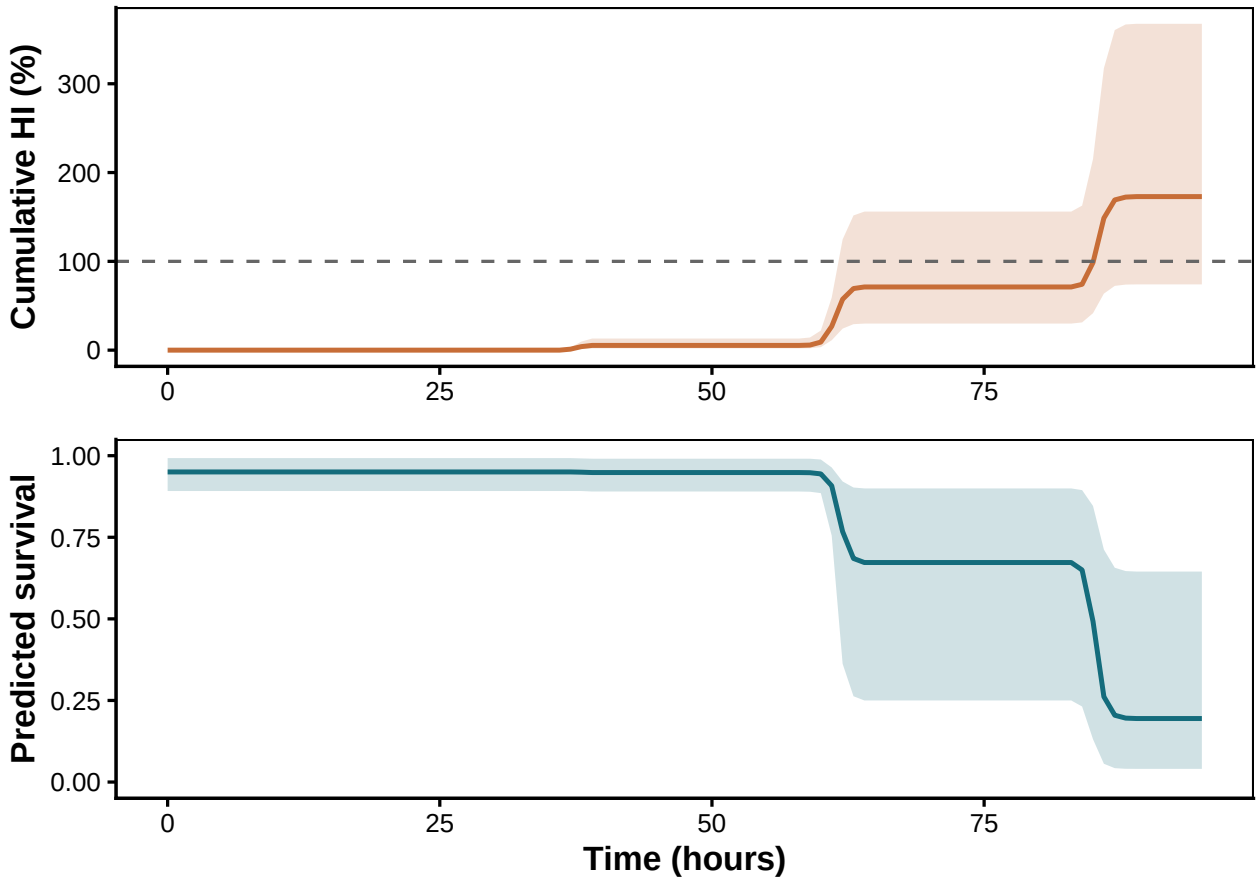


Figure S19: **4-day diurnal heatwave** trace, **no repair**. The cool first day flat-lines below T_{crit}; days 2–4 each accrue an injury bout scaling with the daily peak.

```
plot_heat_injury(shrimp_hi_diurnal_rep)
```

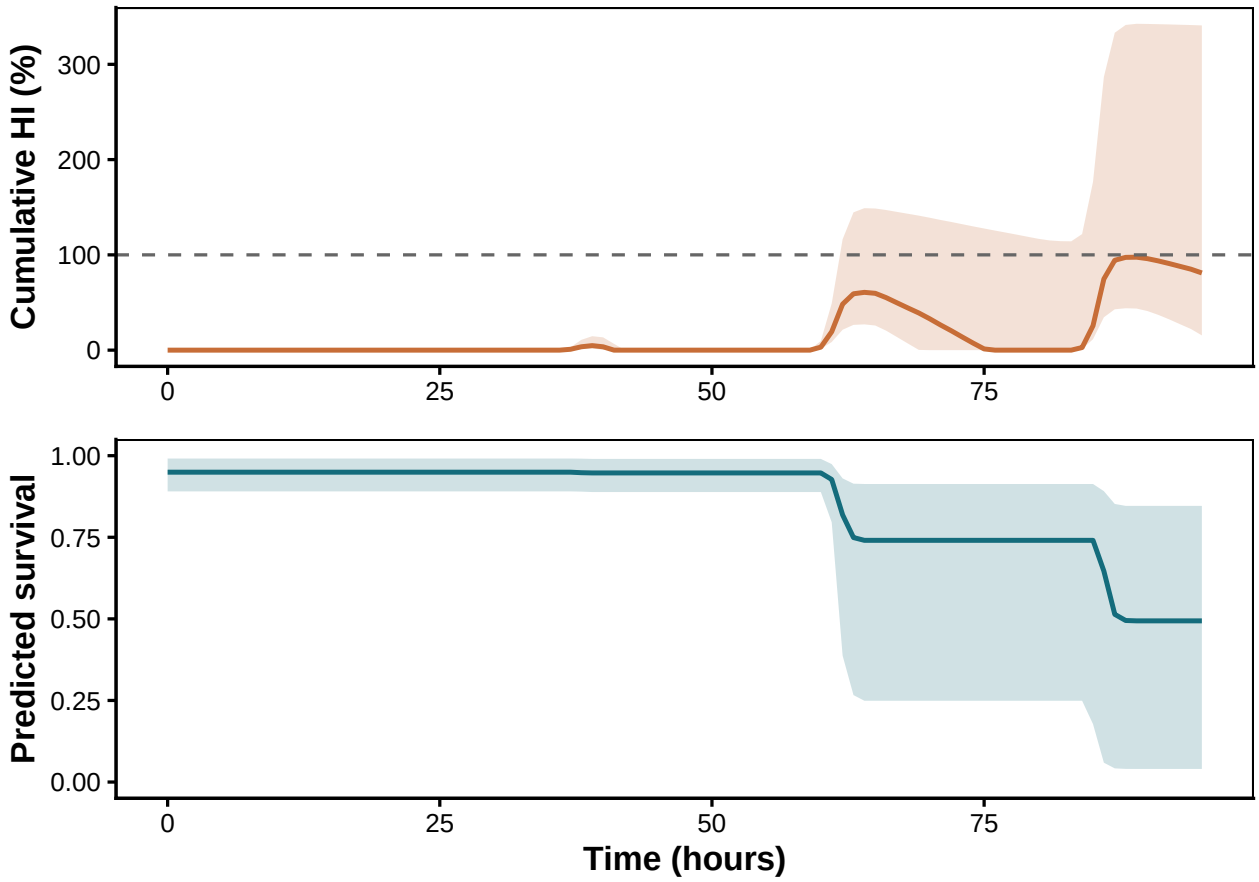


Figure S20: Same 4-day heatwave trace, **with Sharpe-Schoolfield repair active**. The cool first day and overnight troughs now act as recovery phases.

```
final_row <- function(hi) {
  s <- tail(hi$summary, 1)
  tibble::tibble(
    `HI median (%)` = round(s$hi_median, 1),
    `HI lower`      = round(s$hi_lower,  1),
    `HI upper`      = round(s$hi_upper,  1),
    `Survival median`  = round(s$surv_median, 3),
    `Survival lower`   = round(s$surv_lower,  3),
    `Survival upper`   = round(s$surv_upper,  3)
  )
}

shrimp_hi_table <- dplyr::bind_rows(
  final_row(shrimp_hi$flat)         |> dplyr::mutate(Scenario = "Flat",
                                                     Repair = "None"),
  final_row(shrimp_hi$single_spike) |> dplyr::mutate(Scenario = "Single spike",
                                                     Repair = "None"),
  final_row(shrimp_hi$multi_spike)  |> dplyr::mutate(Scenario = "Multi spike",
                                                     Repair = "None"),
```

Table S13: Final cumulative HI and predicted survival under each scenario for brown shrimp, posterior medians with 95% credible intervals.

Scenario	Repair	HI median (%)	HI lower	HI upper	Survival median	Survival lowe
Flat	None	0.0	0.0	0.0	0.951	0.894
Single spike	None	101.7	48.9	246.2	0.465	0.103
Multi spike	None	302.3	146.8	800.3	0.062	0.011
Multi spike	Sharpe-Schoolfield	202.1	0.0	707.2	0.124	0.015
Diurnal (7 days)	None	172.9	74.0	367.5	0.195	0.040
Diurnal (7 days)	Sharpe-Schoolfield	81.0	15.5	340.8	0.494	0.040

```
  final_row(shrimp_hi_multi_rep)     |> dplyr::mutate(Scenario = "Multi spike",
                                                     Repair = "Sharpe-Schoolfield"),
  final_row(shrimp_hi$diurnal)       |> dplyr::mutate(Scenario = "Diurnal (7 days)",
                                                     Repair = "None"),
  final_row(shrimp_hi_diurnal_rep)   |> dplyr::mutate(Scenario = "Diurnal (7 days)",
                                                     Repair = "Sharpe-Schoolfield")
) |>
  dplyr::select(Scenario, Repair, dplyr::everything())

tinytable::tt(shrimp_hi_table, escape = TRUE)
```

The shrimp case study covers the complete loop: raw counts → fitted joint 4PL → classical TDT quantities with calibrated uncertainty → heat-injury predictions under fluctuating field traces, with and without repair. Every quantity inherits the posterior of the same single fit. The same machinery applies to any TDT-style proportion dataset by changing only the column-name arguments passed to `standardize_data()`.

3.8 Sublethal time-to-knockdown data

The shrimp data also include a **sublethal** endpoint, time to loss of response to touch, where each cup contributes one observation of when its individuals were knocked down rather than a binary survival count at a pre-specified duration. The classical analysis fits a log-linear regression of $\log_{10}$(time to event) on temperature and reads $z = -1/\beta_1$ and $CT_{max_{1hr}} = \bar{T} + (\log_{10} 60 - \beta_0)/\beta_1$ off the coefficients (Jørgensen *et al.* 2021; Ørsted *et al.* 2022). Below we reproduce that classical analysis as a hierarchical Bayesian model, then re-cast the same observations as proportion-knocked-down counts and fit the joint 4PL on them (Ørsted *et al.* 2024), and finally compare the two pipelines' TDT lines and posterior summaries.

3.8.1 Data preparation

The raw sublethal sheet records one row per cup with the time when its individuals lost response to touch. We parse the time-of-day strings into minutes, restrict to non-excluded rows, and attach

grouping factors for date, tank, and cup.

```
sublethal_raw <- readr::read_csv(
  here::here("data", "data_sublethal_TDT_brown_shrimp.csv"),
  show_col_types = FALSE
)

parse_time_min <- function(s) {
  t <- lubridate::parse_date_time(s, orders = c("I:M:S p", "H:M:S"),
                                  quiet = TRUE)
  lubridate::hour(t) * 60 + lubridate::minute(t) + lubridate::second(t) / 60
}

tdt_sub <- sublethal_raw |>
  dplyr::filter(Exclude == "no") |>
  dplyr::mutate(
    assay_temp      = as.numeric(Assay_temperature),
    date_experiment = factor(as.character(lubridate::dmy(Date))),
    tank_ID         = factor(Tank),
    cup_ID          = factor(paste0(Trial_ID, "_", Sample)),
    start_min       = parse_time_min(Starting_time),
    end_min         = parse_time_min(Time_to_death),
    time_to_event   = end_min - start_min          # minutes
  ) |>
  dplyr::filter(is.finite(time_to_event), time_to_event > 0) |>
  dplyr::mutate(
    log10_time = log10(time_to_event),
    temp_mean  = mean(assay_temp),
    temp_c     = assay_temp - temp_mean
  )
```

Table S14 summarises the sublethal panel.

```
tdt_sub |>
  dplyr::group_by(assay_temp) |>
  dplyr::summarise(
    `N cups`      = dplyr::n(),
    `Median (min)` = round(stats::median(time_to_event), 2),
    `Lower 95-pct (min)`  = round(stats::quantile(time_to_event, 0.025), 2),
    `Upper 95-pct (min)`  = round(stats::quantile(time_to_event, 0.975), 2),
    .groups       = "drop"
  ) |>
  dplyr::rename(`Temperature (°C)` = assay_temp) |>
  tinytable::tt(escape = TRUE)
```

Table S14: Brown-shrimp sublethal time-to-knockdown panel: number of cups, median exposure time to knockdown, and 95% range, per assay temperature.

Temperature (°C)	N cups	Median (min)	Lower 95-pct (min)	Upper 95-pct (min)
31.0	60	158.99	12.69	348.03
31.5	59	63.82	3.55	197.16
32.0	60	60.15	1.51	176.85
32.5	60	6.41	1.17	77.03
33.0	60	1.24	0.31	19.77

3.8.2 Linear Bayesian time-to-event model

The classical sublethal TDT analysis fits a hierarchical log-linear regression: $\log_{10}(\text{time to event}) = \beta_0 + \beta_1(T - \bar{T}) + \text{random effects} + \varepsilon$, with random intercepts on the three experimental grouping factors (`date_experiment`, `tank_ID`, `cup_ID`). The two scalar TDT quantities follow as transformations of the posterior of (β_0, β_1):

$$z = -\frac{1}{\beta_1}, \qquad CT_{max_{1hr}} = \bar{T} + \frac{\log_{10}(60) - \beta_0}{\beta_1}.$$

```
priors_sub_lin <- c(
  brms::prior(normal(2, 1.5),    class = "Intercept"),
  brms::prior(normal(-1, 1),     class = "b"),
  brms::prior(exponential(2),    class = "sd"),
  brms::prior(exponential(2),    class = "sigma")
)

fit_sub_lin <- brms::brm(
  brms::bf(log10_time ~ temp_c +
           (1 | date_experiment) + (1 | tank_ID) + (1 | cup_ID)),
  data       = tdt_sub,
  prior      = priors_sub_lin,
  chains     = 4,
  cores      = 4,
  iter       = 8000,
  warmup     = 4000,
  init       = 0,
  control    = list(adapt_delta = 0.995, max_treedepth = 14),
  backend    = "cmdstanr",
  seed       = 123,
  silent     = 2,
  refresh    = 0,
  file       = file.path(models_dir, "fit_shrimp_sublethal_linear"),
  file_refit = "on_change"
)
```

Table S15: Posterior summaries from the linear Bayesian sublethal time-to-event model: thermal sensitivity z and the critical temperature at the 1-hour reference duration $CT_{max_{1hr}}$, derived from the regression coefficients.

Quantity	Median	Lower	Upper
z (°C)	1.1	0.997	1.23
CTmax at 1 hr (°C)	31.5	31.207	31.75

Posterior summaries on the natural scale follow from the regression coefficients and the mean-centring temperature:

```
post_sub_lin <- posterior::as_draws_df(fit_sub_lin) |> as.data.frame()
temp_mean_sub <- mean(tdt_sub$assay_temp)

z_lin_draws       <- -1 / post_sub_lin$b_temp_c
CTmax_1hr_lin_draws <- temp_mean_sub +
  (log10(60) - post_sub_lin$b_Intercept) / post_sub_lin$b_temp_c

summarise_post <- function(x) {
  q <- stats::quantile(x, c(0.025, 0.5, 0.975), na.rm = TRUE, names = FALSE)
  tibble::tibble(median = q[2], lower = q[1], upper = q[3])
}
```

```
tibble::tibble(
  Quantity     = c("z (°C)", "CTmax at 1 hr (°C)"),
  Median       = c(stats::median(z_lin_draws),
                   stats::median(CTmax_1hr_lin_draws)),
  Lower        = c(stats::quantile(z_lin_draws, 0.025, names = FALSE),
                   stats::quantile(CTmax_1hr_lin_draws, 0.025, names = FALSE)),
  Upper        = c(stats::quantile(z_lin_draws, 0.975, names = FALSE),
                   stats::quantile(CTmax_1hr_lin_draws, 0.975, names = FALSE))
) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

3.8.3 Joint 4PL on proportion knocked-down

Following Ørsted *et al.* (2024), the same time-to-knockdown observations can be re-cast as proportion-alive counts: at each assay temperature, the fraction of cups not yet knocked down by time t is a cumulative survival curve, the canonical 4PL input. For each (date × tank × temperature) trial we count cups still responding at a grid of evaluation times, then feed the resulting $(n_{\text{total}}, n_{\text{alive}})$ panel through `standardize_data()` and `fit_4pl()`.

```
ttd_range <- range(tdt_sub$time_to_event, na.rm = TRUE)
eval_times_min <- 10 ^ seq(log10(max(0.2, ttd_range[1] * 0.5)),
```

```
                            log10(ttd_range[2] * 1.5),
                            length.out = 18)

sub_4pl_panel <- tdt_sub |>
  dplyr::group_by(date_experiment, tank_ID, assay_temp) |>
  dplyr::summarise(n_total = dplyr::n(),
                   ttds    = list(time_to_event),
                   .groups = "drop") |>
  purrr::pmap_dfr(function(date_experiment, tank_ID, assay_temp,
                           n_total, ttds) {
    tibble::tibble(
      date_experiment = date_experiment,
      tank_ID         = tank_ID,
      assay_temp      = assay_temp,
      duration_min    = eval_times_min,
      n_total         = n_total,
      n_alive         = vapply(eval_times_min,
                               function(t) sum(ttds > t), integer(1))
    )
  })

std_sub_4pl <- standardize_data(
  sub_4pl_panel,
  temp           = "assay_temp",
  duration       = "duration_min",
  n_total        = "n_total",
  n_surv         = "n_alive",
  random_effects = c("date_experiment", "tank_ID"),
  duration_unit  = "minutes"
)
```

```
wf_sub_4pl <- fit_4pl(
  std_sub_4pl,
  random_effects = c("date_experiment", "tank_ID"),
  chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  silent  = 2, refresh = 0,
  file    = file.path(models_dir, "fit_shrimp_sublethal_4pl"),
  file_refit = "on_change"
)
```

The fitted survival curves below are on the same axes as the lethal analysis — proportion still alive vs exposure duration — with the aggregated per-trial proportions overlaid.

```
psc_sub_4pl <- predict_survival_curves(
  wf_sub_4pl,
  temps  = sort(unique(std_sub_4pl$temp)),
  ndraws = 500
```

```
)
plot_survival_curves(psc_sub_4pl, observed = std_sub_4pl) +
  ggplot2::labs(x = "Exposure duration (minutes)",
                y = "Proportion of live individuals") +
  ggplot2::coord_cartesian(xlim = c(0, 600))
```

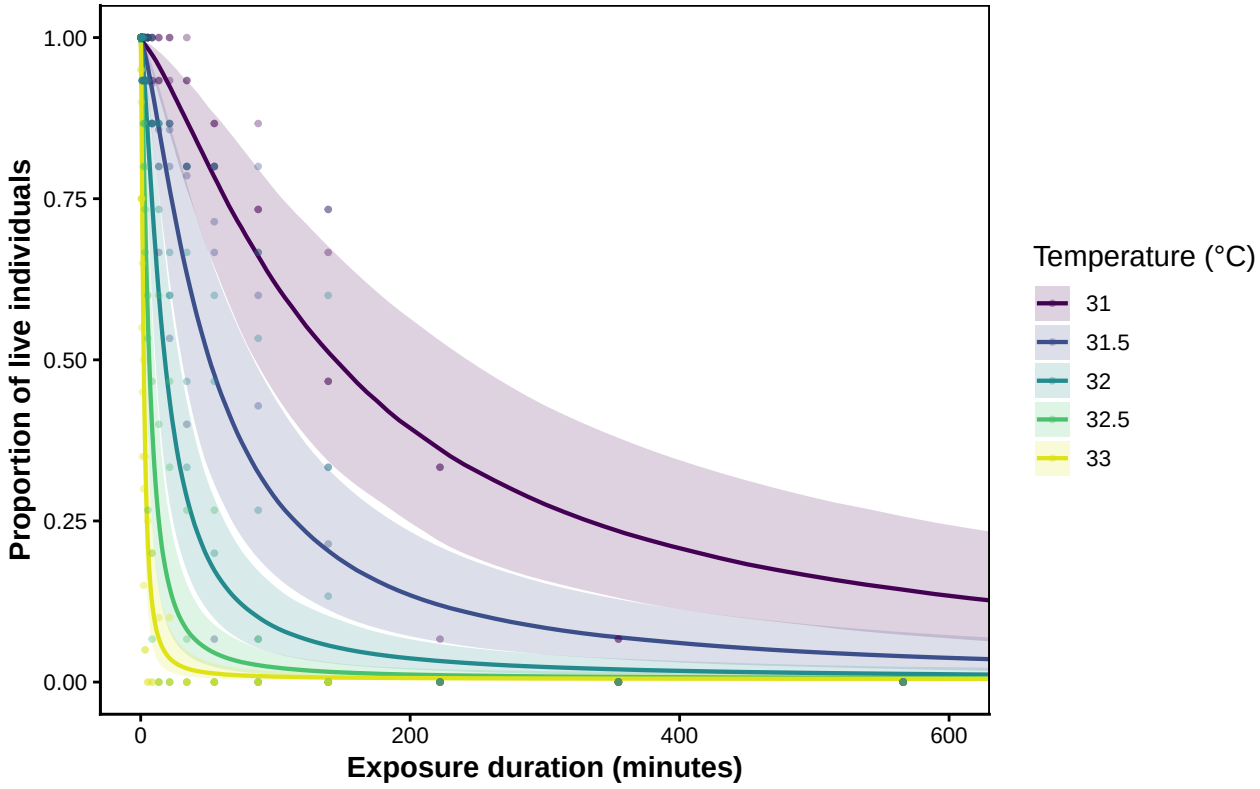


Figure S21: Posterior 4PL survival ('proportion still alive') curves for the brown-shrimp sublethal knockdown data at each assay temperature, with 95 % credible bands. Points are the aggregated proportion still alive per (date × tank × temperature) trial at each evaluation time.

The 4PL aggregation pools cups within each (date × tank × temperature) trial, so `cup_ID` is no longer needed as a grouping variable — the cup-level dispersion is absorbed by the beta-binomial precision parameter ϕ. The 4PL therefore uses two random intercepts (`date_experiment`, `tank_ID`) — one fewer than the cup-level linear fit above.

We then call `extract_tdt()` to summarise z and $CT_{max_{1hr}}$. We leave `lethal = FALSE` because the knockdown endpoint is sublethal: the fitted z measures the slope of performance reduction rather than damage accumulation, and the rate-multiplier T_{crit} formula is not interchangeable between the two (see the callout in Section 2.0.4).

```
et_sub_4pl <- extract_tdt(
  wf_sub_4pl,
  t_ref           = 60,
```

Table S16: Posterior summaries from the joint Bayesian 4PL on proportion-knocked-down data: z and $CT_{max_{1hr}}$. T_{crit} is not reported for sublethal endpoints (see Section 2.0.4).

Quantity	Median	Lower	Upper
z (°C)	1.11	1.02	1.23
CTmax at 1 hr (°C)	31.43	31.19	31.65

```
  time_multiplier = 1,
  ndraws          = 1000
)
```

```
tibble::tribble(
  ~Quantity,             ~Median,                                ~Lower,
  "z (°C)",              et_sub_4pl$z$summary$z_median,          et_sub_4pl$z$summary$z_lower,
  "CTmax at 1 hr (°C)",  et_sub_4pl$CTmax$summary$temp_median, et_sub_4pl$CTmax$summary$temp_lo
) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

3.8.4 Comparing the two sublethal pipelines

The two pipelines analyse the same knockdown observations through different statistical structures: the linear model regresses individual log-times on temperature, while the 4PL fits a sigmoidal cumulative-knockdown curve at each temperature and reads $\log_{10}$ LT50 from each posterior draw. Their posteriors of z and $CT_{max_{1hr}}$ should agree to within the data's information content.

```
temp_grid_sub <- seq(min(tdt_sub$assay_temp) - 0.25,
                     max(tdt_sub$assay_temp) + 0.25, by = 0.05)

lin_grid <- tibble::tibble(temp_c = temp_grid_sub - temp_mean_sub)
lin_lp   <- brms::posterior_linpred(fit_sub_lin, newdata = lin_grid,
                                    re_formula = NA)
lin_pred <- tibble::tibble(
  assay_temp     = temp_grid_sub,
  log10_time_med = apply(lin_lp, 2, stats::median),
  log10_time_lo  = apply(lin_lp, 2, stats::quantile, 0.025, names = FALSE),
  log10_time_hi  = apply(lin_lp, 2, stats::quantile, 0.975, names = FALSE),
  method         = "Linear Bayesian"
)

ltx_sub <- derive_tdt_curve(
  wf_sub_4pl,
  temp_grid       = temp_grid_sub,
  ndraws          = 1000,
  time_multiplier = 1
```

```
)

pl_4pl_pred <- ltx_sub$summary |>
  dplyr::transmute(
    assay_temp     = temp,
    log10_time_med = log10(duration_median),
    log10_time_lo  = log10(duration_lower),
    log10_time_hi  = log10(duration_upper),
    method         = "Joint 4PL (proportion knocked-down)"
  )

tdt_lines_sub <- dplyr::bind_rows(lin_pred, pl_4pl_pred) |>
  dplyr::mutate(method = factor(method,
                                levels = c("Linear Bayesian",
                                           "Joint 4PL (proportion knocked-down)")))
```

```
ggplot2::ggplot(tdt_lines_sub,
                ggplot2::aes(x = assay_temp, y = log10_time_med,
                             colour = method, fill = method)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = log10_time_lo,
                                    ymax = log10_time_hi),
                       alpha = 0.18, colour = NA) +
  ggplot2::geom_line(linewidth = 1) +
  ggplot2::geom_jitter(
    data = tdt_sub,
    ggplot2::aes(x = assay_temp, y = log10_time),
    inherit.aes = FALSE,
    width = 0.05, height = 0,
    alpha = 0.35, size = 1.2, colour = "grey30"
  ) +
  ggplot2::scale_colour_manual(values = c("#2c7fb8", "#d95f02")) +
  ggplot2::scale_fill_manual(values   = c("#2c7fb8", "#d95f02")) +
  ggplot2::labs(x = "Assay temperature (°C)",
                y = expression(log[10]~"(time to 50% knockdown, min)"),
                colour = NULL, fill = NULL) +
  ggplot2::theme_classic() +
  ggplot2::theme(legend.position = "top")
```

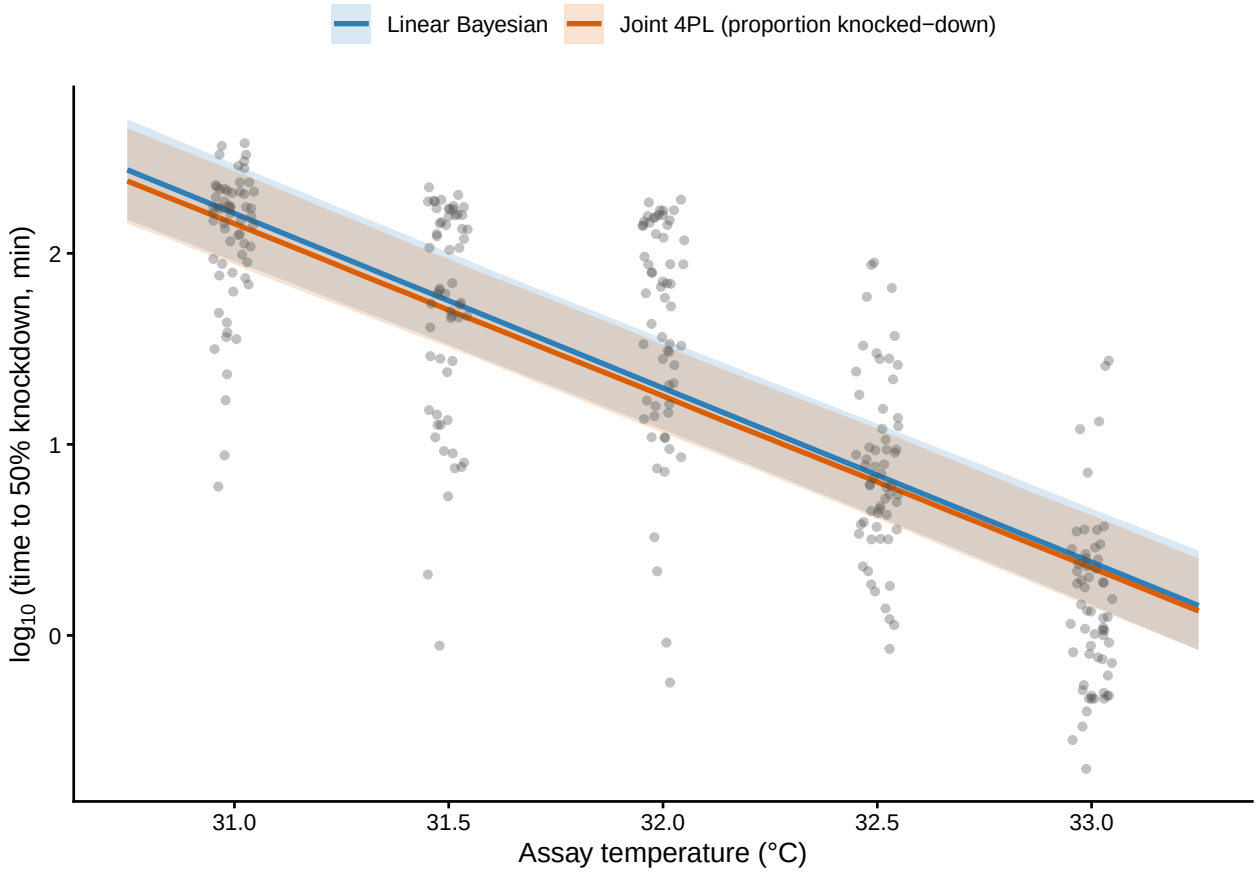


Figure S22: Posterior $\log_{10}$(time to 50% knockdown) versus assay temperature, from the linear Bayesian model (blue) and the joint 4PL on proportion-knocked-down data (orange). Shaded ribbons are 95% credible intervals; grey points are the individual observed knockdown times. The two pipelines analyse the *same* data and produce TDT lines that overlap across the assay range.63

Table S17: Side-by-side posterior summaries of z and $CT_{max_{1hr}}$ from the linear Bayesian time-to-event model and the joint 4PL on proportion-knocked-down data. Both fits use the same sublethal observations and the same hierarchical grouping structure (date and tank; the linear fit additionally carries a per-cup random intercept that is absorbed into `n_total` in the 4PL aggregation).

Quantity	Linear median	Linear lower	Linear upper	4PL median	4PL lower	4PL upper
z (°C)	1.1	0.997	1.23	1.11	1.02	1.23
CTmax at 1 hr (°C)	31.5	31.207	31.75	31.43	31.19	31.65

```
compare_sub <- tibble::tibble(
  Quantity = c("z (°C)", "CTmax at 1 hr (°C)"),
  `Linear median`  = c(stats::median(z_lin_draws),
                       stats::median(CTmax_1hr_lin_draws)),
  `Linear lower`   = c(stats::quantile(z_lin_draws, 0.025, names = FALSE),
                       stats::quantile(CTmax_1hr_lin_draws, 0.025, names = FALSE)),
  `Linear upper`   = c(stats::quantile(z_lin_draws, 0.975, names = FALSE),
                       stats::quantile(CTmax_1hr_lin_draws, 0.975, names = FALSE)),
  `4PL median`     = c(et_sub_4pl$z$summary$z_median,
                       et_sub_4pl$CTmax$summary$temp_median),
  `4PL lower`      = c(et_sub_4pl$z$summary$z_lower,
                       et_sub_4pl$CTmax$summary$temp_lower),
  `4PL upper`      = c(et_sub_4pl$z$summary$z_upper,
                       et_sub_4pl$CTmax$summary$temp_upper)
)

tinytable::tt(compare_sub, digits = 3, escape = TRUE)
```

The two sublethal pipelines yield consistent estimates of z and $CT_{max_{1hr}}$ on this dataset (medians within their mutual credible intervals; see Table S17). The 4PL has the practical advantage that the same `extract_tdt()` workflow used on the lethal counts also produces a posterior TDT curve and $CT_{max_{1hr}}$ — quantities the linear time-to-event model does not deliver directly. T_{crit}, however, should be read from the lethal fit, not the sublethal one (Section 2.0.4).

3.8.5 A caution on cross-endpoint comparison of T_{crit}

If T_{crit} is derived from both the lethal and the sublethal fit on this dataset, the sublethal value sits ~3.5 °C above the lethal one and their 95 % credible intervals do not overlap. This is *not* a biological signal — loss of response to touch precedes death — but a mechanical consequence of the rate-multiplier formula $T_{crit} = CT_{max_{1hr}} + z \cdot \log_{10}(r^*/100)$, which inherits a 2.5× sensitivity to z around the centre of the integrated r^* range. $CT_{max_{1hr}}$ is similar across endpoints (~31.4–31.6 °C), but z for the sublethal fit ($\approx$ 1.1 °C) is less than half the lethal value ($\approx$ 2.6 °C); the resulting T_{crit} gap of $\approx 2.5 \times (2.6 - 1.1)$ follows entirely from that. The lethal and sublethal TDT lines therefore cross near $CT_{max_{1hr}}$ and diverge sharply outside the assay window — the steeper sublethal line, extrapolated, predicts times to knockdown longer than times to death, which is biologically impossible. The

rate-multiplier T_{crit} is best read as an **endpoint-conditional operational summary** of where the fitted TDT line crosses a low-rate floor, not as a universal physiological threshold (Faber *et al.* 2026; Jørgensen *et al.* 2021), and lethal and sublethal T_{crit} should not be compared directly.

4 Case Study 2: Zebrafish lethal-TDT across life stages

We now apply the same TLS / TDT workflow to a lethal-TDT dataset on zebrafish (*Danio rerio*) collected across three early life stages — `young_embryos`, `old_embryos`, and `larvae`. Each row records mortality counts across days of post-exposure observation; we collapse those into a single `n_dead` per trial and treat them as proportion-survival data, just like the shrimp panel. The new wrinkle is the **grouping factor** `life_stage`: the same machinery from Case Study 1 can be applied either *separately to each stage* (three independent fits, simple to reason about) or *jointly* (one model with `life_stage` as a covariate on every 4PL sub-parameter, which pools information across stages and lets us read off life-stage contrasts directly from the joint posterior). We work through both, then compare.

4.1 Data preparation

The raw spreadsheet has a wide format: one row per replicate trial, with mortality and hatching counts spread across morning/afternoon columns for each day of observation. We sum those into total `n_dead`, derive `n_surv`, filter excluded rows, and centre temperature on the grand mean so all three life-stage fits use the same `temp_c`.

```
zf_raw <- readr::read_csv(
  here::here("data", "data_lethal_TDT_zebrafish.csv"),
  show_col_types = FALSE
)

# Sum all daily-mortality columns into a single n_dead per row.
mort_cols <- grep("^Mortality_day_\\d+_(morning|afternoon)$",
                  names(zf_raw), value = TRUE)

zf <- zf_raw |>
  dplyr::filter(Exclude == "no") |>
  dplyr::mutate(
    n_total       = as.integer(N_total),
    n_dead        = as.integer(rowSums(
      dplyr::across(dplyr::all_of(mort_cols), as.numeric), na.rm = TRUE)),
    n_surv        = pmax(n_total - n_dead, 0L),
    assay_temp    = as.numeric(Temperature_assay),
    duration_h    = as.numeric(Duration_exposure_hours),
    life_stage    = factor(Life_stage,
                           levels = c("young_embryos", "old_embryos", "larvae")),
    Date_experiment = factor(Date_experiment)
  ) |>
```

Table S18: Zebrafish lethal-TDT panel summary, by life stage. Each row reports the number of trials, temperature and duration ranges, total individuals assayed, and number of experimental dates contributing to that stage.

life_stage	n trials	Temperature range (°C)	Duration range (hr)	Total individuals	n Dates
young_embryos	118	38.1–42.2	0.02–6	2944	3
old_embryos	106	38.1–42.2	0.08–10	2632	3
larvae	99	38.4–42.3	0.08–6	988	2

```
  dplyr::filter(is.finite(duration_h), duration_h > 0,
                is.finite(assay_temp), n_total > 0)

zf_temp_mean <- mean(zf$assay_temp, na.rm = TRUE)
```

```
zf |>
  dplyr::group_by(life_stage) |>
  dplyr::summarise(
    `n trials`              = dplyr::n(),
    `Temperature range (°C)` = paste(range(assay_temp), collapse = "-"),
    `Duration range (hr)`    = paste(round(range(duration_h), 2), collapse = "-"),
    `Total individuals`      = sum(n_total),
    `n Dates`                = length(unique(Date_experiment)),
    .groups = "drop"
  ) |>
  tinytable::tt(escape = TRUE)
```

We build one standardised dataset per life stage, sharing the same **temp_mean** (the grand mean across the full panel) so that mean-centred temperature is on a common scale across stages. This makes the three separate-fit intercepts comparable and aligns them with the joint model below.

```
zf_stages <- levels(zf$life_stage)

zf_std_by_stage <- lapply(zf_stages, function(s) {
  standardize_data(
    data           = dplyr::filter(zf, life_stage == s),
    temp           = "assay_temp",
    duration       = "duration_h",
    n_total        = "n_total",
    n_surv         = "n_surv",
    random_effects = "Date_experiment",
    duration_unit  = "hours",
    temp_mean      = zf_temp_mean     # shared centring across stages
  )
})
names(zf_std_by_stage) <- zf_stages
```

Table S19: Sampling diagnostics for the three separate zebrafish 4PL fits, one row per life stage. Healthy targets: Rhat < 1.01, ESS > 400, zero divergences.

Life stage	rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	bfmi_min	rhat
young_embryos	1	2867	5351	0	6536	0.799	TRU
old_embryos	1	2752	4669	0	5	0.789	TRU
larvae	1	3206	3278	0	2	0.834	TRU

4.2 Approach 1: A separate 4PL model per life stage

The simplest option is to repeat the shrimp pipeline three times — one `fit_4pl()` per life stage with a random intercept on `mid` for `Date_experiment`, then one `extract_tdt()` per fit. The three fits are independent: each stage's posterior carries no information about the others.

```
models_dir <- here::here("output", "models")

zf_fits_sep <- lapply(zf_stages, function(s) {
  fit_4pl(
    zf_std_by_stage[[s]],
    random_effects = "Date_experiment",
    chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
    control = list(adapt_delta = 0.99, max_treedepth = 15),
    silent  = 2, refresh = 0,
    file    = file.path(models_dir,
                        paste0("fit_zf_separate_", s)),
    file_refit = "on_change"
  )
})
names(zf_fits_sep) <- zf_stages
```

4.2.1 Sampling diagnostics for the separate fits

```
zf_sep_diag <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  diagnose_tdt_fit(zf_fits_sep[[s]]) |>
    dplyr::mutate(`Life stage` = s, .before = 1)
}))
tinytable::tt(zf_sep_diag, digits = 3, escape = TRUE)
```

4.2.2 Survival curves per life stage

```
zf_sep_survplots <- lapply(zf_stages, function(s) {
  psc <- predict_survival_curves(zf_fits_sep[[s]],
                                 temps = sort(unique(zf_std_by_stage[[s]]$temp)),
                                 ndraws = 500)
  plot_survival_curves(psc, observed = zf_std_by_stage[[s]]) +
    ggplot2::coord_cartesian(xlim = c(0, 10)) +
    ggplot2::labs(title = s,
                  x = "Exposure duration (hours)") +
    ggplot2::theme(legend.position = "none")
})

patchwork::wrap_plots(zf_sep_survplots, ncol = 3)
```

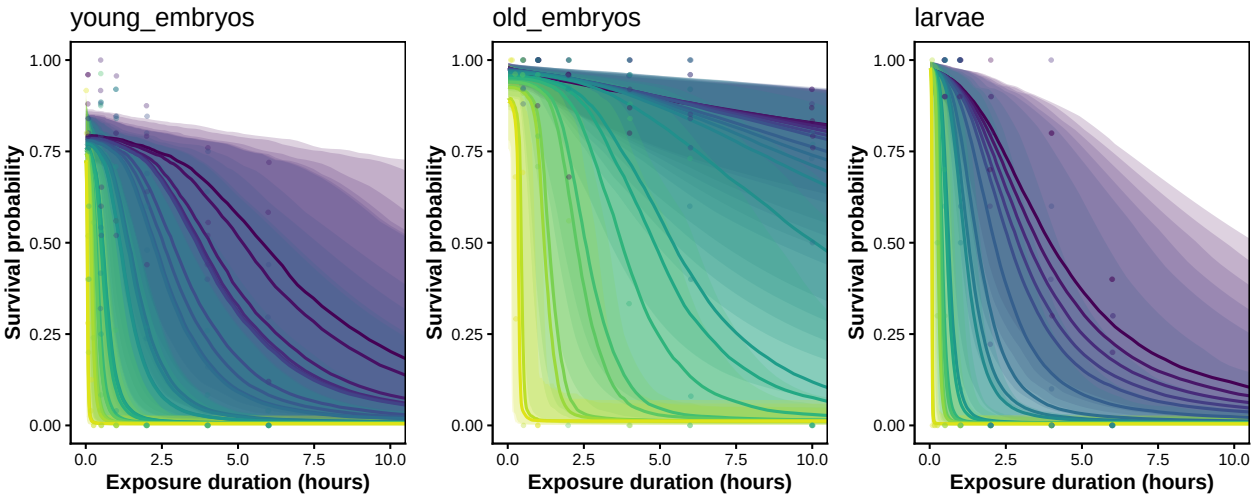


Figure S23: Posterior survival curves for each zebrafish life stage from the three independent 4PL fits, with 95% credible bands. One panel per stage; coloured points are observed per-replicate survival proportions.

4.2.3 TDT landscapes per life stage

```
zf_temp_range <- range(zf$assay_temp, na.rm = TRUE)
zf_dur_range  <- range(zf$duration_h, na.rm = TRUE)
zf_shared_temp_grid <- seq(zf_temp_range[1], zf_temp_range[2],
                           length.out = 120)
zf_shared_dur_grid  <- seq(zf_dur_range[1],  zf_dur_range[2],
                           length.out = 120)

zf_sep_landplots <- lapply(zf_stages, function(s) {
  lsp <- derive_tdt_landscape(zf_fits_sep[[s]],
                              temp_grid     = zf_shared_temp_grid,
                              duration_grid = zf_shared_dur_grid,
```

```
                                ndraws         = 500)
  plot_tdt_landscape(lsp, observed = zf_std_by_stage[[s]]) +
    ggplot2::labs(title = s, y = "Exposure duration (hours)")
})

patchwork::wrap_plots(zf_sep_landplots, ncol = 3) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

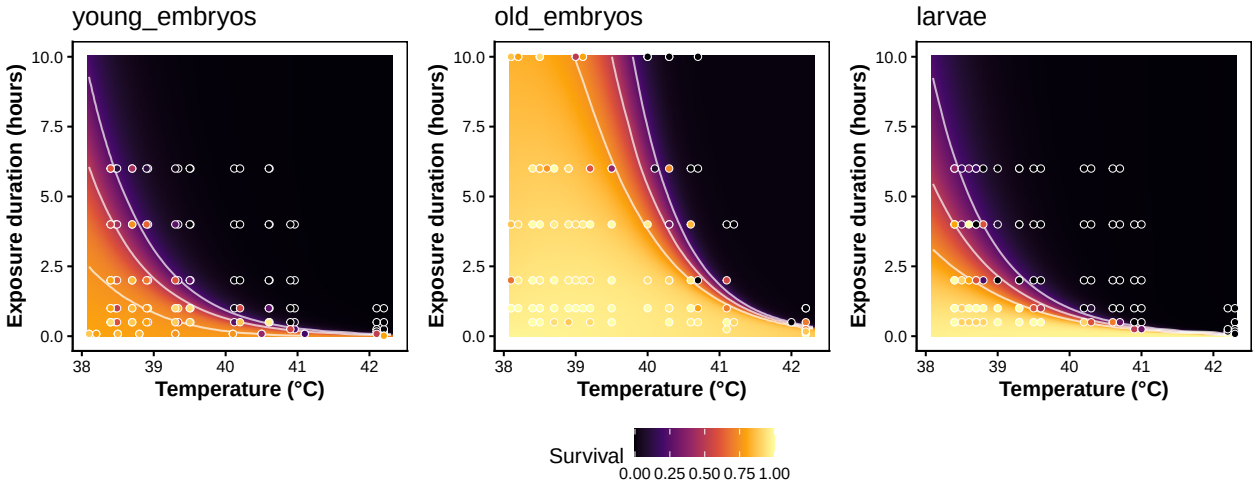


Figure S24: TDT landscape for each zebrafish life stage from the three independent 4PL fits. Heatmap shows posterior median predicted survival across temperature × duration; white contours mark 25, 50, and 75 % survival; points are observed proportions. All three panels share the same axis ranges so the surfaces can be compared directly.

4.2.4 TDT lines from the separate fits

`derive_tdt_curve()` returns the predicted time to the survival threshold at each temperature; computed for each life stage on a common temperature grid, the three lines can be overlaid on the same axes for direct comparison. We use the package-default threshold (`mid`), which coincides with the conventional absolute LT50 when the asymptotes are anchored near 0 and 1 and remains meaningful when intrinsic background mortality pulls the upper asymptote below 1 — as it does for one of the zebrafish life stages (see Section 4.4 for the absolute-0.5 comparison).

```
zf_temp_grid <- seq(min(zf$assay_temp) - 0.25,
                    max(zf$assay_temp) + 0.25, by = 0.05)

zf_sep_ltx <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  ltx <- derive_tdt_curve(
    zf_fits_sep[[s]],
    temp_grid       = zf_temp_grid,
    ndraws          = 500,
    time_multiplier = 60,
```

```
    output_time_unit = "min"
  )
  ltx$summary |>
    dplyr::transmute(
      life_stage   = factor(s, levels = zf_stages),
      assay_temp   = temp,
      time_med_min = duration_median,
      time_lo_min  = duration_lower,
      time_hi_min  = duration_upper
    )
}))
```

```
# Colour-blind-safe triple, distinct in luminance from each other and from
# the panel background; replaces the unreadable viridis yellow for larvae.
zf_palette <- c(young_embryos = "#440154",   # dark purple
                old_embryos   = "#21918c",   # teal
                larvae        = "#f98e09")   # orange (inferno ~0.75)

# Y-axis cap = 48 hours, expressed in the same unit as time_med_min (minutes).
y_cap_min <- 48 * 60

p_lin_sep <- ggplot2::ggplot(zf_sep_ltx,
                             ggplot2::aes(x = assay_temp, y = time_med_min,
                                          colour = life_stage,
                                          fill   = life_stage)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = time_lo_min, ymax = time_hi_min),
                       alpha = 0.18, colour = NA) +
  ggplot2::geom_line(linewidth = 1) +
  ggplot2::scale_colour_manual(values = zf_palette) +
  ggplot2::scale_fill_manual(values   = zf_palette) +
  ggplot2::coord_cartesian(ylim = c(0, y_cap_min)) +
  ggplot2::labs(x = "Assay temperature (°C)",
                y = "Time to 50% survival (min)",
                colour = "Life stage", fill = "Life stage") +
  ggplot2::theme_classic() +
  ggplot2::theme(panel.border = ggplot2::element_rect(colour = "black",
                                                      fill = NA,
                                                      linewidth = 0.6))

p_log_sep <- ggplot2::ggplot(zf_sep_ltx,
                             ggplot2::aes(x = assay_temp, y = time_med_min,
                                          colour = life_stage,
                                          fill   = life_stage)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = time_lo_min, ymax = time_hi_min),
                       alpha = 0.18, colour = NA) +
  ggplot2::geom_line(linewidth = 1) +
  ggplot2::scale_colour_manual(values = zf_palette) +
```

```
  ggplot2::scale_fill_manual(values   = zf_palette) +
  ggplot2::scale_y_log10() +
  ggplot2::coord_cartesian(ylim = c(NA, y_cap_min)) +
  ggplot2::labs(x = "Assay temperature (°C)",
                y = expression(log[10] *
                               " time to 50% survival (min)")) +
  ggplot2::theme_classic() +
  ggplot2::theme(legend.position = "none",
                 panel.border = ggplot2::element_rect(colour = "black",
                                                      fill = NA,
                                                      linewidth = 0.6))

patchwork::wrap_plots(p_lin_sep, p_log_sep, ncol = 2) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

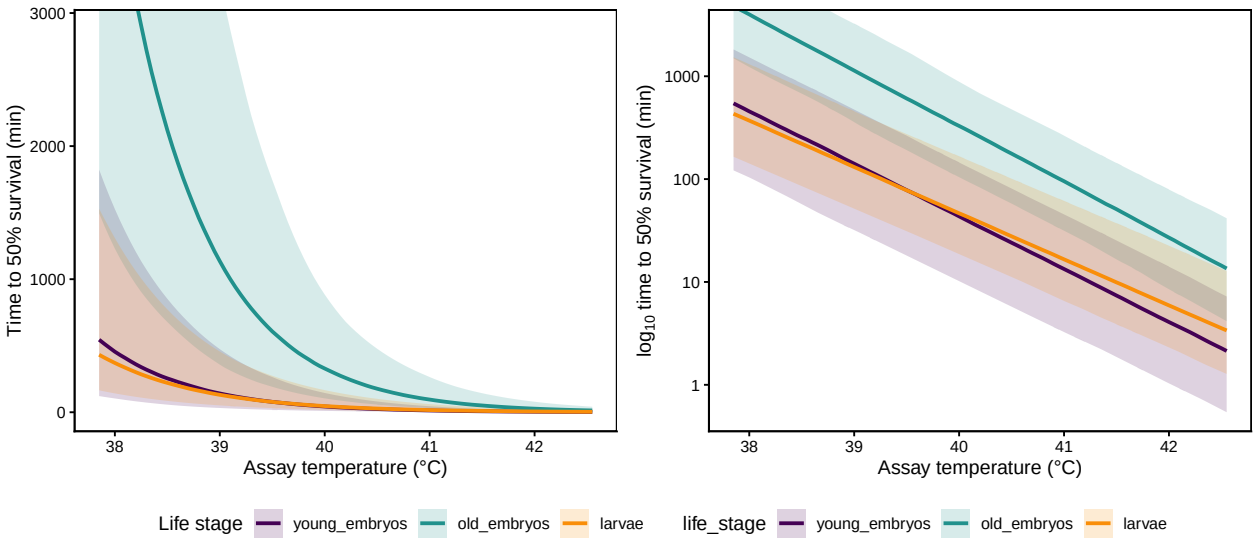


Figure S25: TDT lines for the three zebrafish life stages from the separate 4PL fits, using the package-default threshold. Linear-time panel on the left, $\log_{10}$-time panel on the right (slope is $-1/z$). Lines are posterior medians; shaded bands are 95 % credible intervals.

4.2.5 TDT-quantity posteriors per life stage

`extract_tdt()` summarises z and $CT_{max_{1hr}}$ from each separate fit. We set `lethal = TRUE` because the zebrafish endpoint is mortality, which adds T_{crit} to the output (the absolute-threshold sensitivity check in Section 4.4 reruns the same call with `target_surv = "absolute"`).

```
zf_et_sep <- lapply(zf_stages, function(s) {
  extract_tdt(zf_fits_sep[[s]],
              t_ref         = 60,
              TC_rate_range = c(0.1, 1),
```

Table S20: Posterior summaries of z, $CT_{max_{1hr}}$ and T_{crit} from the three separate zebrafish 4PL fits, under the package-default relative threshold (`(low + up)/2` per draw). Absolute-threshold values are tabulated in Section 4.4. Medians with 95 % credible intervals. T_{crit} is integrated over $r^* \in [0.1, 1]$ %HI per hour, as in Case Study 1.

Life stage	z (°C)	CTmax_1hr (°C)	T_crit (°C)
young_embryos	1.97 [1.77, 2.19]	39.72 [38.56, 40.59]	34.78 [32.87, 36.36]
old_embryos	1.87 [1.58, 2.13]	41.35 [40.36, 42.06]	36.6 [35.1, 37.95]
larvae	2.25 [2.01, 2.42]	39.77 [38.78, 40.78]	34.22 [32.72, 35.73]

```
              ndraws        = 500,
              lethal        = TRUE)
})
names(zf_et_sep) <- zf_stages
```

```
zf_sep_tdt_table <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  et <- zf_et_sep[[s]]
  tibble::tibble(
    `Life stage`        = s,
    `z (°C)`            = format_interval(et$z$summary$z_median,
                                          et$z$summary$z_lower,
                                          et$z$summary$z_upper, digits = 2),
    `CTmax_1hr (°C)`    = format_interval(et$CTmax$summary$temp_median,
                                          et$CTmax$summary$temp_lower,
                                          et$CTmax$summary$temp_upper, digits = 2),
    `T_crit (°C)`       = format_interval(et$T_crit$summary$temp_median,
                                          et$T_crit$summary$temp_lower,
                                          et$T_crit$summary$temp_upper, digits = 2)
  )
}))
tinytable::tt(zf_sep_tdt_table, escape = TRUE)
```

4.2.6 Pairwise life-stage contrasts (separate fits)

Because the three fits are independent, pairwise contrasts $\Delta_{ij} = q^{(i)} - q^{(j)}$ can be computed at the draw level: subtracting matched draws gives a per-draw difference distribution that carries the joint uncertainty in both fits.

```
make_contrast <- function(a, b, var) {
  da <- zf_et_sep[[a]][[var]]$draws
  db <- zf_et_sep[[b]][[var]]$draws
  k <- min(nrow(da), nrow(db))
  draws_a <- if (var == "z") da$z[seq_len(k)] else da$temp[seq_len(k)]
  draws_b <- if (var == "z") db$z[seq_len(k)] else db$temp[seq_len(k)]
```

Table S21: Pairwise differences in z and $CT_{max_{1hr}}$ across zebrafish life stages from the three separate 4PL fits. Positive values mean the first-listed stage has the larger value.

Contrast	Quantity	Median	Lower	Upper
larvae − old_embryos	z (°C)	0.386	0.018	0.717
larvae − old_embryos	CTmax_1hr (°C)	-1.578	-2.732	0.028
larvae − young_embryos	z (°C)	0.281	-0.025	0.549
larvae − young_embryos	CTmax_1hr (°C)	0.083	-1.231	1.753
old_embryos − young_embryos	z (°C)	-0.095	-0.470	0.212
old_embryos − young_embryos	CTmax_1hr (°C)	1.647	0.491	2.965

```
  delta <- draws_a - draws_b
  q <- stats::quantile(delta, c(0.025, 0.5, 0.975), names = FALSE)
  tibble::tibble(
    Contrast = paste(a, "-", b),
    Quantity = switch(var, z = "z (°C)", CTmax = "CTmax_1hr (°C)"),
    Median   = round(q[2], 3),
    Lower    = round(q[1], 3),
    Upper    = round(q[3], 3)
  )
}

pairs <- list(c("larvae", "old_embryos"),
              c("larvae", "young_embryos"),
              c("old_embryos", "young_embryos"))

zf_sep_contrasts <- dplyr::bind_rows(lapply(pairs, function(p) {
  dplyr::bind_rows(
    make_contrast(p[1], p[2], "z"),
    make_contrast(p[1], p[2], "CTmax")
  )
}))

tinytable::tt(zf_sep_contrasts, escape = TRUE)
```

4.3 Approach 2: A joint 4PL with `life_stage` as a covariate

A **joint model** fits all three life stages together and lets every 4PL sub-parameter depend on `life_stage`, with the dispersion ϕ and the date random-intercept variance σ_{date} pooled across stages. The advantages over separate fits are:

- σ_{date} is estimated from all eight experimental dates rather than from 2–3 dates per stage, giving more information about the random-effect scale.

- The priors on the per-stage asymptote and slope coefficients are shared, so each stage's posterior is gently regularised by the others where data are sparse.
- Pairwise life-stage contrasts are linear combinations of draws from a single posterior, with no cross-fit alignment to worry about.

We write the formula by hand to keep the same disjoint-bounds reparameterisation used by `make_4pl_formula()`. The life-stage covariate enters via **cell-means coding** (`~ 0 + life_stage + temp_c:life_stage`) rather than treatment-contrast coding. Cell-means estimates each stage's intercept and slope directly, with the same prior width on every stage's coefficient. Treatment contrasts (`temp_c * life_stage`) would place asymmetric priors on the non-reference stages — the effective slope for a non-reference stage is the sum of two `normal(0, 0.5)` coefficients, with implicit SD $\sqrt{2} \cdot 0.5 \approx 0.71$, $\sqrt{2}\times$ wider than the reference stage. Zebrafish life stages are biologically distinct enough that we do not want one of them designated as a "baseline".

All four sub-parameters (`lowraw`, `upraw`, `logk`, `mid`) receive this cell-means structure. No observation-level random effect is included: the beta-binomial family already absorbs the extra-binomial variance via ϕ.

```
zf_bounds <- bayesTLS:::compute_4pl_bounds(lower = 0, upper = 1)
low_expr  <- sprintf("(%.6f + inv_logit(lowraw) * %.6f)",
                     zf_bounds$low_min, zf_bounds$low_w)
up_expr   <- sprintf("(%.6f + inv_logit(upraw)  * %.6f)",
                     zf_bounds$up_min,  zf_bounds$up_w)
main_rhs  <- sprintf("%s + (%s - %s) / (1 + exp(exp(logk) * (logd - mid)))",
                     low_expr, up_expr, low_expr)

f_zf_joint <- brms::bf(
  stats::as.formula(sprintf("n_surv | trials(n_total) ~ %s", main_rhs)),
  stats::as.formula("lowraw ~ 0 + life_stage + temp_c:life_stage"),
  stats::as.formula("upraw  ~ 0 + life_stage + temp_c:life_stage"),
  stats::as.formula("logk   ~ 0 + life_stage + temp_c:life_stage"),
  stats::as.formula("mid    ~ 0 + life_stage + temp_c:life_stage + (1 | Date_experiment)"),
  nl     = TRUE,
  family = brms::beta_binomial(link = "identity")
)
```

We assemble the standardised per-stage datasets back into a single tibble for the joint fit, keeping the shared mean-centring.

```
zf_joint_data <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  d <- zf_std_by_stage[[s]]
  d$life_stage <- factor(s, levels = zf_stages)
  d
}))
```

Priors mirror those used by `make_4pl_priors()`. Cell-means coding suppresses the global intercept, so each per-stage intercept (`life_stage<stage>`) receives its own prior centred on a plausible value for the corresponding sub-parameter; per-stage slopes pick up the general `class = "b"` priors.

```
low_target  <- 0 + 0.02 * 1
up_target   <- 0 + 0.98 * 1
lowraw_loc  <- stats::qlogis((low_target - zf_bounds$low_min) / zf_bounds$low_w)
upraw_loc   <- stats::qlogis((up_target  - zf_bounds$up_min)  / zf_bounds$up_w)
logk_loc    <- log(2)
mid_loc     <- stats::median(zf_joint_data$logd, na.rm = TRUE)

per_stage_intercept_priors <- function(centre, sigma, nlpar) {
  do.call(c, lapply(zf_stages, function(s) {
    brms::set_prior(
      sprintf("normal(%.6f, %.6f)", centre, sigma),
      class = "b", nlpar = nlpar,
      coef  = paste0("life_stage", s)
    )
  }))
}

zf_joint_priors <- c(
  per_stage_intercept_priors(lowraw_loc, 1.0, "lowraw"),
  per_stage_intercept_priors(upraw_loc,  1.0, "upraw"),
  per_stage_intercept_priors(logk_loc,   1.0, "logk"),
  per_stage_intercept_priors(mid_loc,    1.5, "mid"),
  brms::set_prior("normal(0, 0.5)", class = "b", nlpar = "lowraw"),
  brms::set_prior("normal(0, 0.5)", class = "b", nlpar = "upraw"),
  brms::set_prior("normal(0, 0.3)", class = "b", nlpar = "logk"),
  brms::set_prior("normal(0, 0.6)", class = "b", nlpar = "mid"),
  brms::set_prior("exponential(2)", class = "sd", nlpar = "mid",
                  group = "Date_experiment"),
  brms::set_prior("gamma(2, 0.1)",  class = "phi")
)
```

```
fit_zf_joint <- brms::brm(
  formula    = f_zf_joint,
  data       = zf_joint_data,
  prior      = zf_joint_priors,
  chains     = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  control    = list(adapt_delta = 0.995, max_treedepth = 15),
  init       = 0,
  backend    = "cmdstanr",
  silent     = 2, refresh = 0,
  file       = file.path(models_dir, "fit_zf_joint_4pl"),
  file_refit = "on_change"
)
```

Table S22: Sampling diagnostics for the joint zebrafish 4PL fit (one model, all three life stages). Healthy targets: Rhat < 1.01, ESS > 400, zero divergences.

rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	rhat_pass	ess_pass	divergence_
1	2729	4354	0	10	TRUE	TRUE	TRUE

4.3.1 Sampling diagnostics for the joint fit

```
np_zj      <- brms::nuts_params(fit_zf_joint)
divs_zj    <- sum(subset(np_zj, Parameter == "divergent__")$Value)
td_max_zj  <- max(np_zj$Value[np_zj$Parameter == "treedepth__"], na.rm = TRUE)
td_hits_zj <- sum(subset(np_zj, Parameter == "treedepth__")$Value >= td_max_zj)
rh_zj      <- brms::rhat(fit_zf_joint)
ess_zj     <- posterior::summarise_draws(posterior::as_draws(fit_zf_joint),
                                         "ess_bulk", "ess_tail")
tibble::tibble(
  rhat_max        = round(max(rh_zj,        na.rm = TRUE), 4),
  ess_bulk_min    = round(min(ess_zj$ess_bulk, na.rm = TRUE)),
  ess_tail_min    = round(min(ess_zj$ess_tail, na.rm = TRUE)),
  divergences     = divs_zj,
  treedepth_hits  = td_hits_zj,
  rhat_pass       = max(rh_zj, na.rm = TRUE) < 1.01,
  ess_pass        = min(ess_zj$ess_bulk, ess_zj$ess_tail, na.rm = TRUE) > 400,
  divergence_pass = divs_zj == 0
) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

4.3.2 TDT lines: independent vs joint fit

To compare the two approaches we build a per-stage TDT curve from the joint fit's posterior. The joint model is a hand-coded `brms::bf()` rather than a `bayes_tls` workflow, so we apply the same shortcut `derive_tdt_curve()` uses internally for the default (mid-based) threshold: extract the per-stage `mid` posterior and exponentiate. We also store the asymptote and slope posteriors here because the absolute-threshold sensitivity check in Section 4.4 consumes them.

```
ndraws_joint <- 500

zf_joint_pred_grid <- tidyr::expand_grid(
  assay_temp = zf_temp_grid,
  life_stage = factor(zf_stages, levels = zf_stages)
) |>
  dplyr::mutate(
    temp_c  = assay_temp - zf_temp_mean,
    logd    = 0,
```

```
    n_total = 1L
  )

pp_low <- brms::posterior_linpred(fit_zf_joint, newdata = zf_joint_pred_grid,
                                  nlpar = "lowraw", re_formula = NA,
                                  ndraws = ndraws_joint)
pp_up  <- brms::posterior_linpred(fit_zf_joint, newdata = zf_joint_pred_grid,
                                  nlpar = "upraw",  re_formula = NA,
                                  ndraws = ndraws_joint)
pp_lk  <- brms::posterior_linpred(fit_zf_joint, newdata = zf_joint_pred_grid,
                                  nlpar = "logk",   re_formula = NA,
                                  ndraws = ndraws_joint)
pp_mid <- brms::posterior_linpred(fit_zf_joint, newdata = zf_joint_pred_grid,
                                  nlpar = "mid",    re_formula = NA,
                                  ndraws = ndraws_joint)

low_mat <- zf_bounds$low_min + plogis(pp_low) * zf_bounds$low_w
up_mat  <- zf_bounds$up_min  + plogis(pp_up)  * zf_bounds$up_w
k_mat   <- exp(pp_lk)
mid_mat <- pp_mid

# At the mid-based threshold the exponential term vanishes, so
# log10(t_mid) = mid(T) directly. Convert hours -> minutes for plotting.
t_min <- 60 * 10 ^ mid_mat

zf_joint_summary <- zf_joint_pred_grid |>
  dplyr::mutate(
    time_med_min = apply(t_min, 2, stats::median, na.rm = TRUE),
    time_lo_min  = apply(t_min, 2, stats::quantile, 0.025,
                         na.rm = TRUE, names = FALSE),
    time_hi_min  = apply(t_min, 2, stats::quantile, 0.975,
                         na.rm = TRUE, names = FALSE)
  ) |>
  dplyr::filter(is.finite(time_med_min)) |>
  dplyr::select(assay_temp, life_stage,
                time_med_min, time_lo_min, time_hi_min)
```

```
zf_overlay_df <- dplyr::bind_rows(
  dplyr::mutate(zf_sep_ltx,        method = "Separate fits"),
  dplyr::mutate(zf_joint_summary,  method = "Joint fit")
) |>
  dplyr::mutate(method = factor(method,
                                levels = c("Separate fits", "Joint fit")))

overlay_alphas    <- c("Separate fits" = 0.12, "Joint fit" = 0.32)
overlay_linetypes <- c("Separate fits" = "dashed", "Joint fit" = "solid")
```

```r
build_overlay <- function(y_log = FALSE) {
  p <- ggplot2::ggplot(zf_overlay_df,
                       ggplot2::aes(x = assay_temp, y = time_med_min,
                                    colour = life_stage,
                                    fill   = life_stage,
                                    linetype = method,
                                    group = interaction(life_stage, method))) +
    ggplot2::geom_ribbon(ggplot2::aes(ymin  = time_lo_min,
                                      ymax  = time_hi_min,
                                      alpha = method),
                         colour = NA) +
    ggplot2::geom_line(linewidth = 1) +
    ggplot2::scale_colour_manual(values = zf_palette) +
    ggplot2::scale_fill_manual(values   = zf_palette) +
    ggplot2::scale_alpha_manual(values  = overlay_alphas,
                                name    = "Method") +
    ggplot2::scale_linetype_manual(values = overlay_linetypes,
                                   name   = "Method") +
    ggplot2::labs(x = "Assay temperature (°C)",
                  colour = "Life stage", fill = "Life stage") +
    ggplot2::theme_classic() +
    ggplot2::theme(panel.border = ggplot2::element_rect(colour = "black",
                                                        fill = NA,
                                                        linewidth = 0.6))
  if (y_log) {
    p + ggplot2::scale_y_log10() +
      ggplot2::coord_cartesian(ylim = c(NA, y_cap_min)) +
      ggplot2::labs(y = expression(log[10] *
                                   " time to 50% survival (min)"))
  } else {
    p + ggplot2::coord_cartesian(ylim = c(0, y_cap_min)) +
      ggplot2::labs(y = "Time to 50% survival (min)")
  }
}

patchwork::wrap_plots(build_overlay(FALSE), build_overlay(TRUE), ncol = 2) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

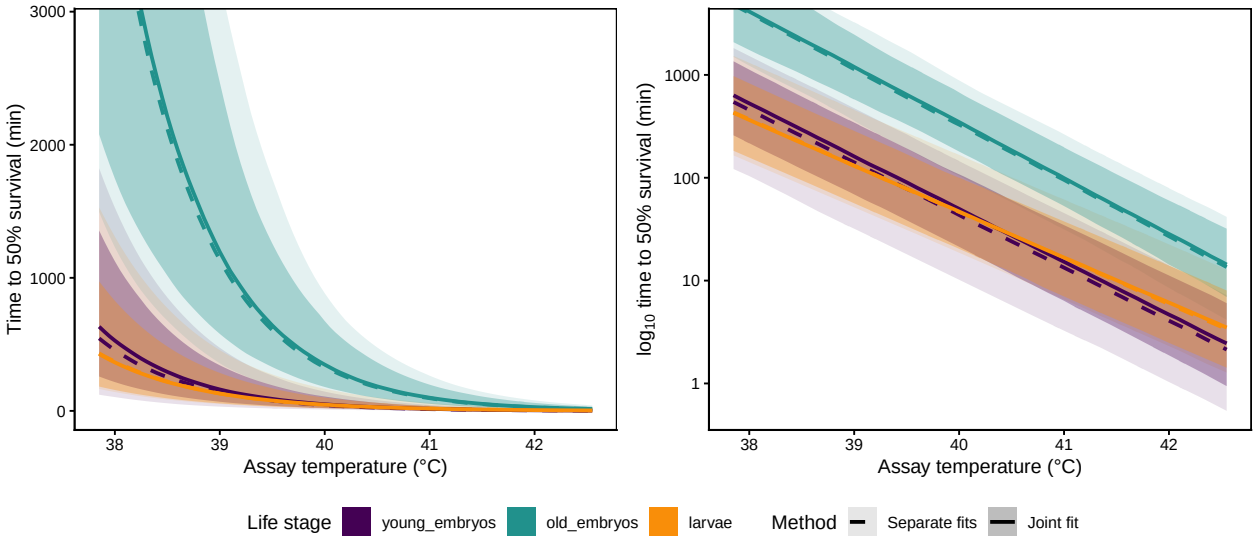


Figure S26: TDT lines for zebrafish comparing the three independent per-life-stage fits (dashed, lighter ribbons) with the joint 4PL fit (solid, darker ribbons); 95 % credible bands shown for both. Linear-time panel on the left, $\log_{10}$-time panel on the right. Same hue per life stage across methods. The joint-fit bands are typically narrower thanks to partial pooling of the priors and the shared ϕ. See Section 4.4 for the same overlay under the absolute 0.5 threshold.

4.3.3 TDT quantities from the joint fit

We can reuse `derive_tdt_parameters()` per life stage by feeding it the per-stage relative-LT line we just built. The result is a posterior of z and $CT_{max_{1hr}}$ per life stage drawn from the *joint* fit's posterior, with all life-stage covariation correctly propagated. Under the relative threshold (`(low + up)/2`), $\log_{10} \mathrm{LT}(T) = \mathrm{mid}(T)$, so z is read directly from the slope posterior and $CT_{max_{1hr}}$ comes from per-draw OLS of $\mathrm{mid}(T)$ on T across the assay grid. Section Section 4.4 reproduces the same quantities under the *absolute* 0.5 threshold for comparison.

```
TC_rate_range_zf <- c(0.1, 1)
log10_low_zf     <- log10(TC_rate_range_zf[1] / 100)
log10_high_zf    <- log10(TC_rate_range_zf[2] / 100)

zf_joint_tdt <- lapply(zf_stages, function(s) {
  idx <- zf_joint_pred_grid$life_stage == s
  draws <- as.data.frame(t(t_min[, idx, drop = FALSE]))
  colnames(draws) <- paste0("V", seq_len(ncol(draws)))
  draws$temp        <- zf_joint_pred_grid$assay_temp[idx]
  draws$target_surv <- "(low+up)/2"
  long <- tidyr::pivot_longer(draws,
                              cols = -c(temp, target_surv),
                              names_to = ".draw_id",
                              values_to = "duration_out") |>
    dplyr::mutate(.draw = as.integer(sub("V", "", .draw_id)),
```

```
                  duration_model = duration_out / 60) |>
    dplyr::filter(is.finite(duration_out), duration_out > 0) |>
    dplyr::select(.draw, temp, target_surv, duration_model, duration_out)

  ltx_obj <- list(draws = long, target_surv = "(low+up)/2",
                  time_multiplier = 60, output_time_unit = "min")
  zct  <- derive_tdt_parameters(ltx_obj, t_ref = 60)

  per_draw <- zct$draws |>
    dplyr::select(.draw, z, CTmax) |>
    dplyr::filter(is.finite(z), is.finite(CTmax)) |>
    dplyr::mutate(log10_rate = stats::runif(dplyr::n(),
                                            min = log10_low_zf,
                                            max = log10_high_zf),
                  T_crit     = CTmax + z * log10_rate)
  q_tc <- stats::quantile(per_draw$T_crit,
                          c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
  T_crit_block <- list(
    draws   = tibble::tibble(.draw = per_draw$.draw,
                             temp  = per_draw$T_crit,
                             log10_rate = per_draw$log10_rate),
    summary = tibble::tibble(TC_rate_low  = TC_rate_range_zf[1],
                             TC_rate_high = TC_rate_range_zf[2],
                             temp_lower   = q_tc[1],
                             temp_median  = q_tc[2],
                             temp_upper   = q_tc[3])
  )

  list(z = zct, summary = zct$summary, T_crit = T_crit_block)
})
names(zf_joint_tdt) <- zf_stages
```

```
zf_joint_table <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  z_summary  <- zf_joint_tdt[[s]]$z$summary
  tc_summary <- zf_joint_tdt[[s]]$T_crit$summary
  tibble::tibble(
    `Life stage`     = s,
    `z (°C)`         = format_interval(z_summary$z_median,
                                       z_summary$z_lower,
                                       z_summary$z_upper, digits = 2),
    `CTmax_1hr (°C)` = format_interval(z_summary$CTmax_median,
                                       z_summary$CTmax_lower,
                                       z_summary$CTmax_upper, digits = 2),
    `T_crit (°C)`    = format_interval(tc_summary$temp_median,
                                       tc_summary$temp_lower,
                                       tc_summary$temp_upper, digits = 2)
  )
```

Table S23: Posterior summaries of z, $CT_{max_{1hr}}$, and T_{crit} from the joint 4PL fit, per life stage. Medians with 95 % credible intervals. T_{crit} uses the same rate-multiplier integration ($r^* \in [0.1, 1]$ % HI per hour) as the shrimp case study. See Section 4.4 for the same quantities under the absolute 0.5 threshold.

Life stage	z (°C)	CTmax_1hr (°C)	T_crit (°C)
young_embryos	1.95 [1.73, 2.19]	39.85 [39.12, 40.51]	34.92 [33.51, 36.18]
old_embryos	1.86 [1.57, 2.15]	41.4 [40.9, 42]	36.8 [35.37, 37.95]
larvae	2.25 [2.08, 2.42]	39.76 [38.94, 40.53]	34.04 [32.56, 35.58]

```
}))
tinytable::tt(zf_joint_table, escape = TRUE)
```

4.3.4 Comparison of the two approaches

```
zf_compare <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  et   <- zf_et_sep[[s]]
  jt   <- zf_joint_tdt[[s]]$z$summary
  tibble::tibble(
    `Life stage`            = s,
    `Separate z (°C)`       = format_interval(et$z$summary$z_median,
                                              et$z$summary$z_lower,
                                              et$z$summary$z_upper, digits = 2),
    `Joint z (°C)`          = format_interval(jt$z_median,
                                              jt$z_lower,
                                              jt$z_upper, digits = 2),
    `Separate CTmax (°C)`   = format_interval(et$CTmax$summary$temp_median,
                                              et$CTmax$summary$temp_lower,
                                              et$CTmax$summary$temp_upper,
                                              digits = 2),
    `Joint CTmax (°C)`      = format_interval(jt$CTmax_median,
                                              jt$CTmax_lower,
                                              jt$CTmax_upper, digits = 2)
  )
}))
tinytable::tt(zf_compare, escape = TRUE)
```

Figure S27 visualises the same comparison at the posterior level: one row per life stage, one column per TDT quantity, with separate-fit and joint-fit posterior densities overlaid. The joint posteriors are typically narrower than the separate ones, while medians agree closely — partial pooling makes the joint fit more precise without changing the location of each quantity.

Table S24: Side-by-side comparison of z and $CT_{max_{1hr}}$ from the three separate fits (Approach 1) and the joint 4PL (Approach 2). The joint fit's credible intervals are typically narrower thanks to partial pooling, while medians are similar — the two approaches agree on the direction of life-stage effects.

Life stage	Separate z (°C)	Joint z (°C)	Separate CTmax (°C)	Joint CTmax (°C)
young_embryos	1.97 [1.77, 2.19]	1.95 [1.73, 2.19]	39.72 [38.56, 40.59]	39.85 [39.12, 40.51]
old_embryos	1.87 [1.58, 2.13]	1.86 [1.57, 2.15]	41.35 [40.36, 42.06]	41.4 [40.9, 42]
larvae	2.25 [2.01, 2.42]	2.25 [2.08, 2.42]	39.77 [38.78, 40.78]	39.76 [38.94, 40.53]

```
zf_post_long <- dplyr::bind_rows(
  lapply(zf_stages, function(s) {
    dplyr::bind_rows(
      tibble::tibble(life_stage = s, quantity = "z",         method = "Separate",
                     value = zf_et_sep[[s]]$z$draws$z),
      tibble::tibble(life_stage = s, quantity = "CTmax_1hr", method = "Separate",
                     value = zf_et_sep[[s]]$CTmax$draws$temp),
      tibble::tibble(life_stage = s, quantity = "T_crit",    method = "Separate",
                     value = zf_et_sep[[s]]$T_crit$draws$temp),
      tibble::tibble(life_stage = s, quantity = "z",         method = "Joint",
                     value = zf_joint_tdt[[s]]$z$draws$z),
      tibble::tibble(life_stage = s, quantity = "CTmax_1hr", method = "Joint",
                     value = zf_joint_tdt[[s]]$z$draws$CTmax),
      tibble::tibble(life_stage = s, quantity = "T_crit",    method = "Joint",
                     value = zf_joint_tdt[[s]]$T_crit$draws$temp)
    )
  })
) |>
  dplyr::filter(is.finite(value)) |>
  dplyr::mutate(
    life_stage = factor(life_stage, levels = zf_stages),
    quantity   = factor(quantity,   levels = c("z", "CTmax_1hr", "T_crit")),
    method     = factor(method,     levels = c("Separate", "Joint"))
  )

zf_post_summary <- zf_post_long |>
  dplyr::group_by(life_stage, quantity, method) |>
  dplyr::summarise(
    med = stats::median(value, na.rm = TRUE),
    lo  = stats::quantile(value, 0.025, names = FALSE, na.rm = TRUE),
    hi  = stats::quantile(value, 0.975, names = FALSE, na.rm = TRUE),
    .groups = "drop"
  )

method_colours <- c("Separate" = "#2c7fb8", "Joint" = "#d95f02")
```

```
quantity_labels <- ggplot2::as_labeller(
  c(z         = "italic(z)~(degree*C~per~log[10]~time)",
    CTmax_1hr = "CT[max[1*hr]]~(degree*C)",
    T_crit    = "T[crit]~(degree*C)"),
  default = ggplot2::label_parsed
)

ggplot2::ggplot(zf_post_long,
                ggplot2::aes(x = value, fill = method, colour = method)) +
  ggplot2::geom_density(alpha = 0.35, linewidth = 0.4) +
  ggplot2::geom_vline(data = zf_post_summary,
                      ggplot2::aes(xintercept = med, colour = method),
                      linewidth = 0.7, show.legend = FALSE) +
  ggplot2::geom_vline(data = zf_post_summary,
                      ggplot2::aes(xintercept = lo, colour = method),
                      linetype = "dashed", linewidth = 0.4, alpha = 0.8,
                      show.legend = FALSE) +
  ggplot2::geom_vline(data = zf_post_summary,
                      ggplot2::aes(xintercept = hi, colour = method),
                      linetype = "dashed", linewidth = 0.4, alpha = 0.8,
                      show.legend = FALSE) +
  ggplot2::scale_colour_manual(values = method_colours, name = "Method") +
  ggplot2::scale_fill_manual(values   = method_colours, name = "Method") +
  ggplot2::facet_grid(life_stage ~ quantity, scales = "free",
                      labeller = ggplot2::labeller(
                        quantity   = quantity_labels,
                        life_stage = ggplot2::label_value
                      )) +
  ggplot2::labs(x = NULL, y = "Posterior density") +
  ggplot2::theme_classic(base_size = 11) +
  ggplot2::theme(
    legend.position  = "top",
    strip.background = ggplot2::element_blank(),
    panel.border     = ggplot2::element_rect(colour = "black",
                                             fill = NA, linewidth = 0.5)
  )
```

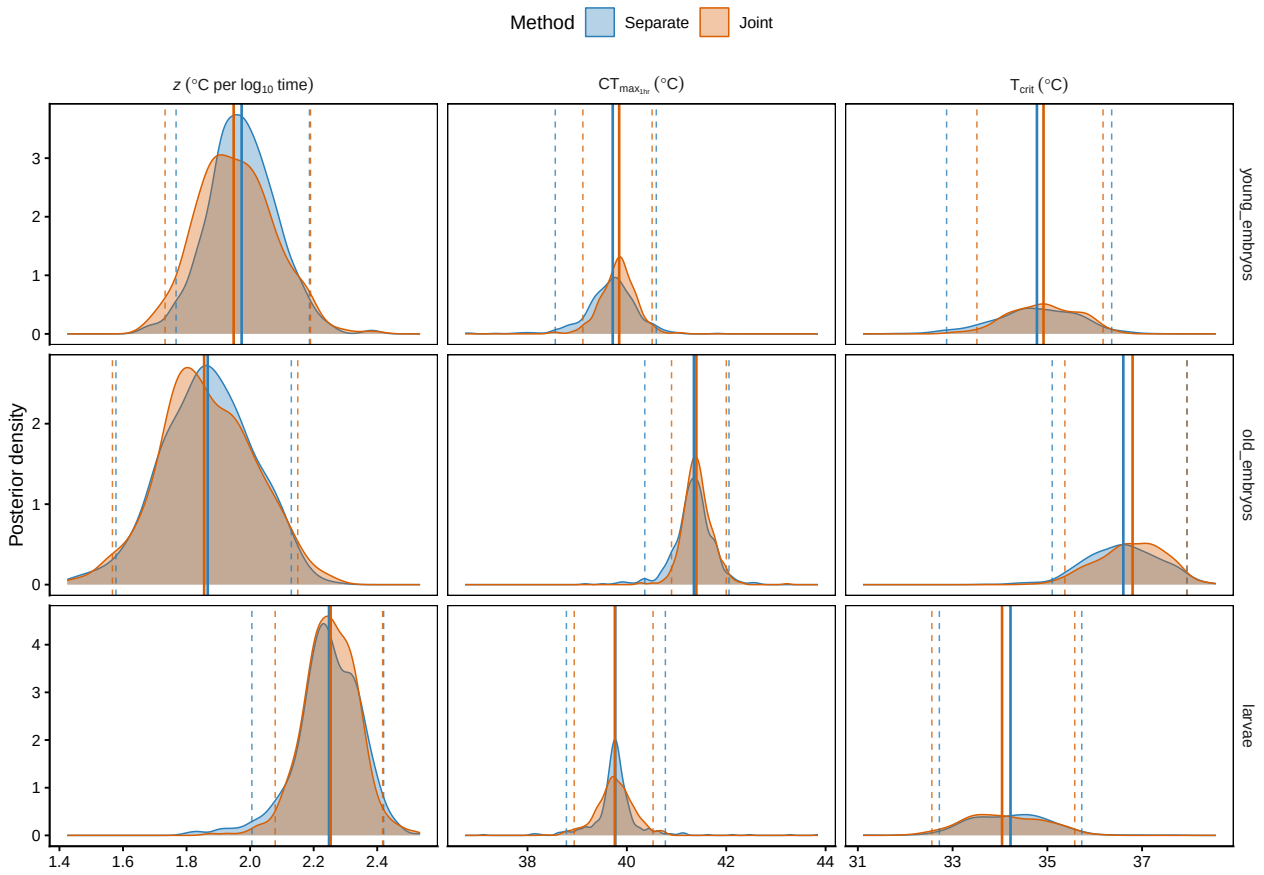


Figure S27: Posterior densities of z (left), $CT_{max_{1hr}}$ (middle), and T_{crit} (right) for each zebrafish life stage, separate-fit (blue) vs joint-fit (orange). Solid vertical lines mark the posterior median; dashed lines mark the 95 % credible interval.

The joint approach is the recommended default when the dataset spans an identifiable grouping factor (life stage here, but it could equally be population, sex, or species). It is the natural extension of the workflow in §[Joint Bayesian 4PL] from a single 4PL surface to a *family* of 4PL surfaces tied together by shared priors and (where the design allows) shared random-effect grouping.

4.4 Sensitivity check: an absolute survival threshold

All TDT curves and $CT_{max_{1hr}}$ posteriors above use the package-default threshold (the per-draw `mid` parameter, the midpoint between the fitted asymptotes). This coincides with the classical LT50 whenever the asymptotes are anchored near 0 and 1; it diverges from absolute LT50 when intrinsic background mortality pulls the upper asymptote below 1. For operational questions framed as "no more than half the cohort survives", an absolute 50 % survival threshold is the right anchor. Pass `target_surv = "absolute"` to `extract_tdt()` or `derive_tdt_curve()` (or any numeric in $(0, 1)$ for other percentiles) to swap the threshold without refitting. This section reruns the zebrafish analyses at the absolute threshold and compares the result to the default.

The joint fit makes the contrast concrete: the three life stages have very different upper asymptotes.

Table S25: Posterior summary of the 4PL upper asymptote α_{up} at the grand-mean temperature, per life stage, from the joint fit. Medians with 95 % credible intervals on the natural survival scale. Values markedly below 1 indicate that a stage cannot reach saturating survival even at near-zero exposure, which biases absolute-LT50 comparisons across stages.

Life stage	Upper asymptote (raw)	Upper asymptote (probability)
young_embryos	0.1 [-0.53, 0.86]	0.763 [0.686, 0.851]
old_embryos	2.48 [1.87, 3.34]	0.961 [0.932, 0.982]
larvae	2.94 [1.75, 4.58]	0.974 [0.925, 0.994]

```
zf_joint_post <- brms::as_draws_df(fit_zf_joint)

zf_up_intercepts <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  raw <- zf_joint_post[[paste0("b_upraw_life_stage", s)]]
  asym <- zf_bounds$up_min + plogis(raw) * zf_bounds$up_w
  tibble::tibble(
    `Life stage`             = s,
    `Upper asymptote (raw)`        = format_interval(stats::median(raw),
                                                     stats::quantile(raw, 0.025,
                                                                     names = FALSE),
                                                     stats::quantile(raw, 0.975,
                                                                     names = FALSE),
                                                     digits = 2),
    `Upper asymptote (probability)` =
      format_interval(stats::median(asym),
                      stats::quantile(asym, 0.025, names = FALSE),
                      stats::quantile(asym, 0.975, names = FALSE),
                      digits = 3)
  )
}))
tinytable::tt(zf_up_intercepts, escape = TRUE)
```

The young-embryo upper asymptote, ~0.76 at the grand-mean temperature, does not overlap the near-saturating values for the other two stages. Biologically, this means substantial intrinsic mortality unrelated to heat stress in that life stage; statistically, it means absolute 0.5 sits proportionally lower on the young-embryo survival curve than on the others. The mapping below makes the difference visible.

4.4.1 Recomputing the per-stage quantities with `target_surv = "absolute"`

```
zf_et_sep_abs <- lapply(zf_stages, function(s) {
  extract_tdt(zf_fits_sep[[s]],
              target_surv   = "absolute",   # = 0.5
```

Table S26: Posterior summaries of z, $CT_{max_{1hr}}$ and T_{crit} from the three separate zebrafish 4PL fits, under the **absolute 50 % survival** threshold (`target_surv = "absolute"`). Medians with 95 % credible intervals. Compare row-by-row with Table S20 (relative threshold).

Life stage	z (°C)	CTmax_1hr (°C)	T_crit (°C)
young_embryos	1.97 [1.72, 2.23]	39.59 [38.41, 40.52]	34.66 [33.02, 36.1]
old_embryos	1.86 [1.58, 2.15]	41.38 [40.46, 42.15]	36.75 [35.1, 38.1]
larvae	2.25 [2.03, 2.45]	39.77 [38.56, 40.81]	34.15 [32.48, 35.53]

```
              t_ref         = 60,
              TC_rate_range = c(0.1, 1),
              ndraws        = 500,
              lethal        = TRUE)
})
names(zf_et_sep_abs) <- zf_stages
```

```
zf_sep_tdt_table_abs <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  et <- zf_et_sep_abs[[s]]
  tibble::tibble(
    `Life stage`        = s,
    `z (°C)`            = format_interval(et$z$summary$z_median,
                                          et$z$summary$z_lower,
                                          et$z$summary$z_upper, digits = 2),
    `CTmax_1hr (°C)`    = format_interval(et$CTmax$summary$temp_median,
                                          et$CTmax$summary$temp_lower,
                                          et$CTmax$summary$temp_upper, digits = 2),
    `T_crit (°C)`       = format_interval(et$T_crit$summary$temp_median,
                                          et$T_crit$summary$temp_lower,
                                          et$T_crit$summary$temp_upper, digits = 2)
  )
}))
tinytable::tt(zf_sep_tdt_table_abs, escape = TRUE)
```

4.4.2 Joint-fit absolute-threshold posterior

The joint fit was hand-coded from `brms::bf()` and lives outside the `bayes_tls` workflow class, so we cannot point `extract_tdt()` at it directly. We reuse the posterior matrices `up_mat`, `low_mat`, `k_mat`, `mid_mat` built in Section 4.3.2 and apply the analytical inverse of the 4PL at survival = 0.5 per draw:

$$\log_{10} t_{0.5}(T) \;=\; \mathrm{mid}(T) \;+\; \tfrac{1}{k(T)} \log \tfrac{\mathrm{up}(T)-0.5}{0.5-\mathrm{low}(T)}.$$

```
arg_abs <- ifelse(up_mat > 0.5 & low_mat < 0.5,
                  (up_mat - 0.5) / (0.5 - low_mat), NA_real_)
log10_t_h_abs <- mid_mat + log(arg_abs) / k_mat
t_min_abs     <- 60 * 10 ^ log10_t_h_abs

zf_joint_summary_abs <- zf_joint_pred_grid |>
  dplyr::mutate(
    time_med_min = apply(t_min_abs, 2, stats::median, na.rm = TRUE),
    time_lo_min  = apply(t_min_abs, 2, stats::quantile, 0.025,
                         na.rm = TRUE, names = FALSE),
    time_hi_min  = apply(t_min_abs, 2, stats::quantile, 0.975,
                         na.rm = TRUE, names = FALSE)
  ) |>
  dplyr::filter(is.finite(time_med_min)) |>
  dplyr::select(assay_temp, life_stage,
                time_med_min, time_lo_min, time_hi_min)
```

```
zf_threshold_overlay <- dplyr::bind_rows(
  dplyr::mutate(zf_joint_summary,     threshold = "Relative (default)"),
  dplyr::mutate(zf_joint_summary_abs, threshold = "Absolute 50 %")
) |>
  dplyr::mutate(threshold = factor(threshold,
                                   levels = c("Relative (default)",
                                              "Absolute 50 %")))

thr_alphas    <- c("Relative (default)" = 0.32, "Absolute 50 %" = 0.12)
thr_linetypes <- c("Relative (default)" = "solid", "Absolute 50 %" = "dashed")

build_thr_overlay <- function(y_log = FALSE) {
  p <- ggplot2::ggplot(zf_threshold_overlay,
                       ggplot2::aes(x = assay_temp, y = time_med_min,
                                    colour = life_stage,
                                    fill   = life_stage,
                                    linetype = threshold,
                                    group = interaction(life_stage, threshold))) +
    ggplot2::geom_ribbon(ggplot2::aes(ymin  = time_lo_min,
                                      ymax  = time_hi_min,
                                      alpha = threshold),
                         colour = NA) +
    ggplot2::geom_line(linewidth = 1) +
    ggplot2::scale_colour_manual(values = zf_palette) +
    ggplot2::scale_fill_manual(values   = zf_palette) +
    ggplot2::scale_alpha_manual(values  = thr_alphas,    name = "Threshold") +
    ggplot2::scale_linetype_manual(values = thr_linetypes, name = "Threshold") +
    ggplot2::labs(x = "Assay temperature (°C)",
                  colour = "Life stage", fill = "Life stage") +
    ggplot2::theme_classic() +
```

```
    ggplot2::theme(panel.border = ggplot2::element_rect(colour = "black",
                                                        fill = NA,
                                                        linewidth = 0.6))
  if (y_log) {
    p + ggplot2::scale_y_log10() +
      ggplot2::coord_cartesian(ylim = c(NA, y_cap_min)) +
      ggplot2::labs(y = expression(log[10] * " time to threshold (min)"))
  } else {
    p + ggplot2::coord_cartesian(ylim = c(0, y_cap_min)) +
      ggplot2::labs(y = "Time to threshold (min)")
  }
}

patchwork::wrap_plots(build_thr_overlay(FALSE),
                      build_thr_overlay(TRUE), ncol = 2) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

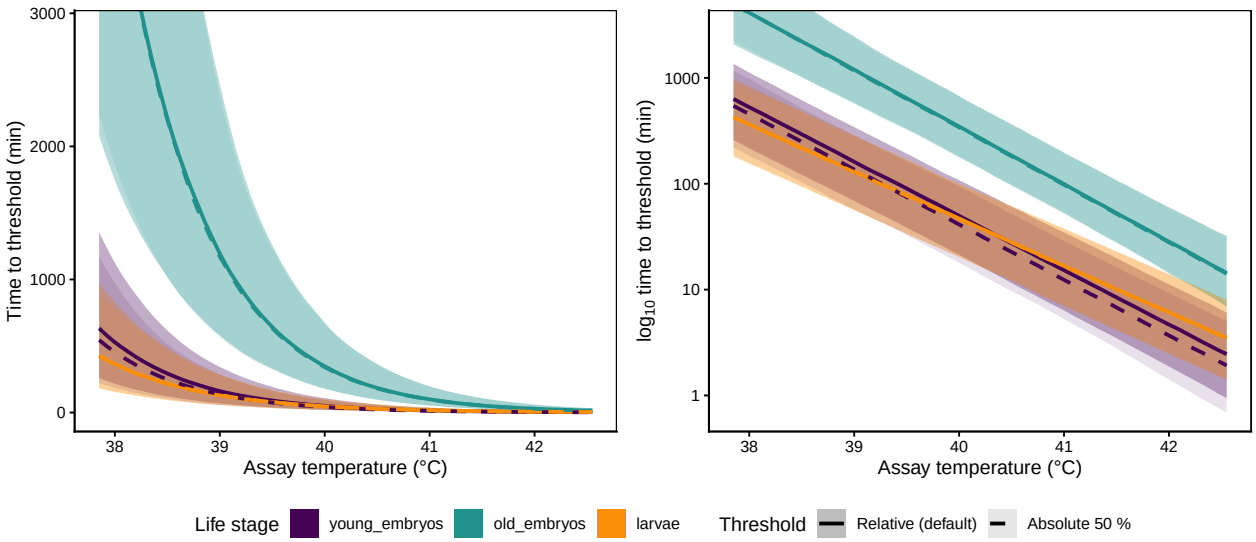


Figure S28: Zebrafish TDT lines from the joint 4PL fit under the two threshold definitions. **Solid, darker ribbons:** package-default mid-based threshold, repeated from Figure S26. **Dashed, lighter ribbons:** absolute 50 % survival. The two definitions are nearly indistinguishable for `old_embryos` and `larvae` (asymptotes close to 1) and diverge for `young_embryos` (asymptote ~0.76, see Table S25), where the absolute target is reached earlier in time.

4.4.3 Side-by-side comparison: relative vs absolute

For the separate fits we can put the two `extract_tdt()` outputs next to each other.

Table S27: Side-by-side comparison of z and $CT_{max_{1hr}}$ from the three separate zebrafish 4PL fits under the two threshold definitions: package-default mid-based and absolute 50 % survival. Absolute-50 % shifts $CT_{max_{1hr}}$ downward for `young_embryos` (whose upper asymptote sits well below 1) but leaves the other two stages essentially unchanged. z is essentially identical under either definition for these data.

Life stage	z (rel, °C)	z (abs, °C)	CTmax_1hr (rel, °C)	CTmax_1hr (abs, °C)
young_embryos	1.97 [1.77, 2.19]	1.97 [1.72, 2.23]	39.72 [38.56, 40.59]	39.59 [38.41, 40.52]
old_embryos	1.87 [1.58, 2.13]	1.86 [1.58, 2.15]	41.35 [40.36, 42.06]	41.38 [40.46, 42.15]
larvae	2.25 [2.01, 2.42]	2.25 [2.03, 2.45]	39.77 [38.78, 40.78]	39.77 [38.56, 40.81]

```
zf_threshold_compare <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  rel_s <- zf_et_sep[[s]]
  abs_s <- zf_et_sep_abs[[s]]
  tibble::tibble(
    `Life stage`           = s,
    `z (rel, °C)`          = format_interval(rel_s$z$summary$z_median,
                                             rel_s$z$summary$z_lower,
                                             rel_s$z$summary$z_upper,
                                             digits = 2),
    `z (abs, °C)`          = format_interval(abs_s$z$summary$z_median,
                                             abs_s$z$summary$z_lower,
                                             abs_s$z$summary$z_upper,
                                             digits = 2),
    `CTmax_1hr (rel, °C)`  = format_interval(rel_s$CTmax$summary$temp_median,
                                             rel_s$CTmax$summary$temp_lower,
                                             rel_s$CTmax$summary$temp_upper,
                                             digits = 2),
    `CTmax_1hr (abs, °C)`  = format_interval(abs_s$CTmax$summary$temp_median,
                                             abs_s$CTmax$summary$temp_lower,
                                             abs_s$CTmax$summary$temp_upper,
                                             digits = 2)
  )
}))
tinytable::tt(zf_threshold_compare, escape = TRUE)
```

The pattern in Table S27 matches the upper-asymptote summary in Table S25: stages whose asymptote is near 1 (`old_embryos`, `larvae`) give essentially identical z and $CT_{max_{1hr}}$ under either threshold, while the stage with a substantially sub-unit asymptote (`young_embryos`) sees $CT_{max_{1hr}}$ shift by a few tenths of a °C when the threshold is switched. The mid-based default is the right choice for cross-group comparisons because it isolates the dose-response shape from the group-level ceiling; the absolute threshold is the right choice for operational questions anchored to a fixed mortality fraction.

5 References

Arnold, P.A., Noble, D.W.A., Nicotra, A.B., Kearney, M.R., Rezende, E.L., Andrew, S.C., *et al.* (2025). A framework for modelling thermal load sensitivity across life. *Global Change Biology*, 31, e70315.

Faber, A.H., Møller, F.D., Ørsted, M., Ehlers, B.K. & Overgaard, J. (2026). Separating good from bad – a methodological assessment of the critical temperature that separates stressful and permissive temperatures in ectotherms.

Jørgensen, L.B., Malte, H. & Overgaard, J. (2021). A unifying model to estimate thermal tolerance limits in ectotherms across static, dynamic and fluctuating exposures to thermal stress. *Scientific Reports*, 11, 12840.

Noble, D.W.A., Mayfield, M.M., Hoffmann, A.A., Chen, Z.-H., Lade, S.J., Bai, X., *et al.* (2026). A systems modelling approach to predict biological responses to extreme heat. *Trends in Ecology & Evolution*, 41, 451–464.

Ørsted, M., Jørgensen, L.B. & Overgaard, J. (2022). Finding the right thermal limit: A framework to reconcile ecological, physiological and methodological aspects of CTmax in ectotherms. *Journal of Experimental Biology*, 225, jeb244514.

Ørsted, M., Willot, Q., Olsen, A.K., Kongsgaard, V. & Overgaard, J. (2024). Thermal limits of survival and reproduction depend on stress duration: A case study of Drosophila suzukii. *Ecology Letters*, 27, e14421.

Rezende, E.L., Bozinovic, F., Szilágyi, A. & Santos, M. (2020). Predicting temperature mortality and selection in natural Drosophila populations. *Science*, 369, 1242–1245.

Rezende, E.L., Castañeda, L.E. & Santos, M. (2014). Tolerance landscapes in thermal ecology. *Functional Ecology*, 28, 799–809.

6 Session Information

```
pander::pander(sessionInfo())
```

R version 4.5.3 (2026-03-11)

Platform: aarch64-apple-darwin20

locale: C.UTF-8||C.UTF-8||C.UTF-8||C||C.UTF-8||C.UTF-8

attached base packages: *stats*, *graphics*, *grDevices*, *utils*, *datasets*, *methods* and *base*

other attached packages: *bayesTLS(v.0.1.0)*, *readxl(v.1.4.5)*, *pander(v.0.6.6)*, *patchwork(v.1.3.2)*, *tinytable(v.0.16.0)*, *tidybayes(v.3.0.7)*, *posterior(v.1.7.0)*, *brms(v.2.23.0)*, *Rcpp(v.1.1.1-1.1)*, *ggplot2(v.4.0.3)*, *purrr(v.1.2.2)*, *tibble(v.3.3.1)*, *tidyr(v.1.3.2)*, *dplyr(v.1.2.1)* and *here(v.1.0.1)*

loaded via a namespace (and not attached): *gridExtra(v.2.3)*, *inline(v.0.3.21)*, *sandwich(v.3.1-1)*, *rlang(v.1.2.0)*, *magrittr(v.2.0.5)*, *multcomp(v.1.4-28)*, *otel(v.0.2.0)*, *matrixStats(v.1.5.0)*, *compiler(v.4.5.3)*, *mgcv(v.1.9-4)*, *loo(v.2.9.0.9000)*, *vctrs(v.0.7.3)*, *reshape2(v.1.4.5)*, *stringr(v.1.6.0)*, *pkgconfig(v.2.0.3)*, *arrayhelpers(v.1.1-0)*, *crayon(v.1.5.3)*, *fastmap(v.1.2.0)*, *backports(v.1.5.1)*, *labeling(v.0.4.3)*, *cmdstanr(v.0.9.0)*, *rmarkdown(v.2.31)*, *tzdb(v.0.5.0)*, *tinytex(v.0.59)*, *bit(v.4.6.0)*, *xfun(v.0.57)*, *litedown(v.0.7)*, *jsonlite(v.2.0.0)*, *parallel(v.4.5.3)*, *R6(v.2.6.1)*, *stringi(v.1.8.7)*,

RColorBrewer(v.1.1-3), *StanHeaders(v.2.36.0.9000)*, *lubridate(v.1.9.5)*, *cellranger(v.1.1.0)*, *estimability(v.1.5.1)*, *rstan(v.2.36.0.9000)*, *knitr(v.1.51)*, *zoo(v.1.8-14)*, *pacman(v.0.5.1)*, *readr(v.2.2.0)*, *bayesplot(v.1.15.0.9000)*, *Matrix(v.1.7-4)*, *splines(v.4.5.3)*, *timechange(v.0.4.0)*, *tidyselect(v.1.2.1)*, *abind(v.1.4-8)*, *yaml(v.2.3.12)*, *codetools(v.0.2-20)*, *curl(v.7.1.0)*, *processx(v.3.9.0)*, *pkgbuild(v.1.4.8)*, *lattice(v.0.22-9)*, *plyr(v.1.8.9)*, *withr(v.3.0.2)*, *bridgesampling(v.1.2-1)*, *S7(v.0.2.2)*, *coda(v.0.19-4.1)*, *evaluate(v.1.0.5)*, *survival(v.3.8-6)*, *RcppParallel(v.5.1.11-2)*, *isoband(v.0.3.0)*, *ggdist(v.3.3.3)*, *pillar(v.1.11.1)*, *tensorA(v.0.36.2.1)*, *checkmate(v.2.3.4)*, *stats4(v.4.5.3)*, *distributional(v.0.7.0)*, *generics(v.0.1.4)*, *vroom(v.1.7.1)*, *rprojroot(v.2.1.1)*, *hms(v.1.1.4)*, *rstantools(v.2.6.0.9000)*, *scales(v.1.4.0)*, *xtable(v.1.8-8)*, *glue(v.1.8.1)*, *emmeans(v.2.0.3)*, *tools(v.4.5.3)*, *mvtnorm(v.1.3-7)*, *grid(v.4.5.3)*, *QuickJSR(v.1.9.2)*, *nlme(v.3.1-168)*, *cli(v.3.6.6)*, *svUnit(v.1.0.8)*, *viridisLite(v.0.4.3)*, *Brobdingnag(v.1.2-9)*, *V8(v.6.0.4)*, *gtable(v.0.3.6)*, *digest(v.0.6.39)*, *TH.data(v.1.1-3)*, *farver(v.2.1.2)*, *htmltools(v.0.5.9)*, *lifecycle(v.1.0.5)*, *bit64(v.4.8.0)* and *MASS(v.7.3-65)*

:::